Desarrollo Web · Proyecto Final · Universidad Mariano Gálvez de Guatemala

Mi Red Social

Documentación completa del proyecto
Aplicación web social para almacenar y navegar en fotografías

Integrantes del grupo

Nombre	Carné
Karla Gabriela Mendoza Bravo	5090-23-15243
Katherine Melissa Palacios Salguero	5090-23-682
Daniel Eduardo Samayoa	5090-23-9260
Andrea Paola Chacón Segura	5090-23-1225
Jeffrie Otoniel Argueta López	5090-23-8396

Catedrático: MSc. Ing. Carlos Alejandro Arias
Semestre: VIII · 2026 **Fecha:** noviembre de 2026
Sitio: https://miredsocialweb.duckdns.org
Nube: AWS, región us-east-2 (Ohio) · EC2 t3.micro · S3 · IAM
Repositorio: GitHub del equipo (ramas por integrante)

Contenido

N.º	Sección	Qué cubre del enunciado	Página
1	Descripción general del proyecto	Integrantes, stack y funcionalidades	03
2	Arquitectura de la solución	Entregable 1 · Documentación del desarrollador	05
3	Diagrama distribuido de componentes	Manual sugerido 2	07
4	Diagramas de Cloud Computing	Manual sugerido 9	08
5	Esquema de la base de datos	Entregable 1 · Documentación del desarrollador	10
6	Manual técnico y de la aplicación web	Entregable 6 · Manuales técnicos	12
7	Manual de usuario	Entregable 3 · Documentación del usuario	15
8	Instrucciones de despliegue	Entregable 2 · Instrucciones de despliegue	16
9	Evidencias del despliegue en AWS	Capturas del despliegue (entregables 2, 4 y 5)	20
10	Seguridad de la base de datos	Manual sugerido 6	42
11	Mantenimiento de la base de datos	Manual sugerido 11	44
12	Plan de recuperación ante desastres	Manuales sugeridos 7 y 8	46
13	Auditoría de transacciones	Manual sugerido 3	49
14	Segmentación de red	Manual sugerido 4	52
15	Plan de capacitaciones para IT	Manual sugerido 12	54
16	Datos de entrega para el catedrático	Entregables 4 y 5 · Credenciales y llave SSH	56

Cada sección empieza en una página nueva. Los datos secretos (clave de acceso del usuario IAM y llave SSH) no forman parte de este documento y se entregan por un canal privado.

Mi Red Social

Proyecto Final — Desarrollo Web (UMG). Aplicación web social para almacenar y navegar en fotografías (estilo Instagram): login tradicional + Google + reCAPTCHA, feed de miniaturas, subida de fotos con transformaciones y reconocimiento facial automáticos, y almacenamiento en la nube (Amazon S3).

Integrantes del grupo — Universidad Mariano Gálvez de Guatemala, curso Desarrollo Web:

Integrante	Carné
Karla Gabriela Mendoza Bravo	5090-23-15243
Katherine Melissa Palacios Salguero	5090-23-682
Daniel Eduardo Samayoa	5090-23-9260
Andrea Paola Chacón Segura	5090-23-1225
Jeffrie Otoniel Argueta López	5090-23-8396

Stack técnico

Capa	Tecnología
Frontend	ReactJS 18 (Vite + React Router) + CSS3
Backend	Node.js + Express
Base de datos	PostgreSQL (relacional) + Sequelize ORM
Autenticación	JWT propio + bcrypt (hash + sal) + Google OAuth2 + reCAPTCHA v2
Procesamiento de imagenes	Sharp (transformaciones) + face-api.js (reconocimiento facial)
Almacenamiento	Disco local (desarrollo) / Amazon S3 (producción)
Despliegue	AWS EC2 (t3.micro, Ohio us-east-2) + PM2 + nginx + DNS DuckDNS + HTTPS (Let's Encrypt)

Documentación

- `docs/ARQUITECTURA.md` — arquitectura general y diagramas.
- `docs/ESQUEMA_BD.md` — esquema de la base de datos relacional.
- `docs/DESPLIEGUE.md` — instrucciones paso a paso para desplegar en AWS.
- `docs/ENTREGA_CATEDRATICO.md` — datos de acceso y pasos para la revisión.
- `docs/README.md` — índice que relaciona cada entregable y manual del enunciado con su archivo.
- `docs/MANUAL_TECNICO.md` — manual técnico: frontend, API REST, variables de entorno y operación.
- `docs/DIAGRAMA_COMPONENTES.md` — diagrama distribuido de componentes.
- `docs/DIAGRAMAS_CLOUD.md` — diagramas de cloud computing (AWS, IAM, flujo de solicitud).
- `docs/PLAN_CAPACITACION_IT.md` — plan de capacitaciones para el equipo de IT.
- `docs/MANUAL_USUARIO.md` — cómo usar la aplicación.
- `docs/SEGURIDAD_BD.md` — manual de seguridad de la base de datos.
- `docs/MANTENIMIENTO_BD.md` — plan de mantenimiento de la base de datos.
- `docs/PLAN_RECUPERACION_DESASTRES.md` — plan de recuperación contra desastres (Disaster Recovery Plan).
- `docs/AUDITORIA_TRANSACCIONES.md` — sistema de auditoría de transacciones.
- `docs/SEGMENTACION_RED.md` — segmentación de red.

Cómo correr el proyecto en local

1. Requisitos

- Node.js 20+

- PostgreSQL (local, o usa `docker compose up -d` que levanta solo la base de datos — ver `docker-compose.yml`)

2. Instalación

```
npm install
cp .env.example .env
# edita .env: como minimo DB_PASSWORD, JWT_SECRET, GOOGLE_CLIENT_ID, RECAPTCHA_*
npm run db:init    # crea las tablas en PostgreSQL
npm run dev        # levanta el servidor con nodemon en http://localhost:4000
```

3. Abrir el frontend (React + Vite)

El frontend está en la carpeta `frontend/` y está hecho con **ReactJS** (Vite + React Router). Integra Google Login y reCAPTCHA v2 dentro de componentes React.

```
cd frontend
npm install
npm run dev        # http://localhost:5173
```

Usa el backend en `http://localhost:4000/api` por defecto; para cambiarlo copia `frontend/.env.example` como `frontend/.env`. Para producción: `npm run build` (genera `frontend/dist`, que nginx puede servir como sitio estático).

3b. Frontend anterior (HTML + JavaScript)

Abre `index.html` (o `login.html` para empezar) con un servidor estático cualquiera (por ejemplo la extensión "Live Server" de VS Code, o `npx serve .`) — el frontend llama al backend en `http://localhost:4000/api` (configurable en `config.js`).

Estructura del proyecto

```
Controlador/     Logica de negocio (usuarios, publicaciones, follows, imagenes)
rutas/           Definicion de endpoints REST
modelo/          Modelos Sequelize (PostgreSQL) + asociaciones
servicios/       JWT, almacenamiento (local/S3), pipeline de procesamiento de imagenes
middlewares/     Autenticacion JWT
database/        Conexion Sequelize + esquema SQL de referencia
models/          Pesos del modelo de face-api.js (NO son modelos de datos)
deploy/          Scripts y plantillas para el despliegue en AWS (EC2, S3, IAM)
docs/            Documentacion de arquitectura, BD, despliegue y usuario
imagenes/        Fotos subidas en modo de almacenamiento local
*.html, app.js, css/   Frontend estatico
```

Funcionalidades implementadas

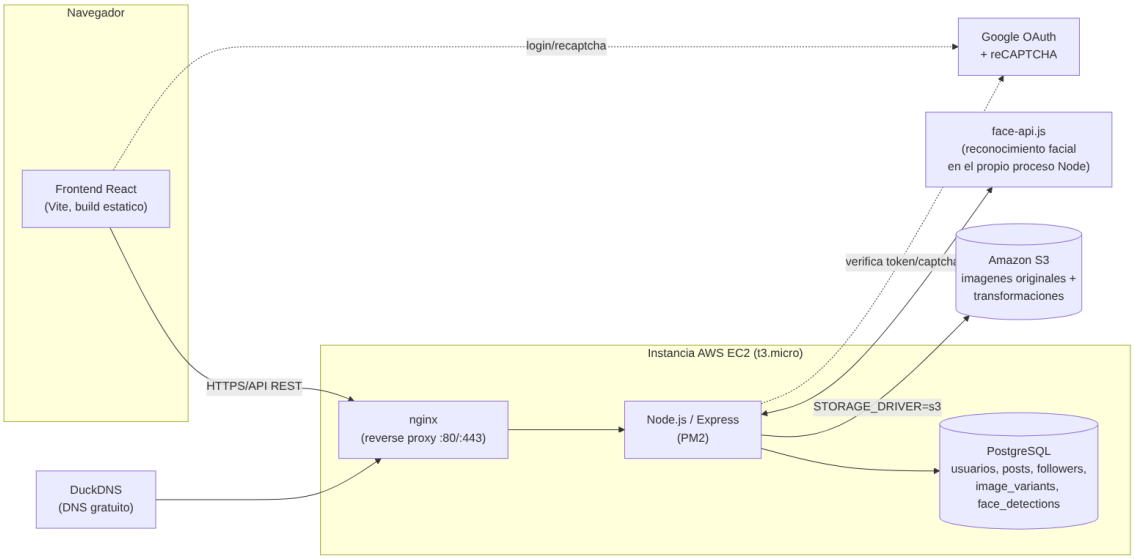
- [x] Bienvenida y autenticación (login/registro) con Google Login y reCAPTCHA
- [x] Feed de miniaturas para usuarios autenticados
- [x] Algoritmo de miniaturas (Sharp) al subir cada foto
- [x] Subida de fotografías
- [x] Reconocimiento facial automático al subir (face-api.js) + análisis manual bajo demanda
- [x] 3 transformaciones automáticas por foto (miniatura, blanco y negro, sepia), + 9 filtros manuales extra
- [x] Visualización en resolución completa con sus transformaciones
- [x] Almacenamiento en la nube (Amazon S3) con capa de abstracción local/S3
- [x] Base de datos relacional (PostgreSQL) para cuentas e índice de imágenes (nunca el binario)
- [x] Contraseñas con hash bcrypt + sal única por usuario
- [x] Artefactos de despliegue para AWS EC2 (PM2, nginx, script de bootstrap)
- [x] Plantillas de política IAM (acceso limitado para el catedrático, acceso S3 de la app)
- [ ] Amazon Rekognition (opcional avanzado, no implementado)

Documentación del desarrollador — Arquitectura

Proyecto Final de Desarrollo Web (UMG) — Red social de fotografías tipo Instagram.

1. Visión general

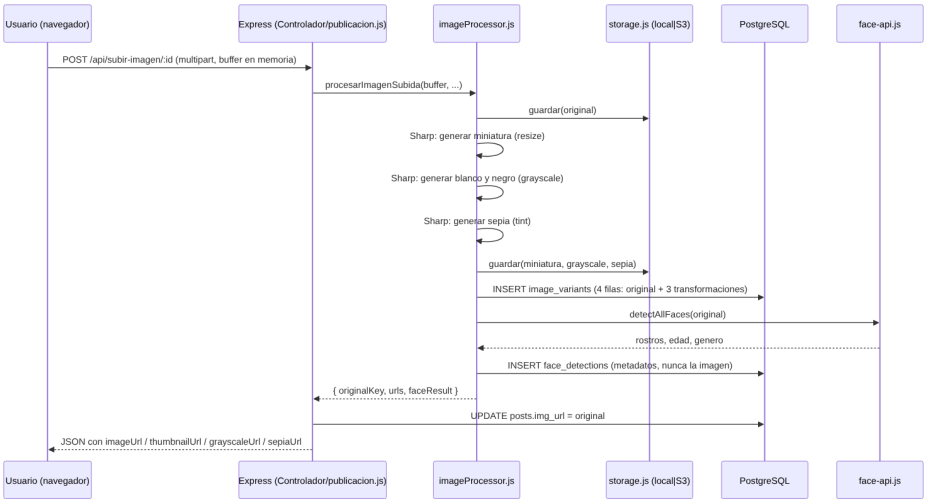
La aplicación es un backend **Node.js / Express** con un frontend en **ReactJS** (Vite + React Router, carpeta `frontend/`), una base de datos **relacional PostgreSQL** y almacenamiento de imágenes desacoplado (**disco local en desarrollo / Amazon S3 en producción**) a través de una capa de abstracción propia.



2. Capas del backend

```
index.js                -> arranque de Express, conexion a BD, servido estatico
rutas/rutas.js           -> define todos los endpoints REST
Controlador/*.js          -> logica de negocio por recurso (user, publicacion, follow, imagen)
modelo/*.js               -> modelos Sequelize (PostgreSQL) + asociaciones (modelo/index.js)
servicios/
  jwt.js                  -> firma/verifica tokens de sesion
  storage.js               -> abstraccion de almacenamiento (local disco <-> S3)
  imageProcessor.js         -> pipeline de transformaciones automaticas + reconocimiento facial
middlewares/auth.js        -> valida el JWT en rutas protegidas
database/
  conexion.js              -> conexion Sequelize + sync de modelos
  schema.sql                -> esquema SQL de referencia (documentacion/manual)
```

3. Flujo de subida de una fotografía (lo más importante del proyecto)



Puntos clave que exige el enunciado y cómo se cumplen:

- **3 transformaciones automáticas al subir la imagen**: miniatura (reducción de escala), blanco y negro y sepia — generadas con **Sharp** sin intervención del usuario, en `servicios/imageProcessor.js`.
- **Reconocimiento facial al subir**: `face-api.js` corre automáticamente sobre la foto original (edad, género, cantidad de rostros) en el mismo flujo.
- **Las imágenes nunca se guardan en la base de datos**: solo se guarda la referencia (`storage_key`, tabla `image_variants`) y PostgreSQL resuelve la URL pública bajo demanda (`servicios/storage.js#urlPublica`).
- **Algoritmo de miniaturas**: todo el feed/listado usa `thumbnailUrl` (imagen reducida), mientras que la vista de detalle usa `imageUrl` (resolución completa) junto a las 3 transformaciones.
- Además de las 3 transformaciones obligatorias, se conserva (como función extra) un editor manual con **9 filtros adicionales** (invertir, desenfoque, brillo, contraste, cálido, frío, psicodélico...) que el usuario aplica bajo demanda sobre su propia foto/avatar (`Controlador/imagen.js`), y un botón de "Analizar rostro" para repetir el análisis facial manualmente.

4. Almacenamiento de imágenes (local vs S3)

`servicios/storage.js` expone una interfaz única (`guardar`, `leer`, `eliminar`, `urlPublica`) que internamente decide, según la variable de entorno `STORAGE_DRIVER`:

STORAGE_DRIVER	Dónde vive el archivo	Para qué se usa
`local` (por defecto)	`./imagenes/<usuarios|post>/...` en disco	Desarrollo sin cuenta de AWS
`s3`	Bucket de Amazon S3 (`AWS_S3_BUCKET`)	Despliegue en producción (requisito del proyecto)

Ningún controlador llama a `fs` o al SDK de AWS directamente: todos pasan por esta capa, así que cambiar de un entorno a otro es solo cuestión de variables de entorno (`.env`), sin tocar código.

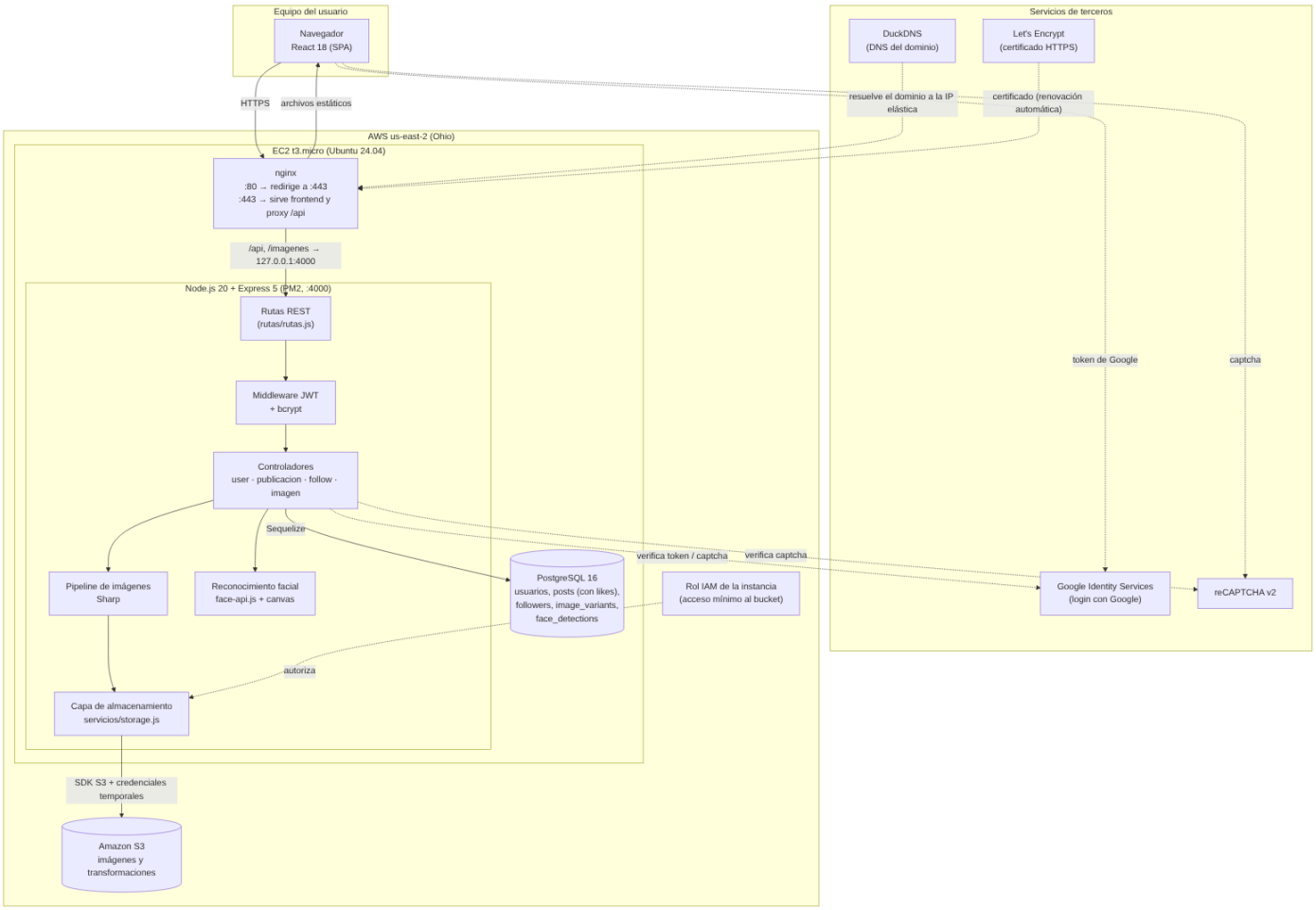
5. Decisiones de diseño y alcance

- **Base de datos relacional (PostgreSQL + Sequelize)**: el proyecto original usaba MongoDB; se migró completamente a PostgreSQL para cumplir el requisito explícito de "base de datos relacional" para cuentas de usuario y ubicación de imágenes (ver `docs/ESQUEMA_BD.md`).
- **Frontend en ReactJS**: el enunciado pide integrar Google Login y reCAPTCHA con ReactJS. El frontend (`frontend/`) está hecho con React 18, Vite y React Router; Google Login y reCAPTCHA v2 son componentes React (`GoogleButton.jsx`, `Recaptcha.jsx`). Para producción se compila con `npm run build` y nginx sirve la carpeta `frontend/dist` como sitio estático. Los archivos HTML y `app.js` de la raíz son la versión anterior del frontend y ya no se usan.
- **Amazon Rekognition**: no implementado (es el punto "opcional avanzado" del enunciado). El reconocimiento facial se resuelve con `face-api.js`, que sí es un requisito obligatorio.
- **Autenticación**: JWT propio (`servicios/jwt.js`) + bcrypt con sal única por usuario (`bcrypt.hash(password, 10)` genera y embebe una sal distinta en cada llamada) + Google OAuth2 + reCAPTCHA v2 (en login y registro).

Diagrama distribuido de componentes

Muestra los componentes de la solución, dónde se ejecuta cada uno y cómo se comunican.

1. Vista de componentes



2. Responsabilidad de cada componente

Componente	Dónde corre	Responsabilidad
React (SPA)	Navegador	Interfaz, formularios, botón de Google, reCAPTCHA, galería y vista ampliada.
nginx	EC2	Termina HTTPS, sirve `frontend/dist`, reenvía `/api` e `/imagenes` a Node.
Node.js / Express	EC2 (PM2)	API REST, autenticación, lógica de negocio.
Middleware JWT	Node	Valida el token en rutas protegidas.
Sharp	Node	Miniatura, blanco y negro, sepia y filtros manuales.
face-api.js + canvas	Node	Detección de rostros, edad y género.
storage.js	Node	Abstracción `local` / `s3`; el resto del código no conoce el destino.
PostgreSQL	EC2	Cuentas, publicaciones, seguidores y el índice de imágenes (nunca el binario).
Amazon S3	AWS	Archivos de imagen.
Rol IAM	AWS	Credenciales temporales de la instancia hacia S3, sin claves en el servidor.
DuckDNS / Let's Encrypt	Terceros	Nombre de dominio gratuito y certificado HTTPS.

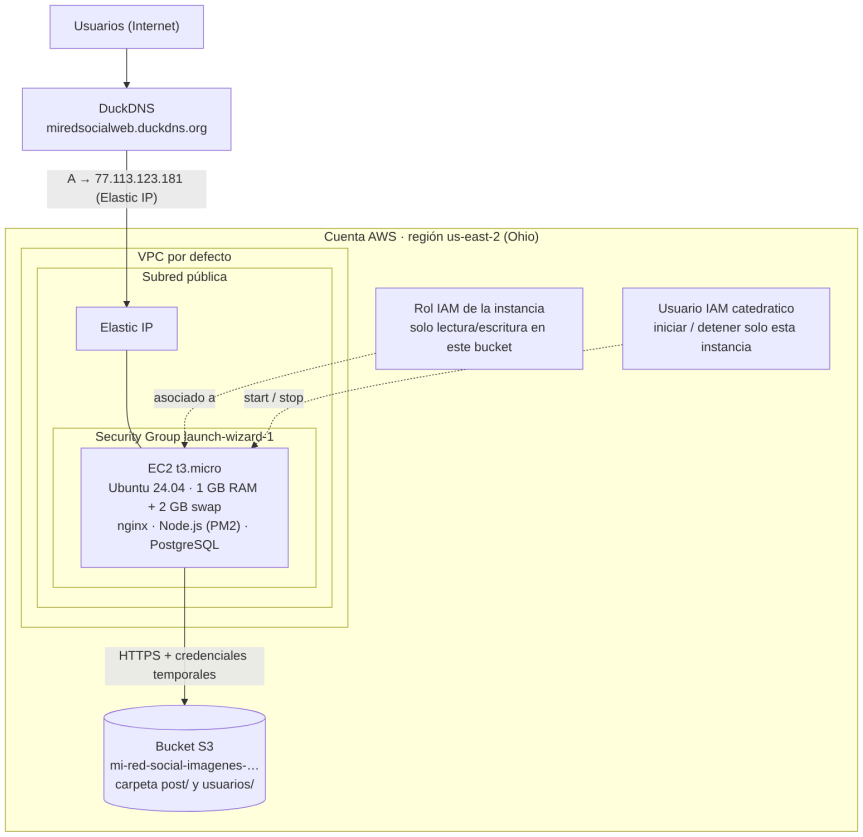
3. Qué es distribuido y qué no

- **Distribuido:** frontend (navegador), servidor de aplicación (EC2), almacenamiento (S3) y servicios externos (Google, reCAPTCHA, DNS).
- **Sin distribuir (decisión de costo):** nginx, Node y PostgreSQL comparten una instancia. En el plan de crecimiento de `SEGMENTACION_RED.md` se propone separarlos (base de datos en RDS en subred privada y balanceador de carga).

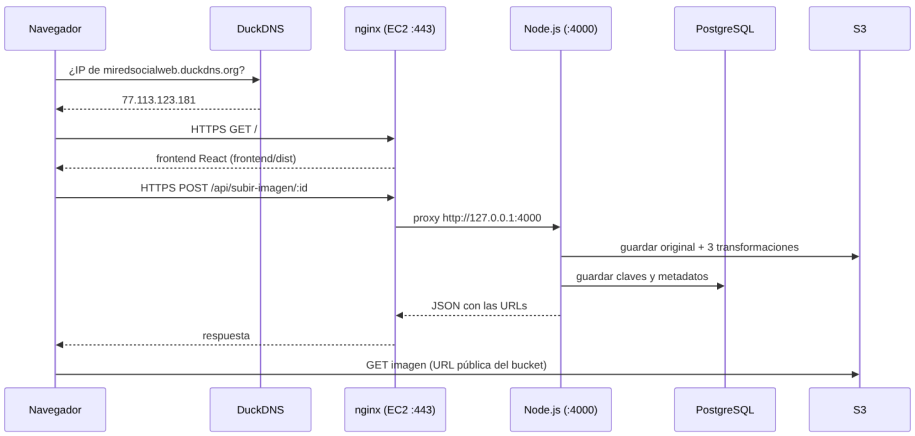
Diagramas de Cloud Computing

Diagramas de la infraestructura en la nube usada para la entrega (AWS, región Ohio, us-east-2).

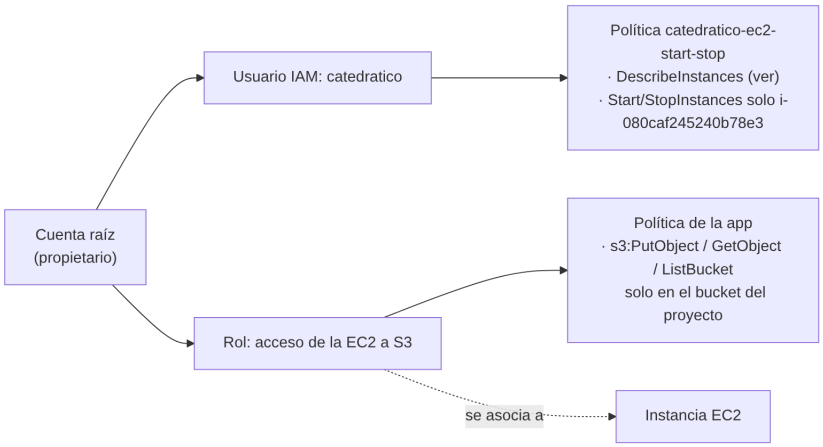
1. Topología general



2. Flujo de una solicitud (del navegador al dato)

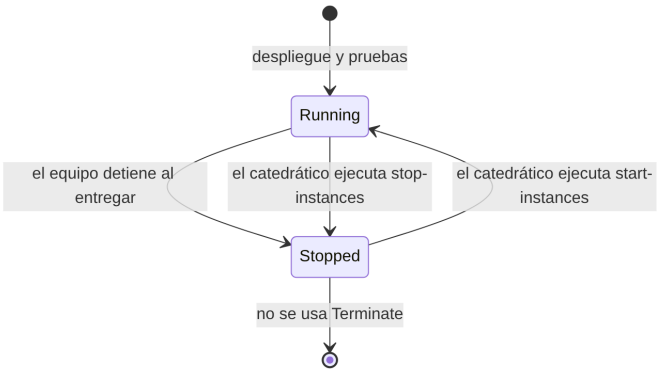


3. Identidad y permisos (IAM)



Principio aplicado: **mínimo privilegio**. El catedrático no puede crear ni borrar recursos, y la aplicación no guarda claves de acceso en el servidor.

4. Ciclo de vida de la instancia para la revisión



Al estar detenida no se cobra la instancia; se conservan el disco (EBS), la base de datos y la IP elástica asociada.

5. Recursos AWS usados

Servicio	Recurso	Uso
EC2	t3.micro, Ubuntu 24.04	Aplicación completa
EC2	Elastic IP	IP fija para el dominio
EC2	Security Group	Firewall: 22, 80 y 443
EC2	Par de claves RSA	Acceso SSH
S3	Bucket de imágenes	Almacén de las fotos
IAM	Rol de instancia + política S3	Acceso a S3 sin claves
IAM	Usuario `catedratico` + política	Iniciar/detener la instancia

6. Estimación de costo

Con el plan gratuito de la cuenta (créditos de $100 por 182 días) la entrega no genera cargo directo. Fuera del plan, el consumo estimado es:

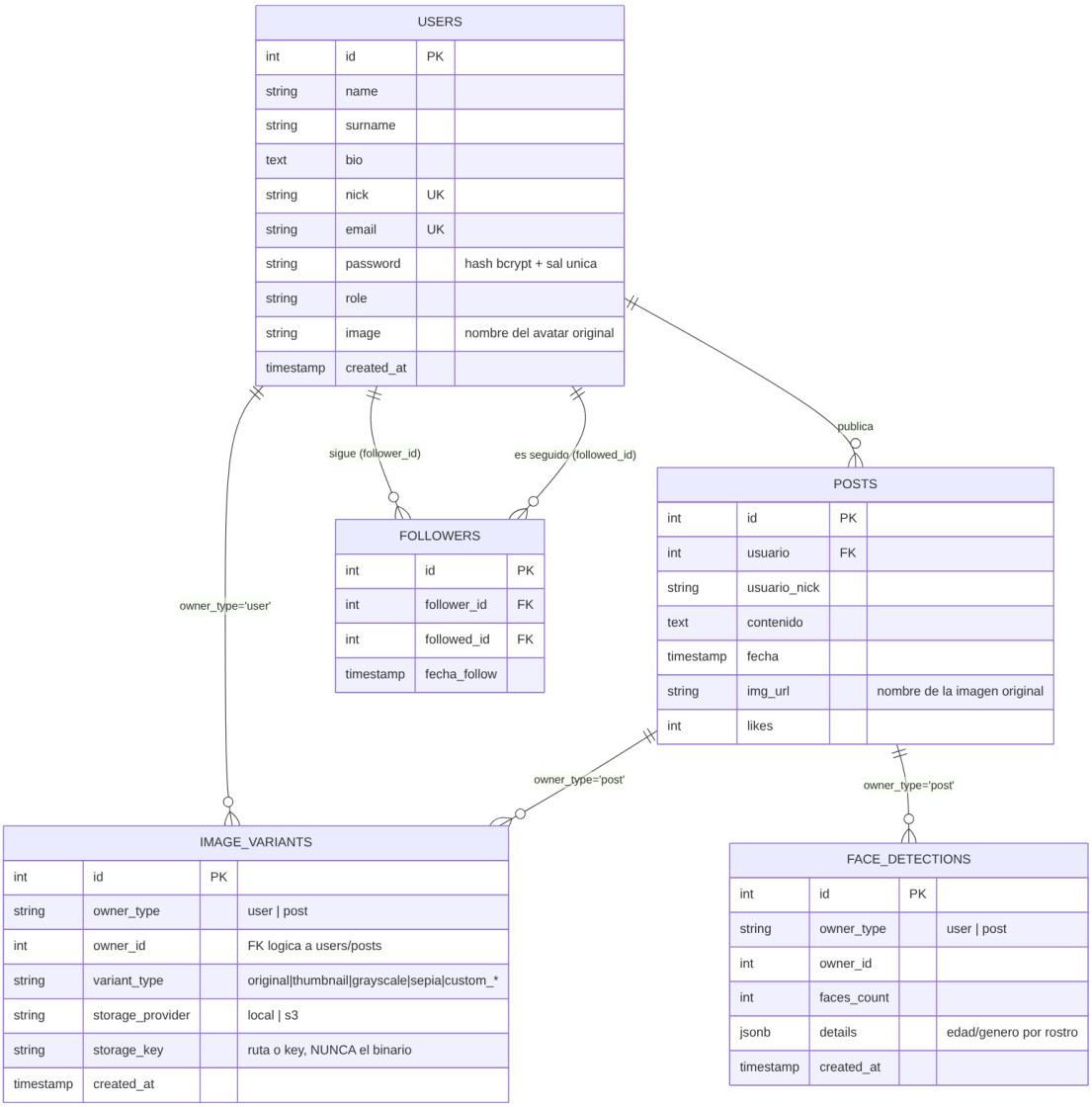
Concepto	Observación
EC2 t3.micro	Se paga por hora encendida; detenida, no cobra cómputo.
EBS (disco)	Se cobra aunque la instancia esté detenida (pocos centavos al mes).
IPv4 pública / Elastic IP	AWS cobra por hora cada dirección IPv4 pública, esté asociada o no; con el crédito del plan gratuito no se paga. No liberarla mientras se revise el proyecto.
S3	Centavos al mes con el volumen de pruebas.

Esquema de la base de datos (PostgreSQL)

Motor: **PostgreSQL 14+**. ORM: **Sequelize** (`modelo/*.js`). Esquema SQL de referencia equivalente en `database/schema.sql`.

La app crea/actualiza las tablas automáticamente al arrancar (`sequelize.sync()`, ver `database/conexion.js`). Para producción se recomienda ejecutar `database/schema.sql` manualmente y desactivar el `sync` automático, pero para la entrega académica el `sync` simplifica el primer arranque.

Diagrama entidad-relación



Notas de diseño

- **Las imágenes nunca se guardan en la base de datos.** `image_variants` y `face_detections` solo guardan metadatos y la referencia de almacenamiento (`storage_key`): una ruta relativa en disco (`post/xxxx.jpg`) o una *key* de S3, según `storage_provider`. El binario vive en disco o en S3; la BD solo sabe *dónde* está.
- `image_variants.owner_id` es una referencia **lógica** (no una FK física de PostgreSQL) porque puede apuntar a `users.id` o a `posts.id` según `owner_type`. Esto es un patrón polimórfico común cuando una tabla referencia a más de una tabla padre.

- `followers` tiene una restricción `UNIQUE(follower_id, followed_id)` y un `CHECK(follower_id <> followed_id)` para impedir seguirse a uno mismo o duplicar el seguimiento, a nivel de base de datos (no solo en el código).
- Los nombres de columnas usan `snake_case` (convención PostgreSQL) mientras que los modelos Sequelize exponen los atributos en `camelCase` (`usuarioNick`, `imgUrl`, etc.) para no romper el contrato ya usado por el frontend.

Mantenimiento de la base de datos

- **Backups**: en RDS, habilitar *automated backups* (retención 7 días mínimo) y tomar un *snapshot* manual antes de cambios grandes de esquema. En una instalación nativa sobre la misma EC2 (como en `deploy/ec2-bootstrap.sh`), usar `pg_dump` programado (cron) hacia S3.
- **Migraciones futuras**: si el proyecto crece más allá de la entrega académica, reemplazar `sequelize.sync({ alter: true })` por migraciones versionadas (`sequelize-cli db:migrate`) para evitar cambios de esquema no controlados en producción.
- **Índices**: ya incluidos para las consultas más frecuentes del feed (`posts(usuario)`, `posts(fecha DESC)`, `image_variants(owner_type, owner_id)`).

Manual técnico y de la aplicación web

Referencia para entender el código, ejecutarlo y mantenerlo. Complementa `ARQUITECTURA.md` y `ESQUEMA_BD.md`.

1. Tecnologías y versiones

Capa	Tecnología
Frontend	React 18, Vite 5, React Router, CSS3
Backend	Node.js 20, Express 5
Base de datos	PostgreSQL 16, Sequelize 6
Autenticación	JWT (`jwt-simple`), bcryptjs, `google-auth-library`, reCAPTCHA v2
Imágenes	Sharp (transformaciones), face-api.js + canvas (rostros)
Almacenamiento	Amazon S3 (`@aws-sdk/client-s3`) o disco local
Despliegue	AWS EC2, nginx, PM2, Let's Encrypt

2. Estructura del proyecto

```
Controlador/   user.js, publicacion.js, follow.js, imagen.js  (lógica de negocio)
rutas/         rutas.js                                        (endpoints REST)
modelo/        user, publicacion, follower, imageVariant, faceDetection (+ index.js)
servicios/     jwt.js, storage.js, imageProcessor.js
middlewares/   auth.js                                        (valida el JWT)
database/      conexion.js, initSchema.js, schema.sql
models/        pesos de face-api.js (no son modelos de datos)
frontend/src/  App.jsx, api.js, auth.jsx, config.js, components/, pages/
deploy/        bootstrap, nginx, políticas IAM/S3, respaldo
docs/          documentación
```

3. Aplicación web (frontend)

SPA en React con enrutamiento del lado del cliente. Toda la comunicación con el servidor pasa por `src/api.js`, que añade el encabezado `Authorization` con el token guardado.

Página	Función
`Inicio`	Bienvenida y acceso a iniciar sesión o registrarse.
`Login`	Correo + contraseña + reCAPTCHA, o botón de Google.
`Registro`	Alta de cuenta con foto de perfil opcional y reCAPTCHA.
`FeedVista`	Galería de miniaturas de todas las publicaciones.
`Siguiendo`	Publicaciones de las personas que sigues.
`Perfil`	Datos del usuario y sus fotos; crear, editar y borrar.
`EditarPerfil`	Actualizar datos y foto de perfil.

Componentes principales: `Galeria` (cuadrícula de miniaturas), `Lightbox` (vista ampliada con tira de versiones, 10 filtros y análisis de rostros), `GoogleButton`, `Recaptcha`, `PostCard`, `Navbar` (barra superior en escritorio, inferior en móvil).

Configuración (`src/config.js`): `VITE_API_URL`, `VITE_GOOGLE_CLIENT_ID`, `VITE_RECAPTCHA_SITE_KEY`. En producción se compila con `VITE_API_URL=/api`, así el navegador llama al mismo dominio y nginx reenvía a Node.

4. API REST

Prefijo `/api`. Las rutas marcadas con 🔒 exigen el encabezado `Authorization: <token JWT>`.

Método	Ruta	Descripción
POST	`/registrar/user`	Crea una cuenta (valida reCAPTCHA, guarda hash bcrypt).
POST	`/login/user`	Inicio de sesión con correo y contraseña; devuelve el token.

Método	Ruta	Descripción
POST	`/login/google`	Valida el token de Google en el servidor y devuelve el token propio.
GET 🔒	`/profile/user/:id`	Datos de un usuario.
GET 🔒	`/listar/user/:page`	Lista paginada de usuarios.
PUT 🔒	`/actualizar/user`	Actualiza datos del usuario.
PUT 🔒	`/upload/user`	Sube la foto de perfil.
POST 🔒	`/crear/publicacion`	Crea una publicación.
POST 🔒	`/subir-imagen/:id`	Sube la foto de una publicación; genera transformaciones y analiza rostros.
GET 🔒	`/listar/publicacion`	Feed global.
GET 🔒	`/publicacion/:id`	Una publicación con sus versiones.
PUT 🔒	`/actualizar/publicacion/:id`	Edita el texto.
DELETE 🔒	`/borrar/publicacion/:id`	Borra la publicación.
PUT 🔒	`/likes/publicacion`	Da o quita "me gusta".
POST 🔒	`/follow/seguir/:id`	Seguir a un usuario.
DELETE 🔒	`/follow/dejar/:id`	Dejar de seguir.
GET 🔒	`/follow/comprobar/:id`	¿Ya lo sigo?
GET 🔒	`/listar/siguiendo`	Publicaciones de quienes sigo.
GET 🔒	`/follow/usuario-nick/:nick`	Buscar usuario por nick.
POST 🔒	`/imagen/transformar`	Aplica un filtro manual (grayscale, sepia, invert, blur, bright, contrast, warm, cool, hue) y lo guarda.
POST 🔒	`/analizar-rostro`	Analiza rostros de una foto bajo demanda.

5. Procesamiento de imágenes

Al subir una foto (`servicios/imageProcessor.js`):

1. Se guarda el **original** en el almacenamiento.
2. Con **Sharp** se generan tres transformaciones: **miniatura** (reducción de escala), **blanco y negro** y **sepia**.
3. Se registran cuatro filas en `image_variants` (clave de almacenamiento, no el binario).
4. **face-api.js** detecta rostros, puntos faciales, edad y género; el resultado va a `face_detections`.

Las claves quedan bajo `post/` (publicaciones) y `usuarios/` (perfiles). La base de datos guarda solo la referencia a la clave.

6. Almacenamiento (`servicios/storage.js`)

Interfaz común con dos implementaciones, elegidas con `STORAGE_DRIVER`:

Valor	Destino	Uso
`local`	Carpeta `imagenes/`	Desarrollo, sin cuenta de AWS.
`s3`	Bucket de S3	Producción. Usa la cadena de credenciales por defecto del SDK: en la EC2 toma el rol IAM de la instancia.

7. Variables de entorno

Variable	Descripción
`PORT`	Puerto de Node (4000).
`NODE_ENV`	`production` en el servidor.
`BASE_URL`, `CORS_ORIGIN`	URL pública y origen permitido por CORS.

Variable	Descripción
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_SSL`	Conexión a PostgreSQL.
`JWT_SECRET`	Clave de firma de las sesiones.
`GOOGLE_CLIENT_ID`	Client ID para validar el token de Google.
`RECAPTCHA_SITE_KEY`, `RECAPTCHA_SECRET_KEY`	Claves de reCAPTCHA v2.
`STORAGE_DRIVER`, `AWS_REGION`, `AWS_S3_BUCKET`	Almacenamiento. En EC2 no se definen claves de acceso (rol IAM).
`REKOGNITION_ENABLED`	Activa Amazon Rekognition (opcional, desactivado).

El archivo `.env` nunca se sube al repositorio (`.gitignore`); `.env.example` es la plantilla.

8. Ejecución en desarrollo

```
npm install
cp .env.example .env     # completar valores
npm run db:init          # crea las tablas
npm run dev              # API en http://localhost:4000

cd frontend
npm install
npm run dev              # interfaz en http://localhost:5173
```

9. Operación en producción

Tarea	Comando
Estado de la aplicación	`pm2 status`
Ver registros	`pm2 logs mi-red-social`
Reiniciar	`pm2 restart mi-red-social`
Recompilar el frontend	`cd frontend && VITE_API_URL=/api npm run build`
Probar y recargar nginx	`sudo nginx -t && sudo systemctl reload nginx`
Estado del certificado	`sudo certbot certificates`
Respaldo de la BD	`deploy/backup-db.sh`

PM2 está configurado para arrancar con el sistema; si la instancia se reinicia, la aplicación se levanta sola.

9.1 Convenciones de código

- Código dividido por capas (rutas → controladores → modelos/servicios) y comentado en español.
- Los controladores responden JSON con `status` y `mensaje`.
- Los secretos solo viven en variables de entorno.
- Ramas de GitHub por integrante para identificar quién desarrolló cada parte.

10. Problemas conocidos

- El análisis facial usa el backend JavaScript de TensorFlow (sin aceleración nativa), por lo que tarda varios segundos en la t3.micro.
- Las imágenes del bucket son de lectura pública para poder mostrarse en el feed; los datos personales no se guardan en las imágenes.
- El Login con Google exige un origen HTTPS autorizado en el Client ID del equipo.

Manual de usuario

Guía rápida de uso de **Mi Red Social**, una aplicación para compartir y explorar fotografías (estilo Instagram).

1. Crear una cuenta

1. Abre la página de inicio; si no has iniciado sesión verás los enlaces **Iniciar sesión** / **Registrarse**.
2. En **Registrarse**, completa nombre, apellidos, usuario (nick), correo y contraseña. Opcionalmente adjunta una foto de perfil.
3. Marca la casilla **"No soy un robot"** (reCAPTCHA) — es obligatoria.
4. Presiona **Registrarse**. Si todo es correcto, se te redirige al inicio de sesión (o directo al feed si subiste foto de perfil).

2. Iniciar sesión

Tienes dos formas:

- **Formulario tradicional**: correo + contraseña + reCAPTCHA.
- **Botón "Sign in with Google"**: inicia sesión con tu cuenta de Google sin necesidad de crear una contraseña (si es tu primera vez, se crea la cuenta automáticamente).

3. Explorar el feed (inicio)

Al iniciar sesión llegas al **Feed Global**, donde ves miniaturas de las fotos publicadas por todos los usuarios, con el nombre de usuario del autor y un botón de "me gusta" (❤). Desde la barra inferior puedes cambiar entre:

- **Feed Global**: todas las publicaciones.
- **Siguiendo**: solo publicaciones de las personas que sigues.
- **Mi Perfil**: tu información y tus propias publicaciones.

4. Publicar una fotografía

1. Ve a **Mi Perfil** → **+ Crear Nueva Publicación**.
2. Escribe el texto de tu publicación y, opcionalmente, selecciona una imagen.
3. Al publicar, el sistema procesa automáticamente la foto: genera una **miniatura**, una versión en **blanco y negro** y una versión **sepia**, y analiza si hay rostros en la imagen (edad y género estimados). Todo esto ocurre en segundo plano, sin que tengas que hacer nada más.

5. Ver una foto en resolución completa y sus transformaciones

Haz clic sobre cualquier miniatura del feed o de tu perfil para abrir el visor: verás la foto en **resolución completa**, una pequeña galería con las **3 transformaciones automáticas** (miniatura/blanco y negro/sepia), y —si es tuya— un editor con **9 filtros adicionales** (sepia, invertir, desenfoque, brillo, contraste, cálido, frío, psicodélico, etc.) que puedes aplicar y guardar sobre tu propia foto o avatar.

También puedes presionar **"Analizar Rostro"** para repetir el análisis facial y ver el detalle de edad/género estimados por rostro detectado.

6. Seguir a otros usuarios

Haz clic en el nombre de usuario (encabezado de cualquier publicación) para abrir su mini-perfil: ahí puedes ver sus publicaciones y presionar **Seguir/Dejar de seguir**. Sus publicaciones aparecerán después en tu pestaña **Siguiendo**.

7. Editar tu perfil

Desde **Mi Perfil → Actualizar Perfil** puedes cambiar nombre, apellidos, usuario, correo y contraseña, y subir/cambiar tu foto de perfil.

8. Cerrar sesión

Botón **Cerrar Sesión** en la esquina superior derecha, disponible en cualquier página.

Instrucciones de despliegue (AWS)

Esta guía documenta cómo se desplegó el proyecto y cómo repetirlo. Se hizo con una cuenta nueva de **AWS (plan gratuito / Free plan)**.

Desviaciones respecto al enunciado

Enunciado	Lo que se usó	Motivo
Región N. Virginia (us-east-1)	**Ohio (us-east-2)**	El plan gratuito de la cuenta nueva solo permite operar en us-east-2 sin pasar a plan de pago.
Instancia t2.small	**t3.micro** (1 GB RAM) + 2 GB de swap	Es el único tipo elegible en el plan gratuito. El swap compensa la poca memoria.
Route 53 para el DNS	**DuckDNS** (gratuito) + HTTPS con Let's Encrypt	Evita el costo de un dominio de pago; Google exige dominio con HTTPS para el login.

El resto del diseño (EC2, S3, base relacional, PM2, nginx, IAM, SSH) es el mismo.

Resumen de lo desplegado

Elemento	Valor
Región	us-east-2 (Ohio)
Instancia	t3.micro, Ubuntu Server 24.04 LTS, ID `i-080caf245240b78e3`
IP pública (Elastic IP)	`77.113.123.181`
Dominio	`miredsocialweb.duckdns.org` (DuckDNS, gratuito)
DNS público de AWS	`ec2-77-113-123-181.us-east-2.compute.amazonaws.com` (confirmar en la consola)
Sitio	`https://miredsocialweb.duckdns.org`
Frontend	React (Vite) compilado en `frontend/dist`, servido por nginx
Backend	Node.js/Express en el puerto 4000 (PM2), accesible solo vía nginx
Base de datos	PostgreSQL local en la instancia (`mi_red_social`)
Imágenes	Bucket S3 `mi-red-social-imagenes-meli2901` (carpeta `post/`)
Acceso de la app a S3	**Rol IAM asociado a la instancia** (sin claves en el `.env`)

0. Requisitos previos

- Cuenta de AWS con crédito disponible.
- Cliente SSH (Windows: PowerShell con OpenSSH).
- El código en un repositorio de GitHub **privado** del equipo.

1. Par de claves SSH

1. **EC2 → Pares de claves → Crear par de claves**.
2. Tipo RSA, formato `.pem`. Se descarga una sola vez: guardarlo fuera del repositorio (el `.gitignore` excluye `*.pem`).
3. Este archivo es el par de claves SSH que se entrega al catedrático por un canal privado, **nunca** en GitHub.

2. Lanzar la instancia EC2

1. Cambiar la región a **Ohio (us-east-2)**.
2. **EC2 → Lanzar instancia**, AMI **Ubuntu Server 24.04 LTS** (Canonical, elegible para el plan gratuito).
3. Tipo **t3.micro**, par de claves del paso 1, almacenamiento 20-30 GB gp3.
4. Security Group con reglas de entrada:

Tipo	Puerto	Origen
SSH	22	Mi IP (al entregar, restringir; ver paso 8)
HTTP	80	0.0.0.0/0
HTTPS	443	0.0.0.0/0 (cuando exista dominio)

Los puertos 4000 (Node) y 5432 (PostgreSQL) **no** se abren.

5. Asignar una **Elastic IP** a la instancia para que la IP no cambie al detenerla.

3. Conectarse y subir el código

```
ssh -i mi-red-social-key.pem ubuntu@77.113.123.181
```

(También se puede usar **EC2 Instance Connect** desde la consola, sin SSH local.)

```
git clone https://github.com/<usuario>/<repo>.git mi_red_social
cd mi_red_social
```

Para repositorio privado se usa un token de acceso personal; revocarlo al terminar.

4. Ejecutar el script de bootstrap

```
chmod +x deploy/ec2-bootstrap.sh
./deploy/ec2-bootstrap.sh      # 1.ª vez: instala todo y crea .env; se detiene
nano .env                       # completar valores
./deploy/ec2-bootstrap.sh      # 2.ª vez: BD, tablas, build de React, PM2 y nginx
```

El script: crea 2 GB de swap, instala Node 20, PostgreSQL, nginx, PM2 y las librerías de `sharp`/`canvas`; compila el frontend (`VITE_API_URL=/api`); levanta la API con PM2 (arranque automático con systemd) y configura nginx para servir `frontend/dist` y reenviar `/api` e `/imagenes` al puerto 4000.

Valores mínimos del `.env` en producción:

```
PORT=4000
NODE_ENV=production
BASE_URL=http://<IP-o-dominio>
CORS_ORIGIN=http://<IP-o-dominio>
DB_HOST=localhost
DB_NAME=mi_red_social
DB_USER=miredsocial
DB_PASSWORD=<clave fuerte>
JWT_SECRET=<aleatorio de 48 bytes>
GOOGLE_CLIENT_ID=<Google Cloud>
RECAPTCHA_SITE_KEY=<reCAPTCHA Admin>
RECAPTCHA_SECRET_KEY=<reCAPTCHA Admin>
STORAGE_DRIVER=s3
AWS_REGION=us-east-2
AWS_S3_BUCKET=<nombre-del-bucket>
REKOGNITION_ENABLED=false
```

5. Bucket S3 y rol IAM de la instancia

1. **S3 → Crear bucket** en us-east-2, nombre único. Desmarcar "Bloquear todo el acceso público" (el feed debe mostrar las fotos).
2. **Permisos → Política de bucket**: contenido de `deploy/s3-bucket-policy.json` con el nombre del bucket (lectura pública de objetos).
3. **IAM → Políticas → Crear política (JSON)** con `deploy/iam-app-s3-policy.json` (permisos de escritura/lectura solo sobre ese bucket).
4. **IAM → Roles → Crear rol** (entidad: servicio EC2), adjuntar esa política.
5. **EC2 → Instancia → Acciones → Seguridad → Modificar rol de IAM** y asociar el rol.
6. `pm2 restart mi-red-social`. La app usa las credenciales temporales del rol: **no se guardan claves de acceso en el servidor**.

7. Subir una foto y comprobar en S3 (`post/`) los 4 archivos: original, miniatura, blanco y negro, sepia.

6. reCAPTCHA, dominio, HTTPS y Google Login

- **reCAPTCHA v2**: en el panel de administración se desactivó "Verificar el origen de las solicitudes de reCAPTCHA" para que funcione también al acceder por IP. Con dominio propio se puede volver a activar agregando el dominio.
- **Dominio gratuito (DuckDNS)**: se creó `miredsocialweb.duckdns.org` apuntando a la Elastic IP `77.113.123.181`. Es una desviación respecto al requisito de Route 53 (ver tabla del inicio); el servicio equivalente de Route 53 sería una zona alojada con un registro A a la misma IP.
- **HTTPS con Let's Encrypt**: el bloque de nginx usa el nombre del dominio y Certbot emite y renueva el certificado automáticamente:

  ```
  sudo sed -i 's/server_name _;/server_name miredsocialweb.duckdns.org;/' /etc/nginx/sites-available/mi-red-social
  sudo nginx -t && sudo systemctl reload nginx
  sudo apt install certbot python3-certbot-nginx -y
  sudo certbot --nginx -d miredsocialweb.duckdns.org --agree-tos -m <correo> --redirect --no-eff-email
  ```

  Requiere el puerto 443 abierto en el Security Group.
- **Actualizar la dirección de la app** en `.env` (`BASE_URL` y `CORS_ORIGIN` con `https://miredsocialweb.duckdns.org`) y `pm2 restart mi-red-social`.
- **Google Login**: usa el Client ID del equipo (`GOOGLE_CLIENT_ID` en `.env` y en `frontend/src/config.js`, que se incrusta al compilar el frontend). Google exige que el origen `https://miredsocialweb.duckdns.org` esté en "Orígenes autorizados de JavaScript" y, mientras la pantalla de consentimiento esté en modo Prueba, que cada correo esté como usuario de prueba. No acepta IPs.

7. Acceso del catedrático (IAM limitado)

Se creó el usuario IAM `catedratico` con la política `deploy/iam-ec2-start-stop-policy.json`: puede **ver** instancias y **iniciar/detener únicamente** la instancia `i-080caf245240b78e3` en us-east-2. Cualquier otra acción devuelve "Access Denied".

Este tipo de cuenta se creó con **clave de acceso** (CLI), porque la consola del plan gratuito no habilita contraseña de consola para usuarios IAM. Comandos de uso:

```
aws configure            # Access key ID, Secret key, región us-east-2, formato json
aws ec2 describe-instances --region us-east-2
aws ec2 start-instances --instance-ids i-080caf245240b78e3 --region us-east-2
aws ec2 stop-instances  --instance-ids i-080caf245240b78e3 --region us-east-2
```

Detalles de la entrega en `ENTREGA_CATEDRATICO.md`.

8. Antes de la entrega

1. Respaldar la BD (`deploy/backup-db.sh`).
2. Cambiar la regla SSH del Security Group a una IP concreta o dejarla según se acuerde con el catedrático.
3. **Detener (Stop, no Terminate)** la instancia: `aws ec2 stop-instances --instance-ids i-080caf245240b78e3 --region us-east-2` o **EC2 → Estado de la instancia → Detener**. Con la Elastic IP asociada, la IP se conserva al reiniciar. Disco y base de datos se mantienen.
4. Verificar que el estado sea **Stopped**.

9. Checklist de verificación

- [ ] `http://<dominio-o-IP>/` carga la app React
- [ ] Registro e inicio de sesión con correo y reCAPTCHA
- [ ] Login con Google (requiere el dominio HTTPS autorizado)
- [ ] Subir foto: aparecen 4 archivos en S3 (`post/`)
- [ ] Análisis facial funciona (`pm2 logs mi-red-social`)

- [ ] `pm2 status` en línea y sobrevive a un reinicio de la instancia
- [ ] Usuario `catedratico` puede iniciar/detener y nada más
- [ ] Instancia en estado **Stopped** al entregar

10. Solución de problemas

Síntoma	Causa probable	Solución
`pm2 logs` con error de BD	`.env` incorrecto o PostgreSQL detenido	`sudo systemctl status postgresql`, revisar `DB_*`
La instancia se congela al procesar fotos	Poca RAM (t3.micro)	Confirmar swap con `free -h`; el análisis facial tarda varios segundos
Subida de imagen falla con 500	Faltan librerías de `canvas`	`sudo apt install libcairo2-dev libpango1.0-dev libjpeg-dev libgif-dev librsvg2-dev && npm rebuild`
nginx 502	Node no corre	`pm2 status`, `pm2 restart mi-red-social`
nginx 403 al abrir el sitio	nginx sin permiso en la carpeta home	`sudo chmod o+x $HOME`
Fotos no se ven	Política del bucket o rol IAM	Revisar política pública del bucket y el rol asociado a la instancia
reCAPTCHA "dominio no válido"	Verificación de origen activa con IP	Desactivarla o usar el dominio registrado
Error 400 al usar Google	Origen no permitido (IP)	Usar dominio con HTTPS autorizado en Google Cloud

Evidencias del despliegue en AWS

Capturas de pantalla tomadas durante la configuración de la cuenta, la instancia y los servicios. Se omitieron las pantallas que mostraban claves, contraseñas o datos privados.

A. Cuenta de AWS y región

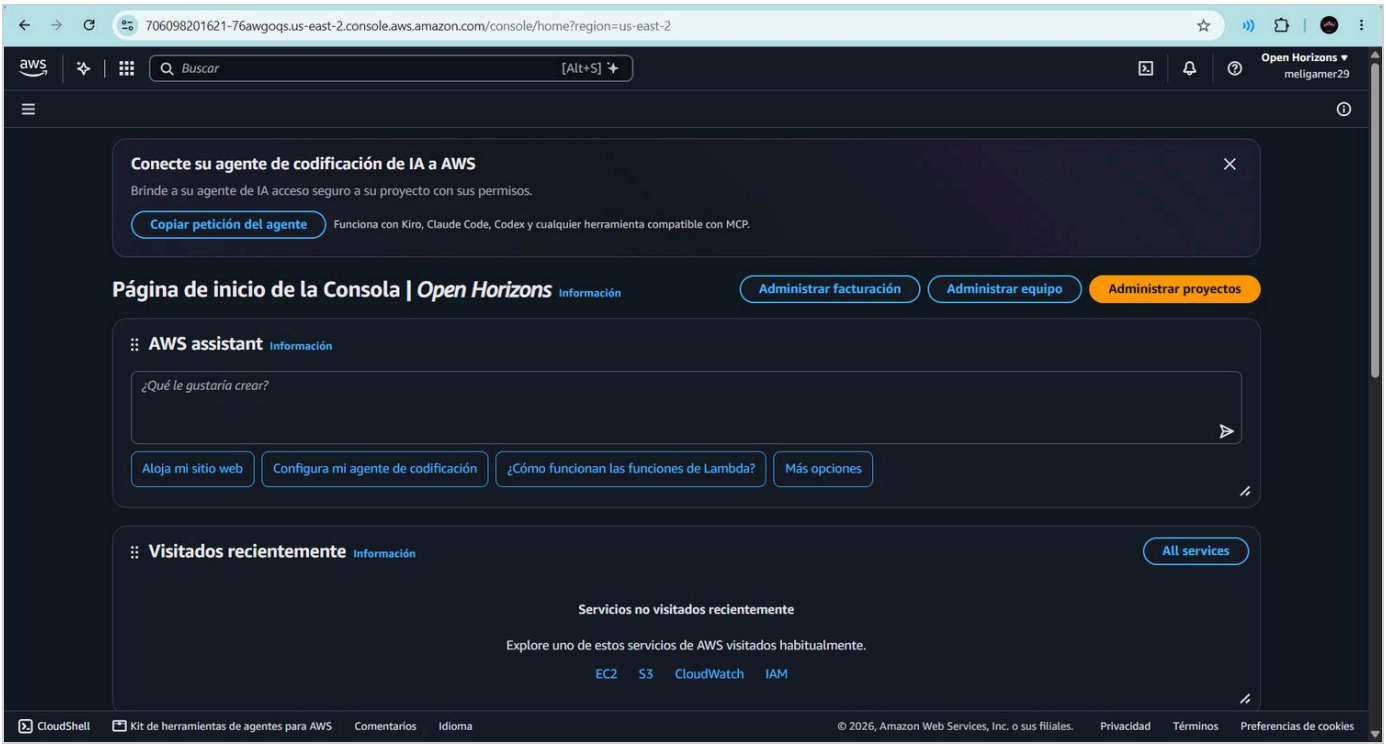


Figura 1. Consola de AWS de la cuenta del proyecto, trabajando en la región Ohio (us-east-2).

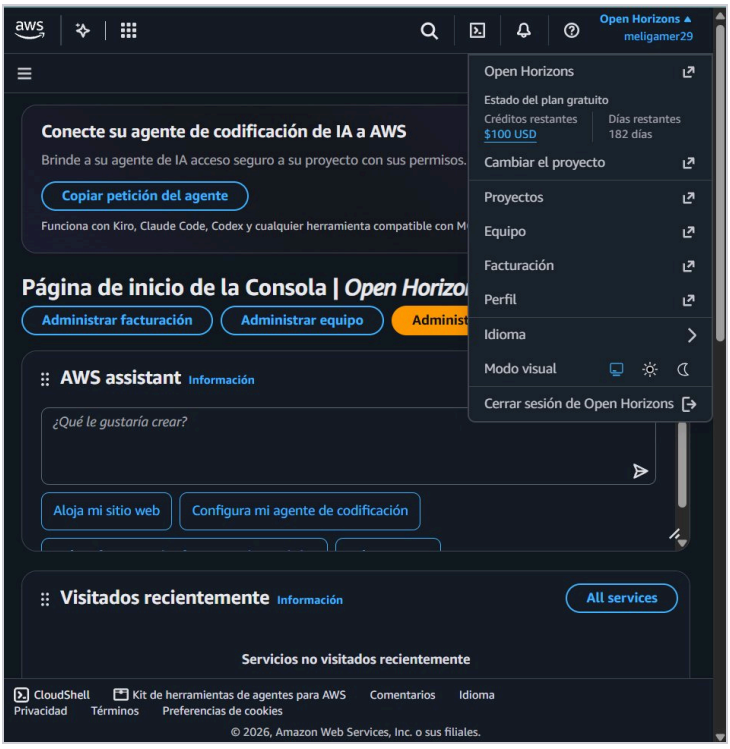


Figura 2. Estado del plan gratuito: 100 USD de crédito y 182 días de vigencia.

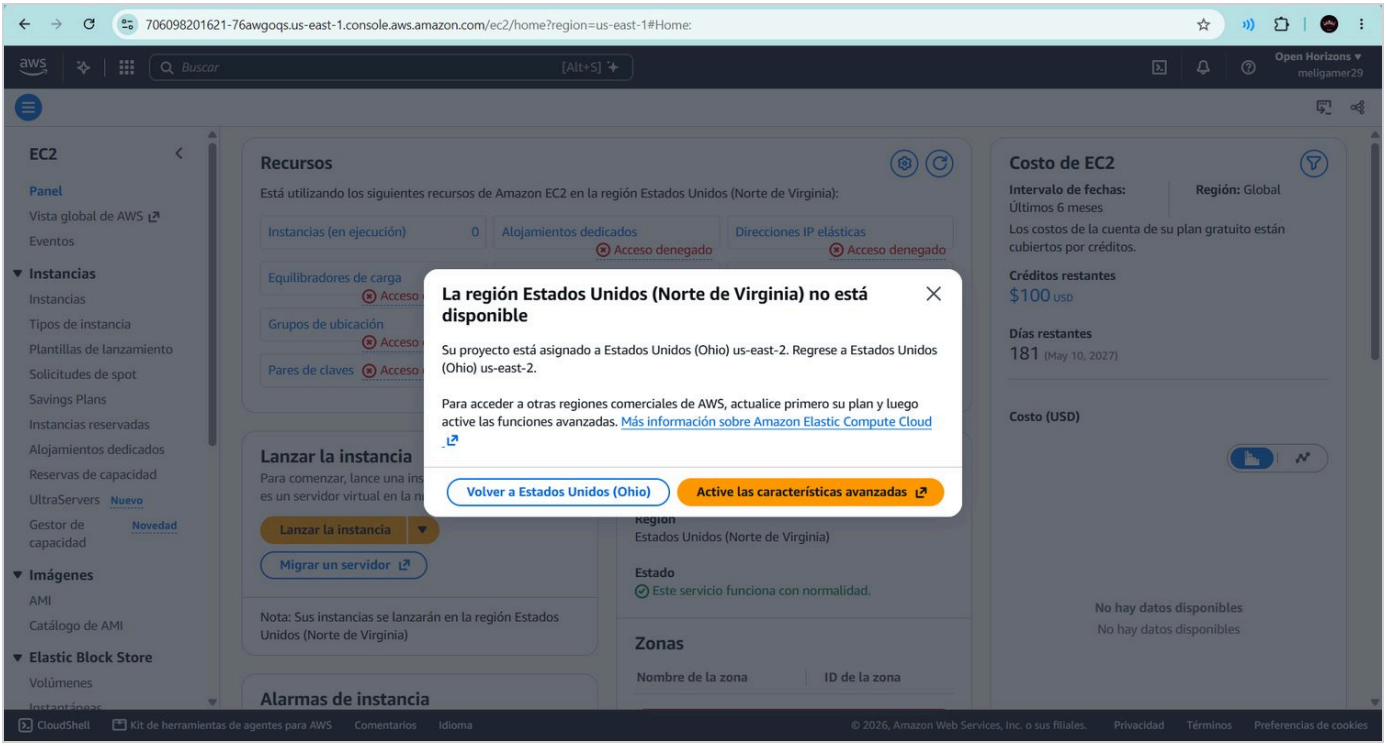


Figura 3. Aviso de AWS: la región N. Virginia no está disponible en el plan gratuito, por eso se trabaja en Ohio.

B. Instancia EC2

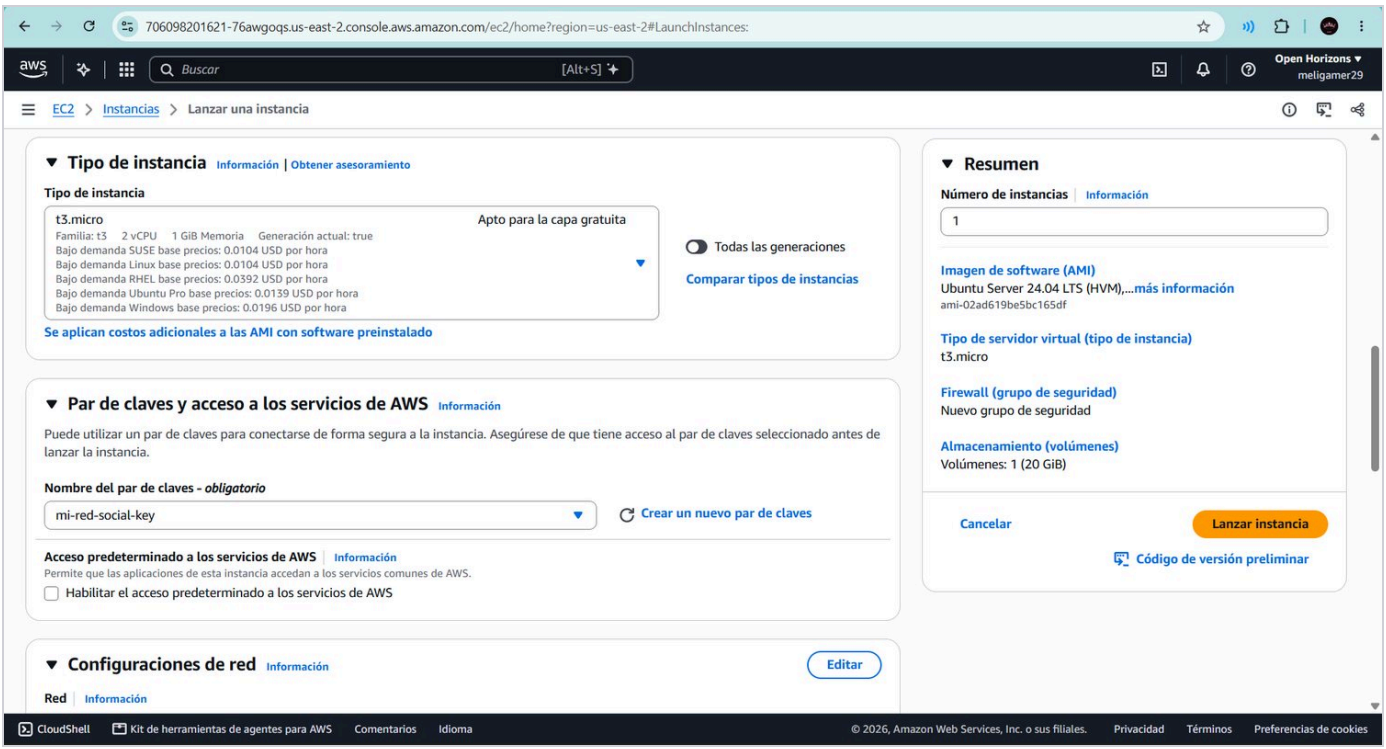


Figura 4. Tipo de instancia t3.micro (apto para la capa gratuita) y par de claves SSH mi-red-social-key.

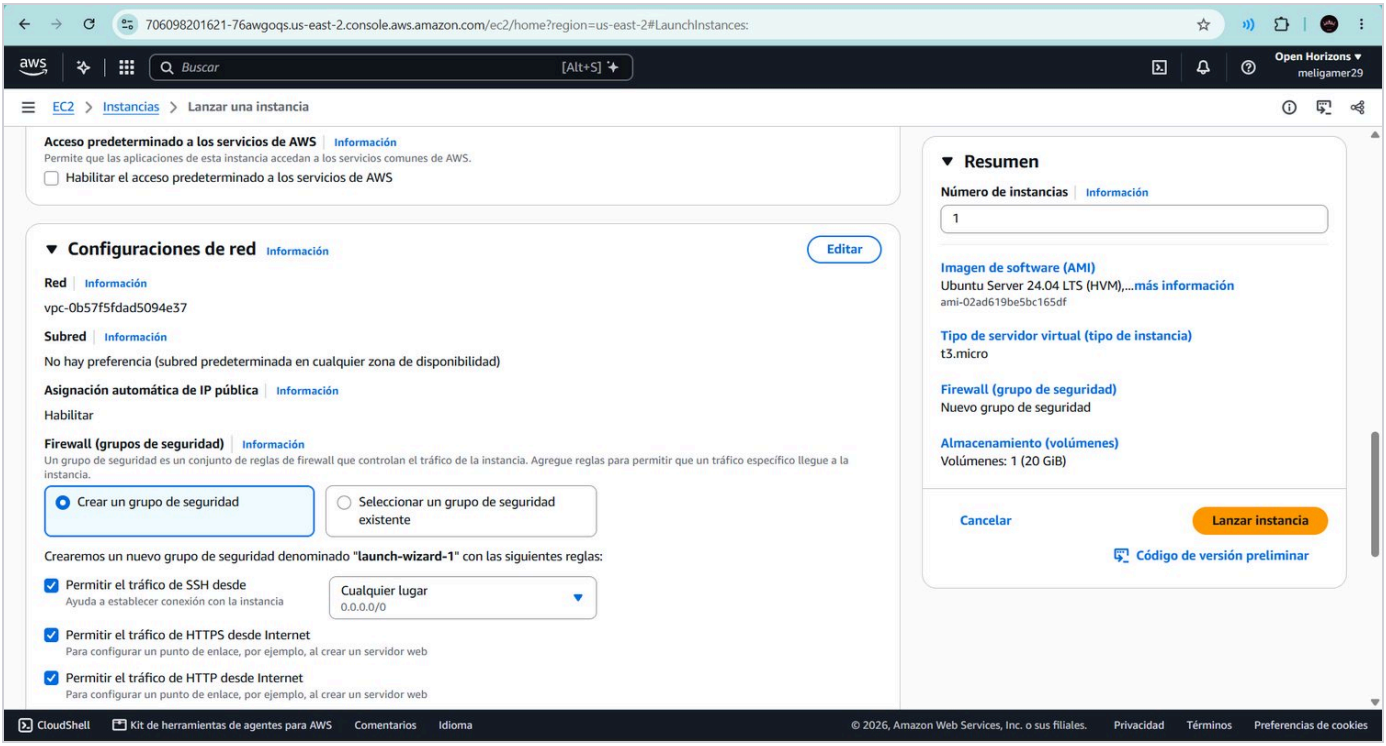


Figura 5. Configuración de red y Security Group: tráfico SSH, HTTP y HTTPS permitido.

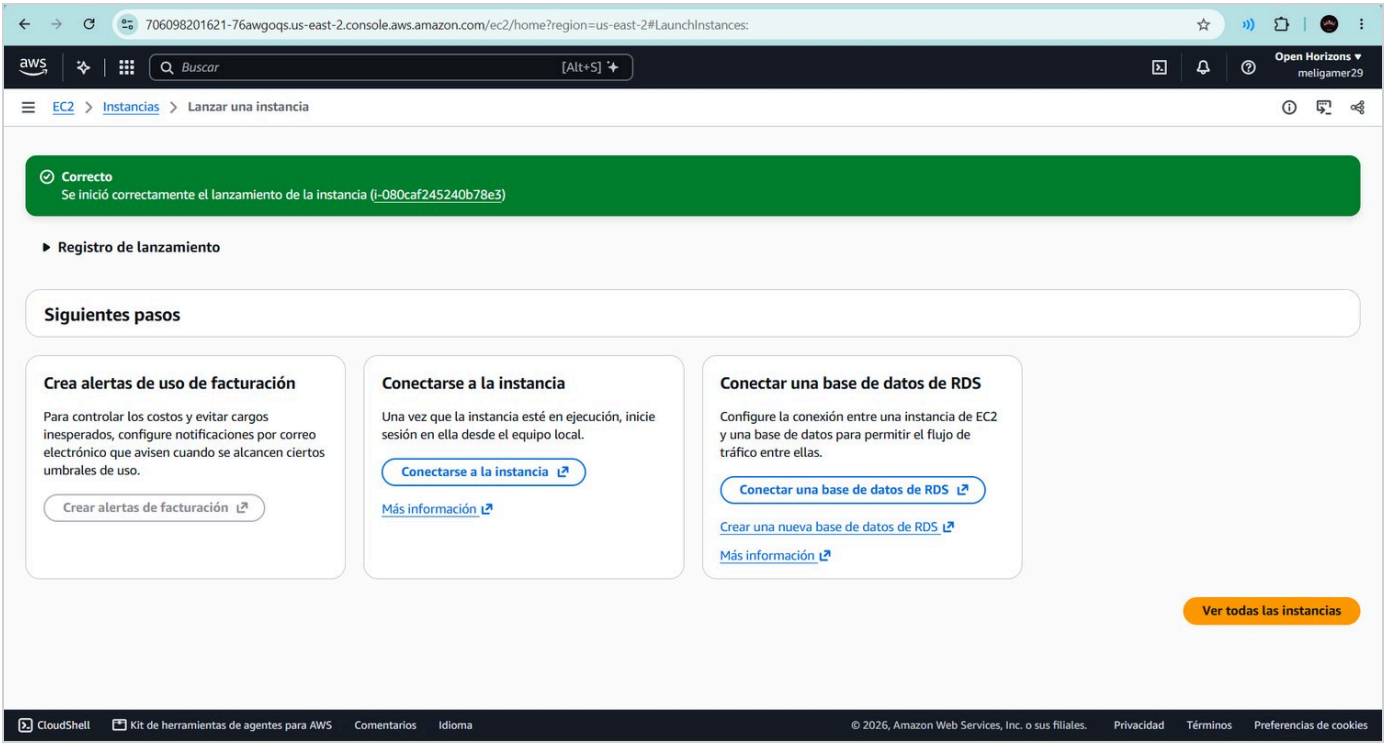


Figura 6. Lanzamiento exitoso de la instancia.

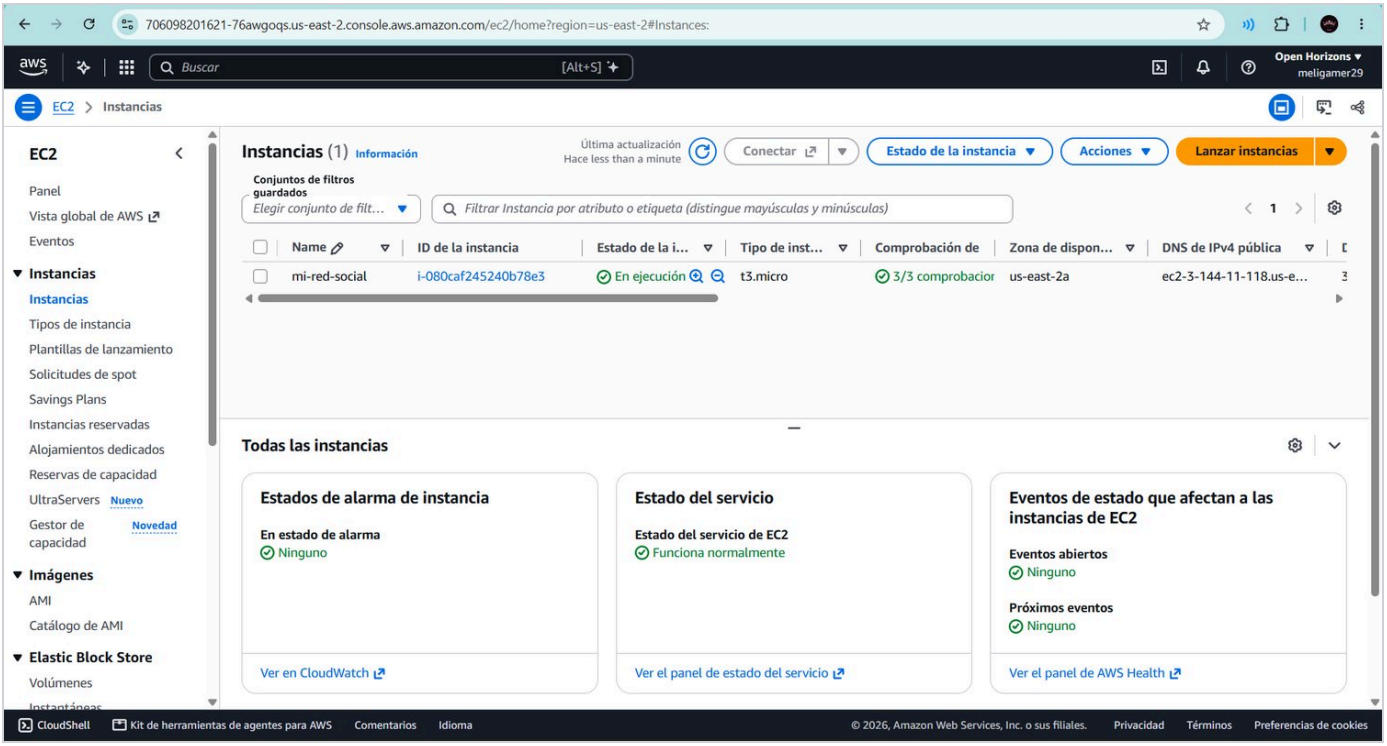


Figura 7. Instancia en ejecución (t3.micro, zona us-east-2a).

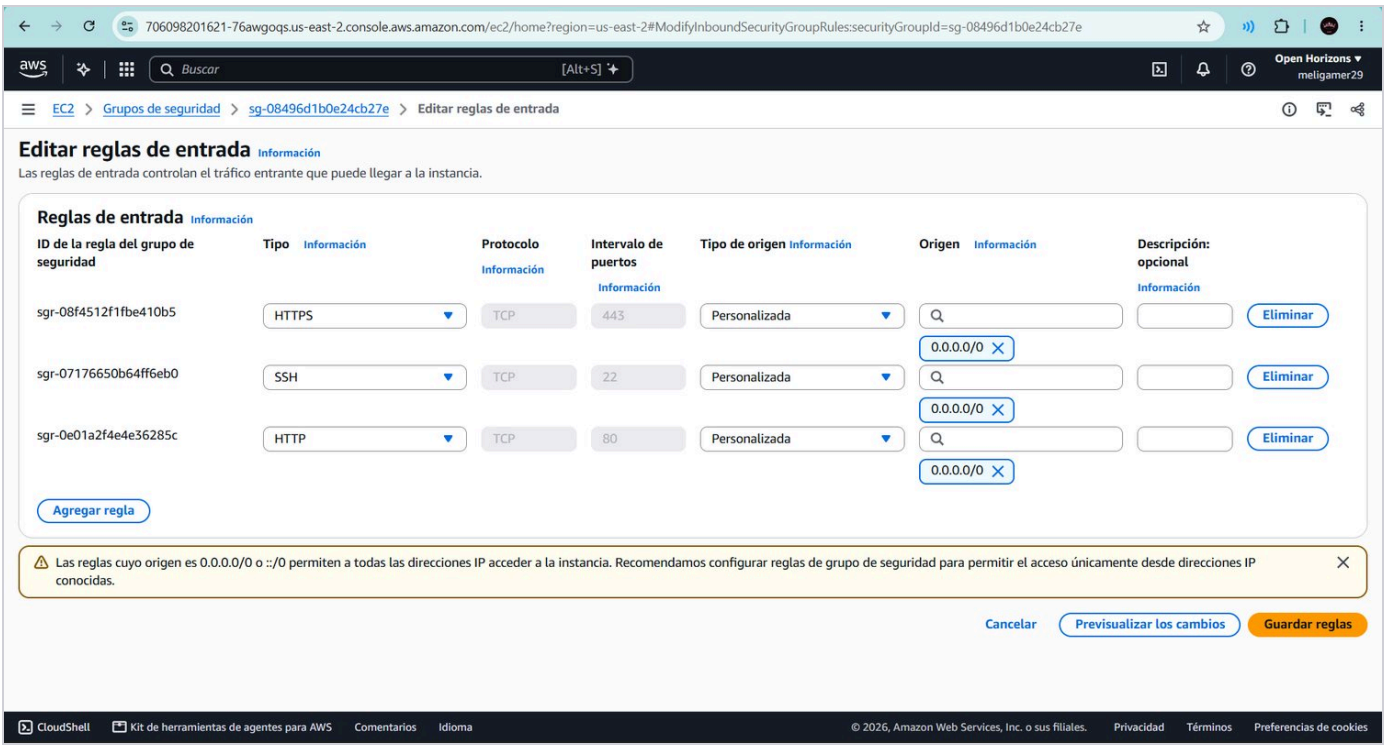


Figura 8. Reglas de entrada del Security Group: HTTPS (443), SSH (22) y HTTP (80).

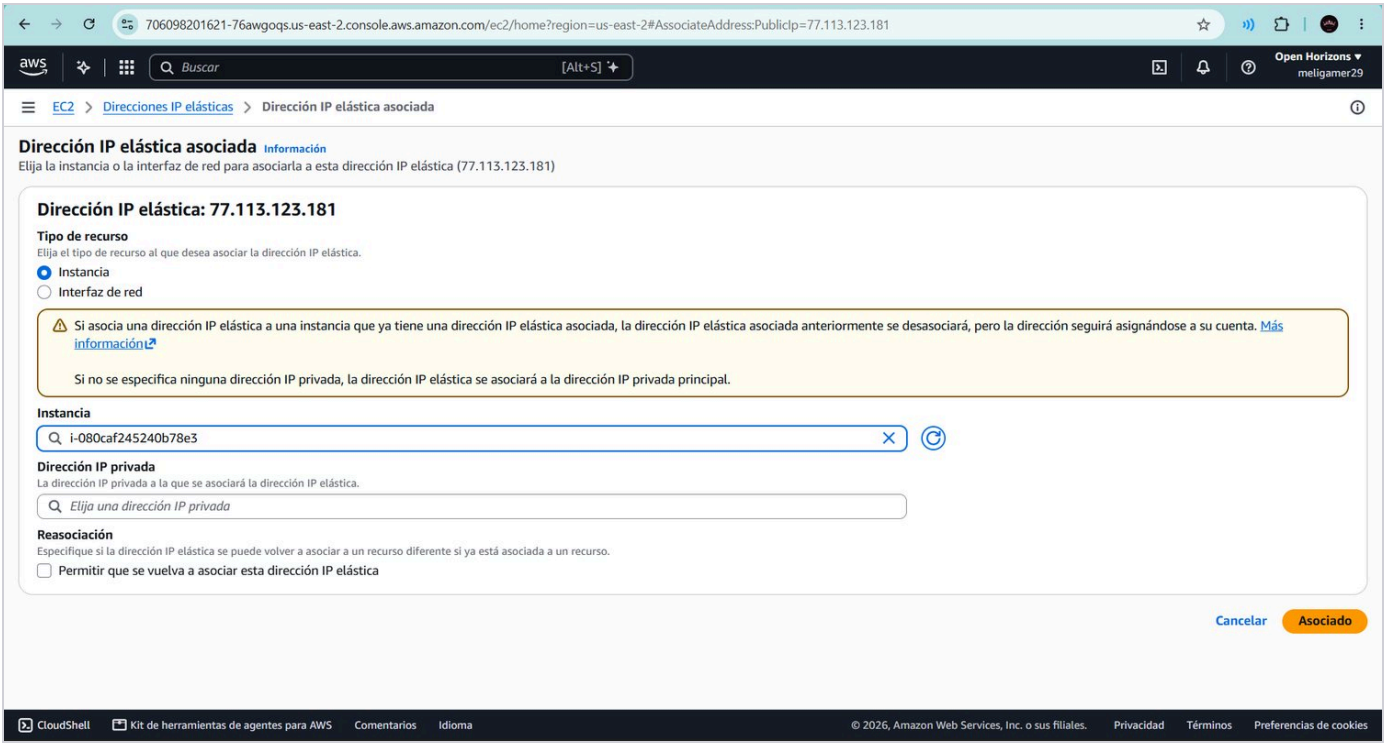


Figura 9. Asociación de la IP elástica 77.113.123.181 a la instancia.

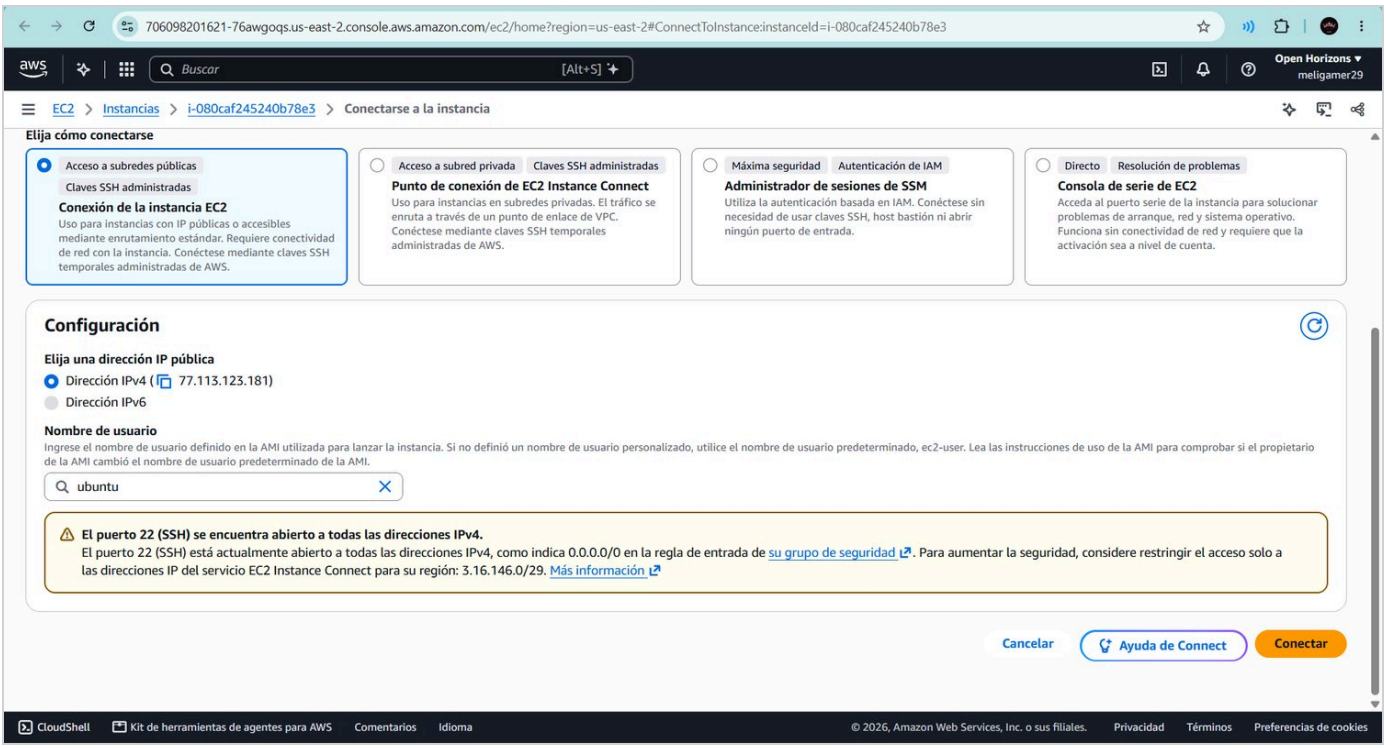


Figura 10. Conexión a la instancia con EC2 Instance Connect (usuario ubuntu).

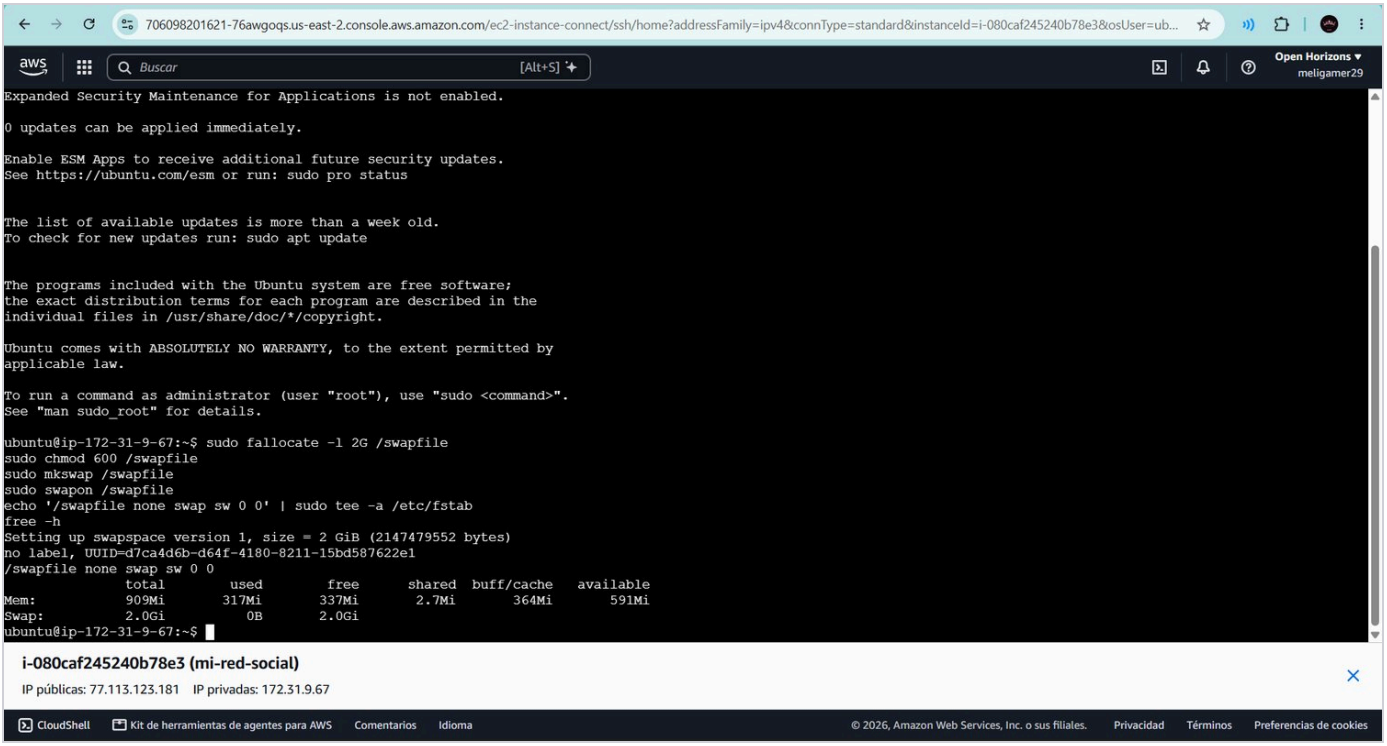


Figura 11. Creación de 2 GB de memoria swap para compensar la poca memoria de la t3.micro.

C. Aplicación en el servidor

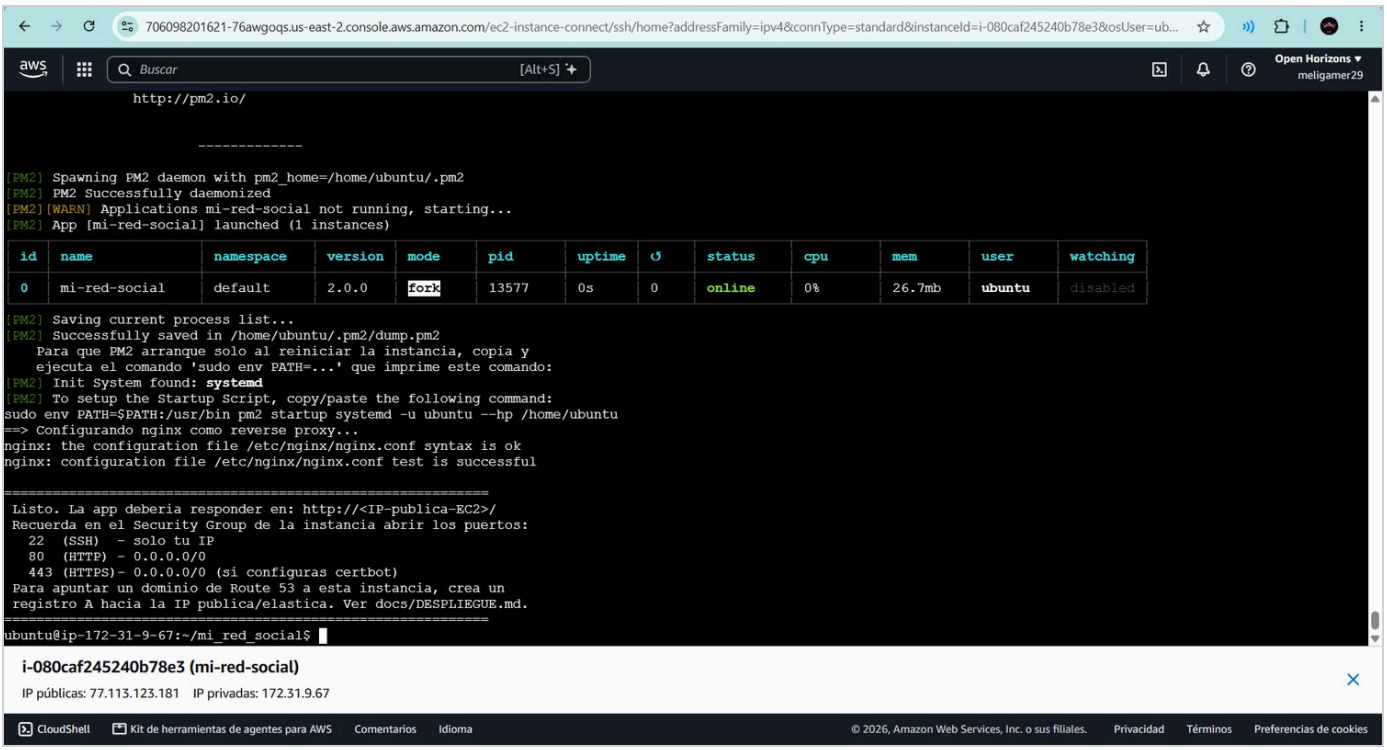


Figura 12. Script de despliegue: PM2 levanta la aplicación y nginx valida su configuración.

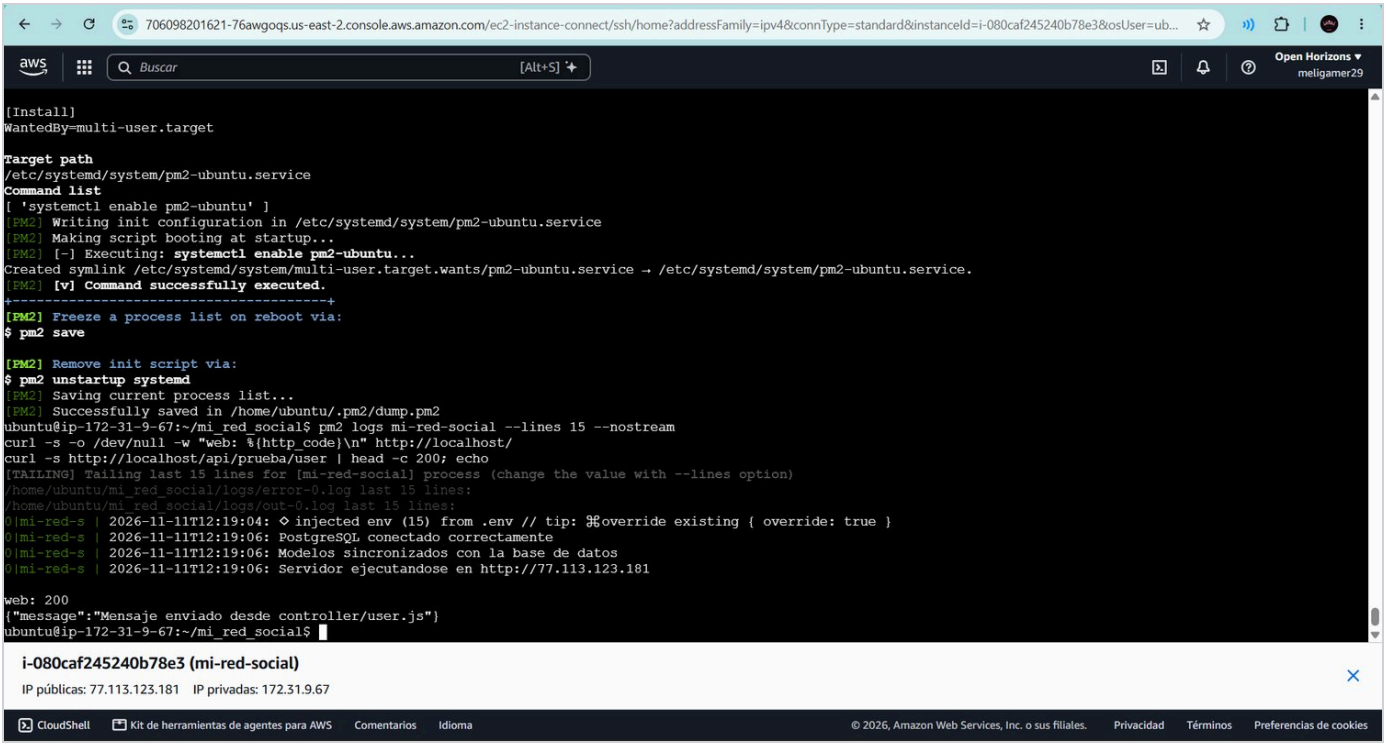


Figura 13. PM2 queda configurado para arrancar con el sistema; la API responde y se conecta a PostgreSQL.

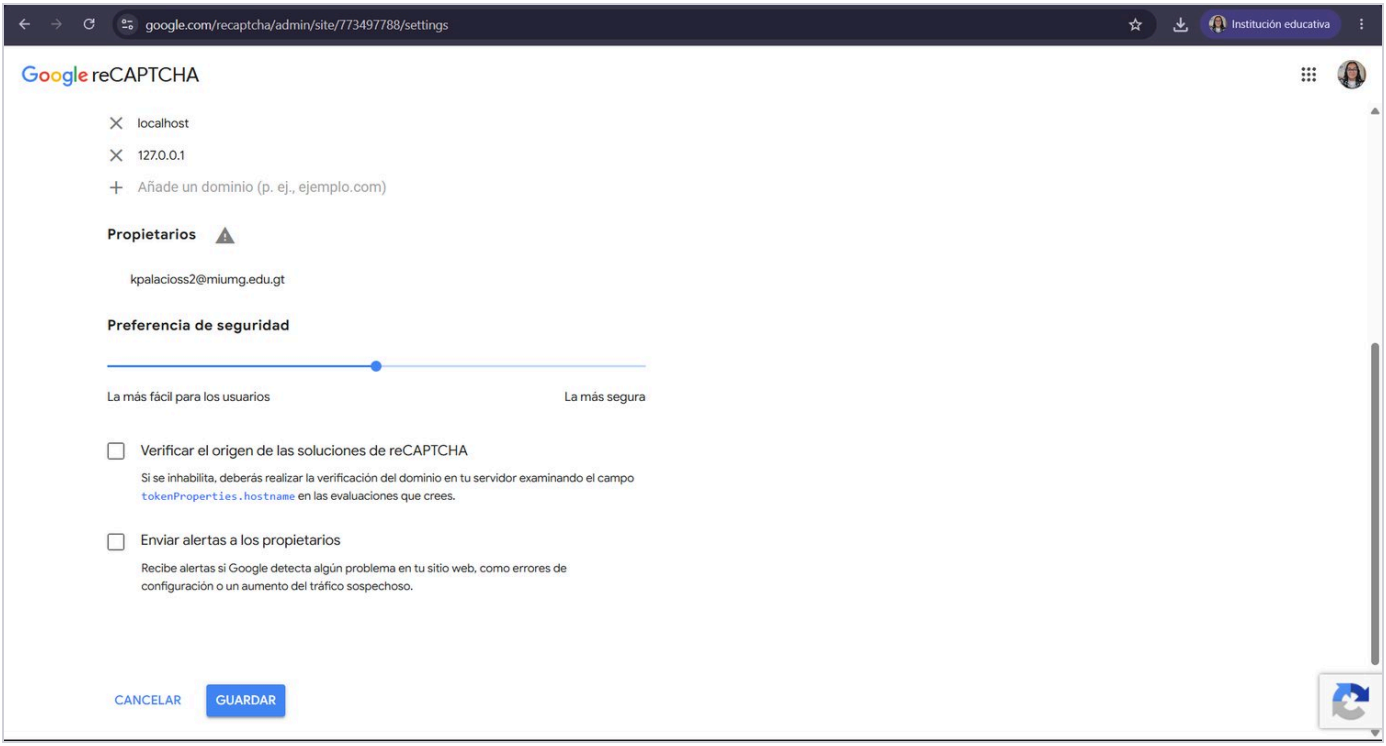


Figura 14. Configuración de dominios del sitio en reCAPTCHA v2.



Figura 15. Análisis facial con face-api.js: un rostro detectado, con edad y género estimados (se muestra solo el recuadro de resultados).

D. Almacenamiento en S3 y rol IAM de la instancia

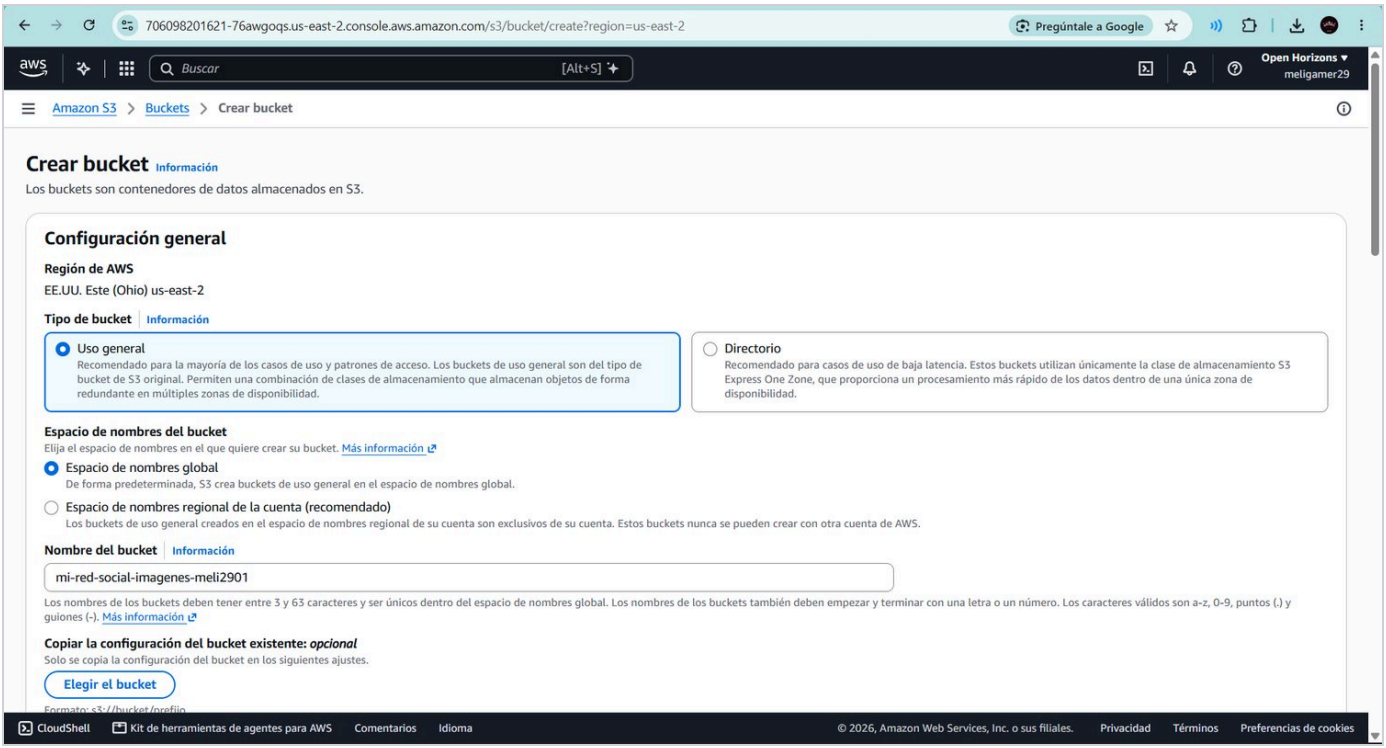


Figura 16. Creación del bucket mi-red-social-imagenes-meli2901 en Ohio.



Figura 17. Lectura pública habilitada en el bucket para mostrar las fotos en el feed.

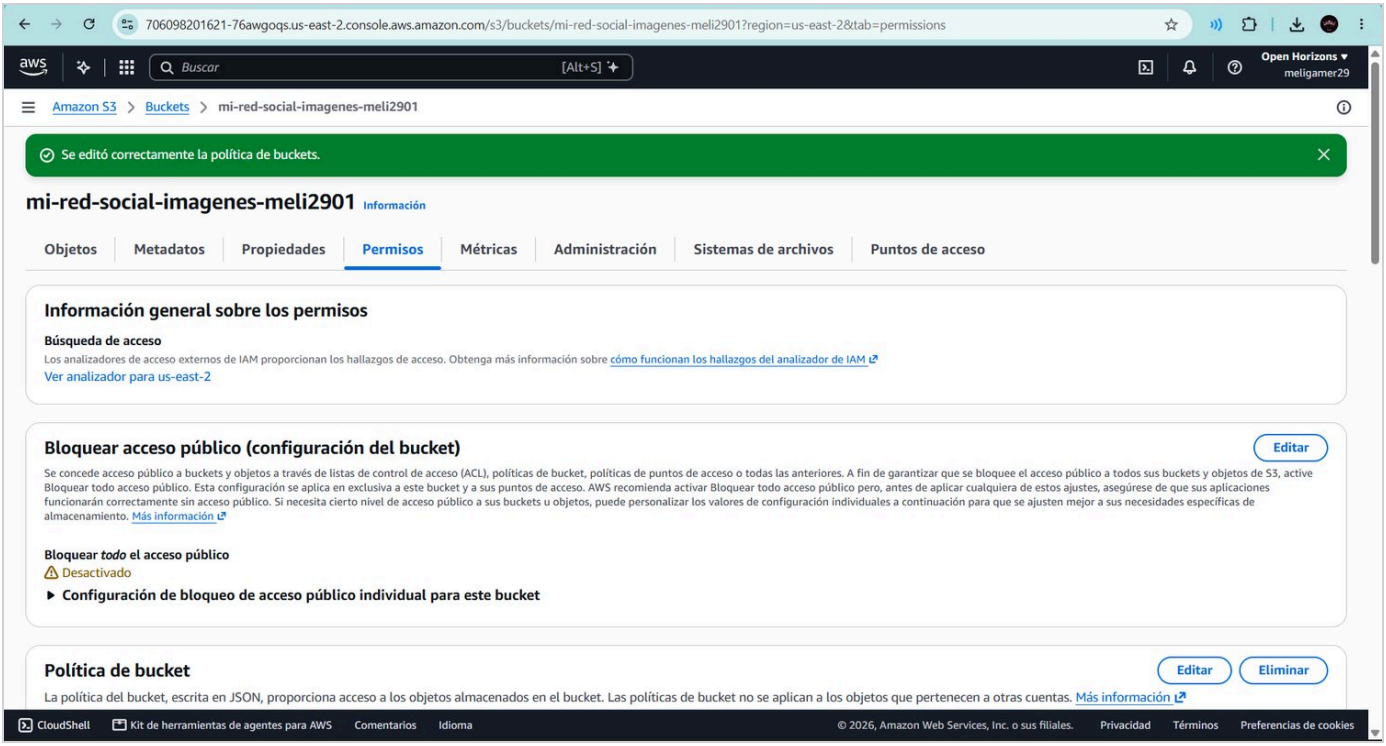


Figura 18. Política de bucket guardada correctamente.

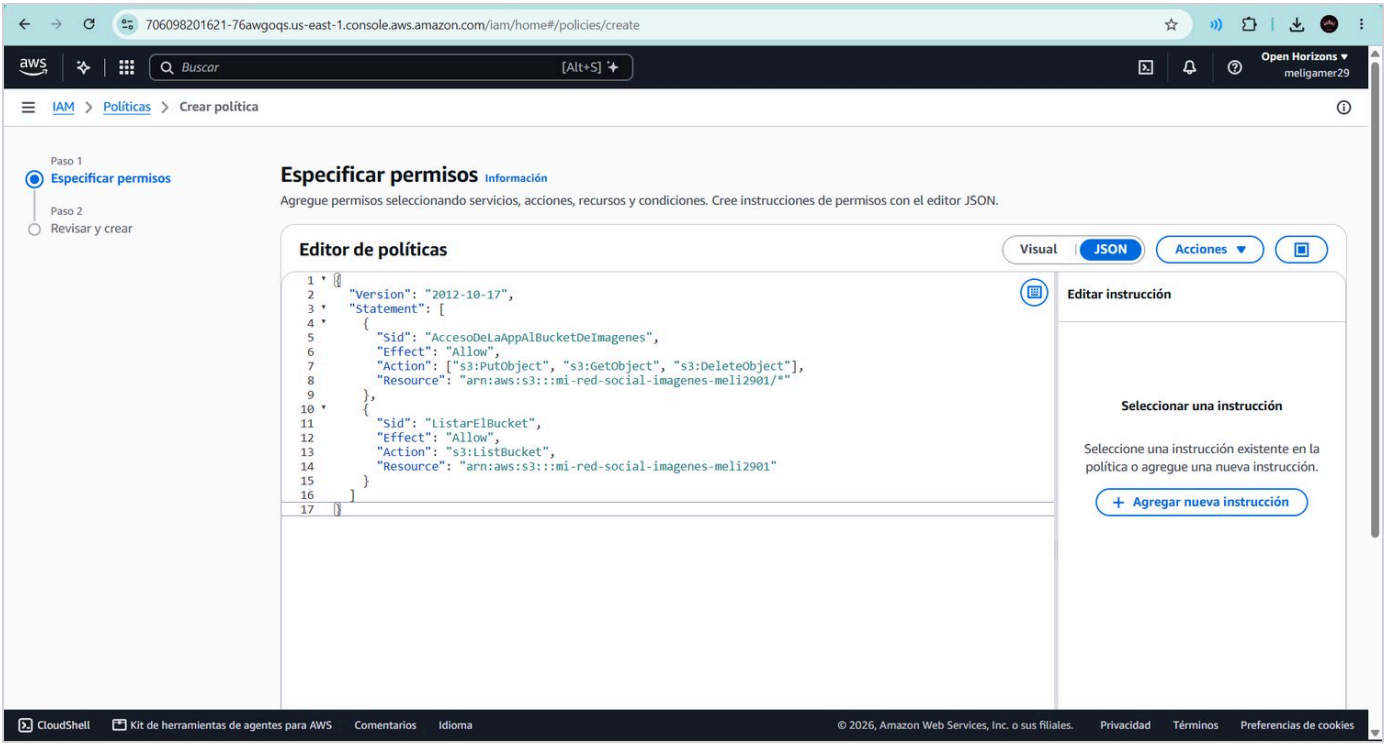


Figura 19. Política IAM de la aplicación: permisos de escritura y lectura solo sobre ese bucket.

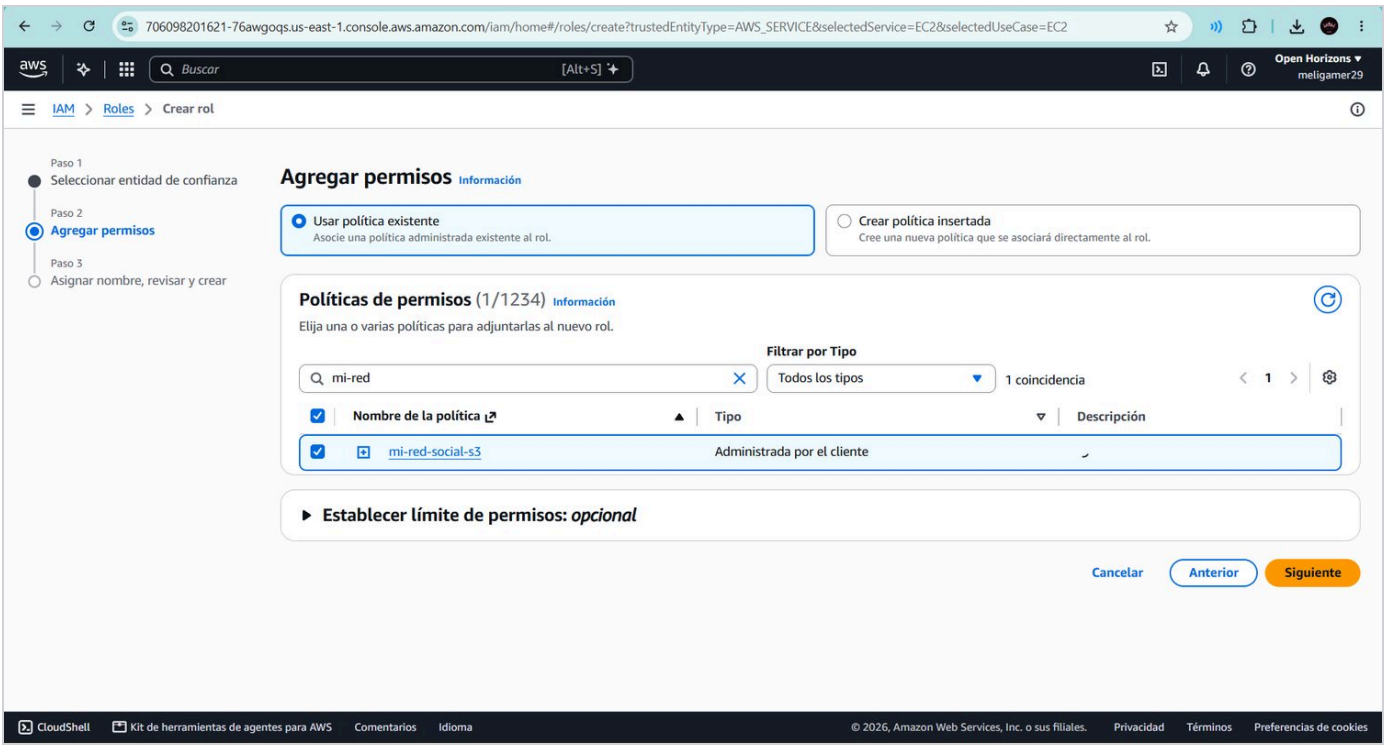


Figura 20. Creación del rol IAM de la instancia con la política de S3 adjunta.

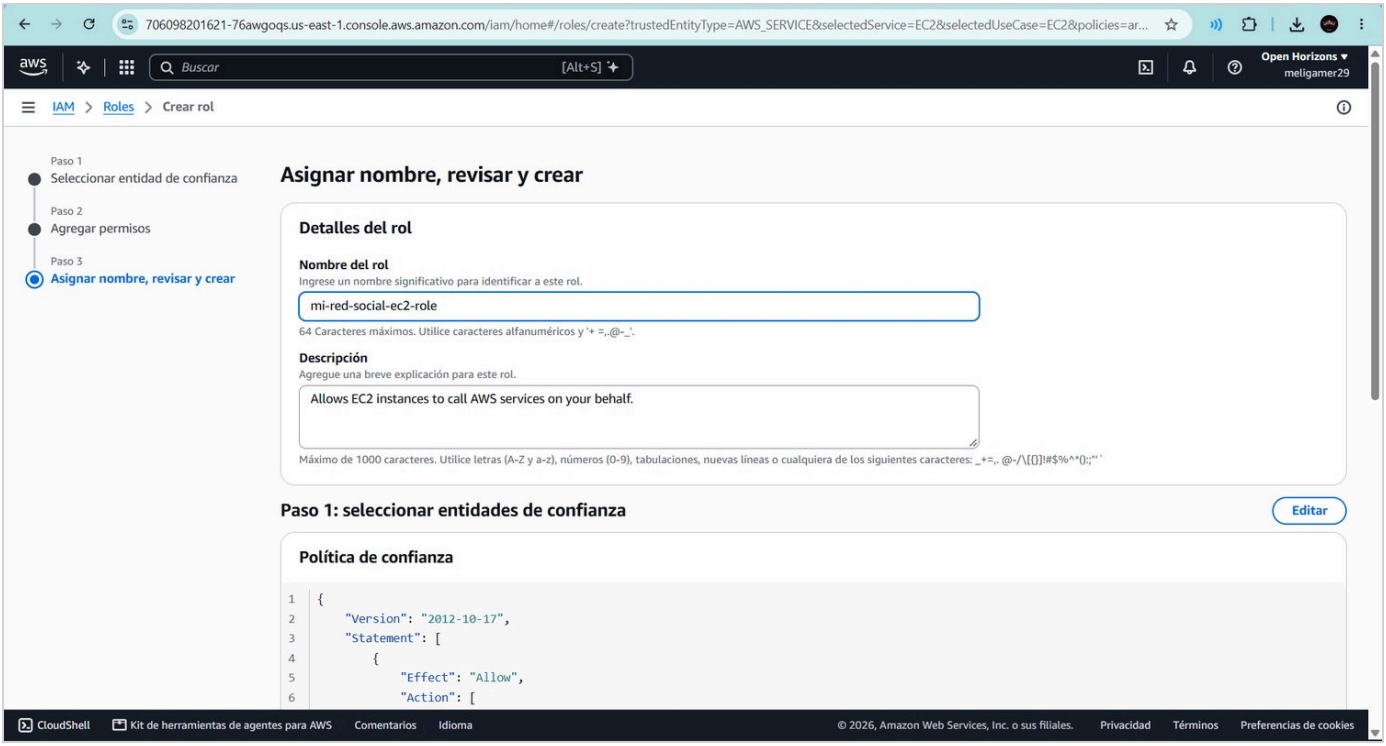


Figura 21. Nombre del rol: mi-red-social-ec2-role.

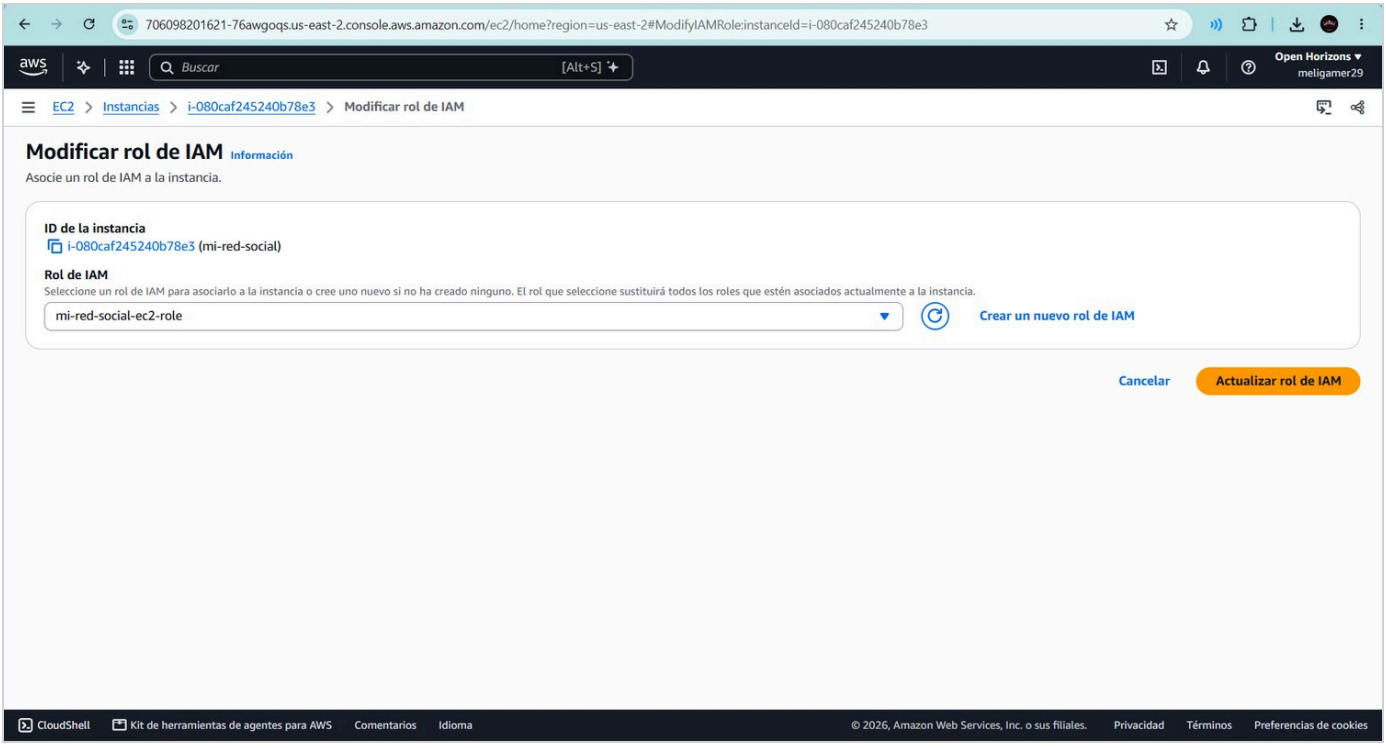


Figura 22. Asociación del rol IAM a la instancia EC2.

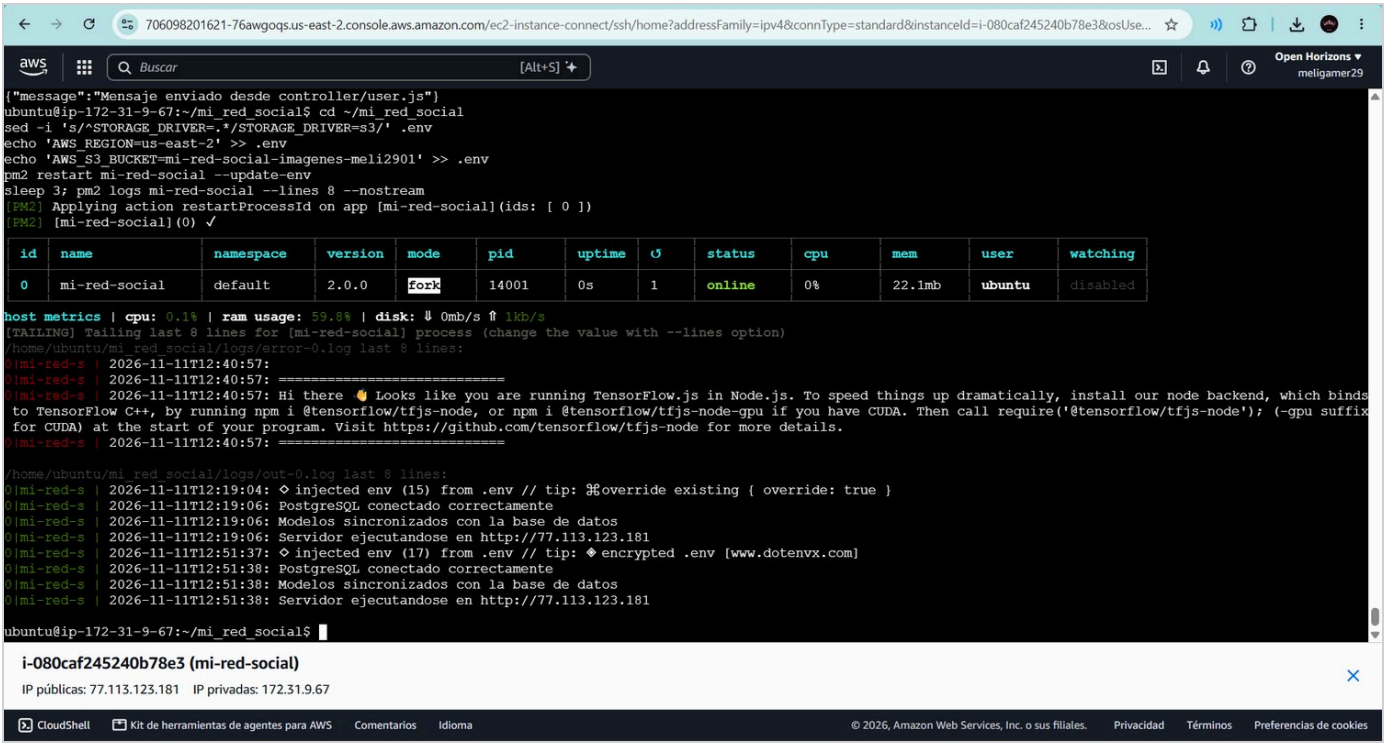


Figura 23. La aplicación se reinicia con STORAGE_DRIVER=s3 y el bucket; no se guardan claves de AWS en el servidor.

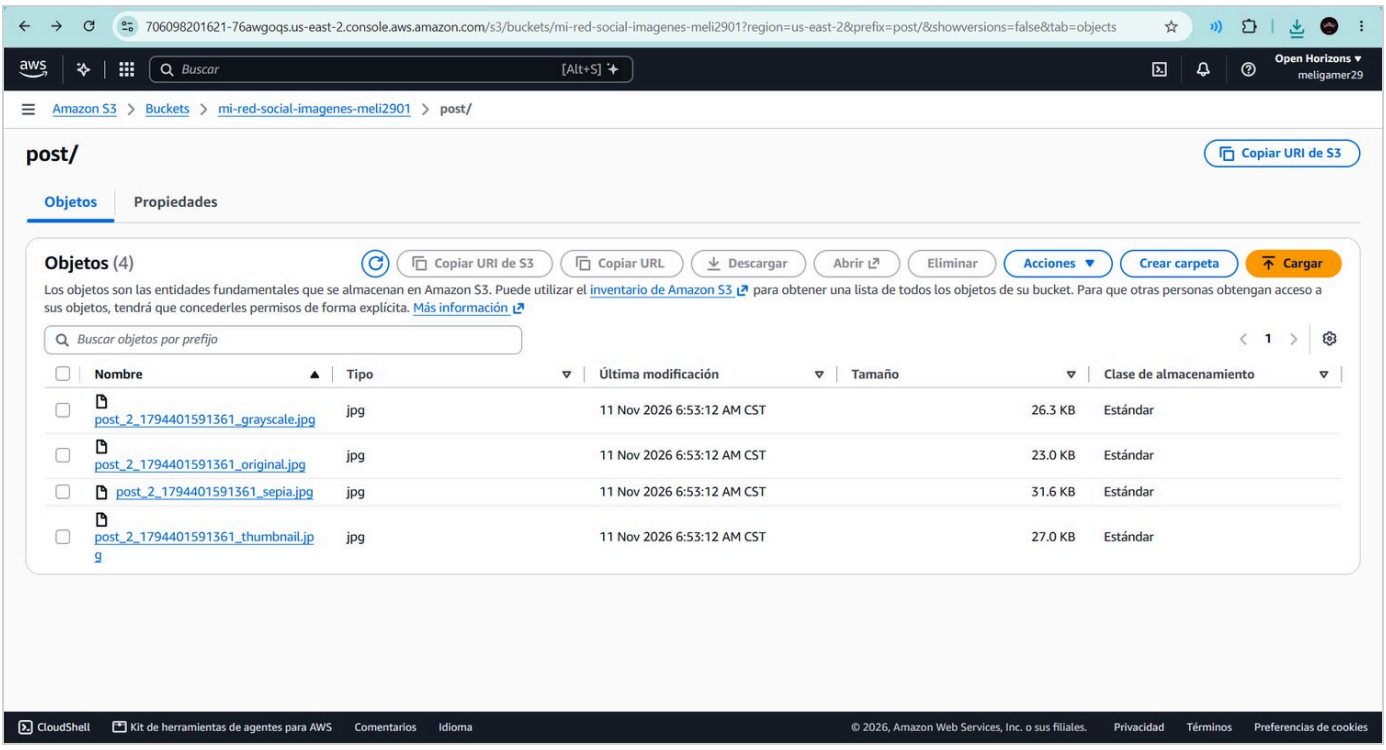


Figura 24. Resultado: el bucket guarda las cuatro versiones de la foto (original, miniatura, blanco y negro y sepia).

E. Acceso del catedrático (IAM con permisos limitados)

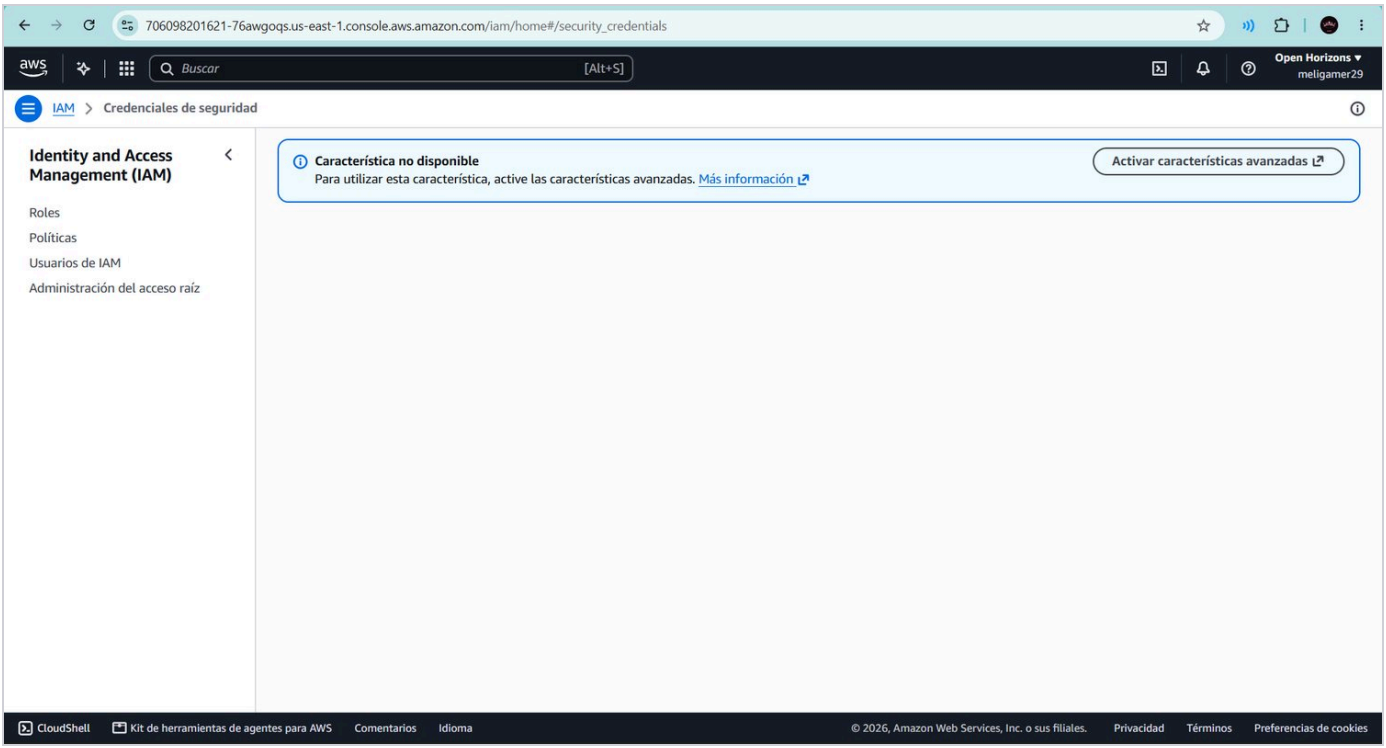


Figura 25. El plan gratuito no habilita las credenciales de consola; el acceso del catedrático se entrega con clave de acceso.

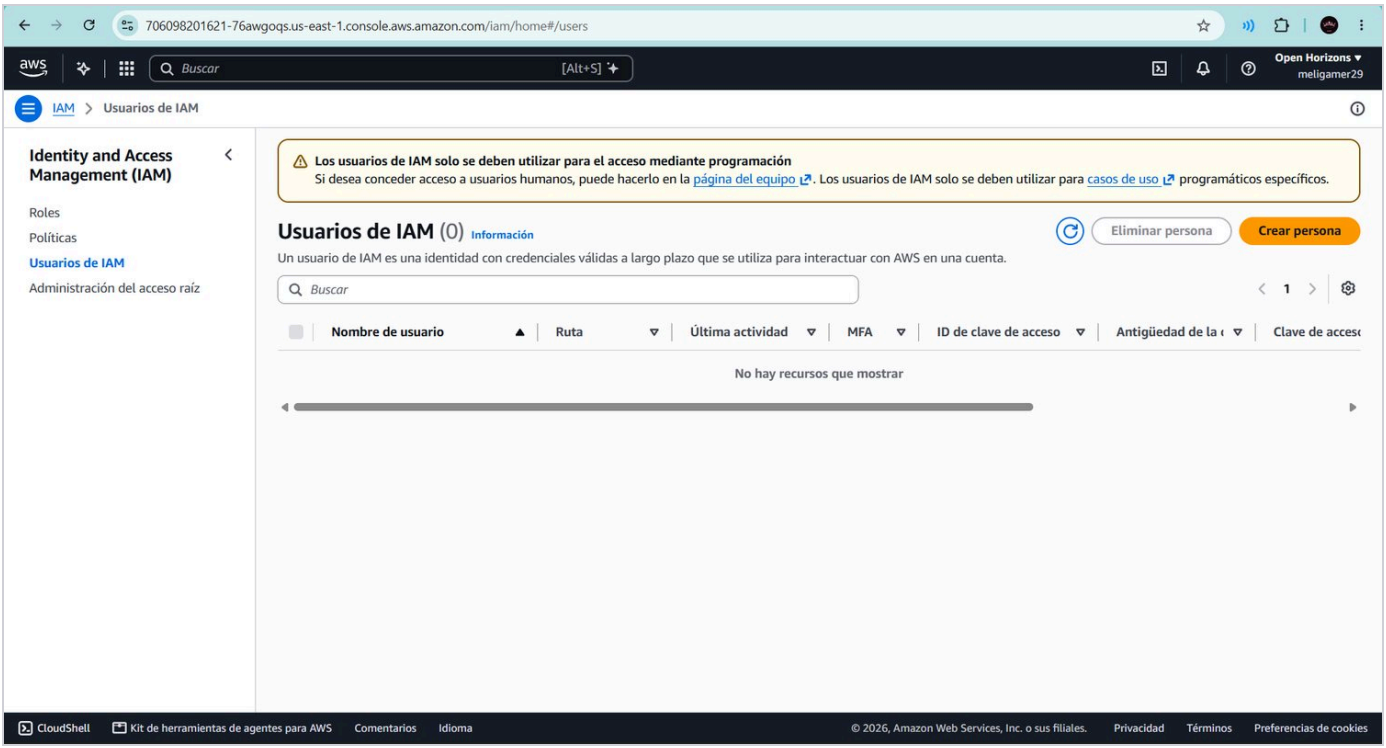


Figura 26. Aviso de IAM: los usuarios IAM se orientan al acceso programático.

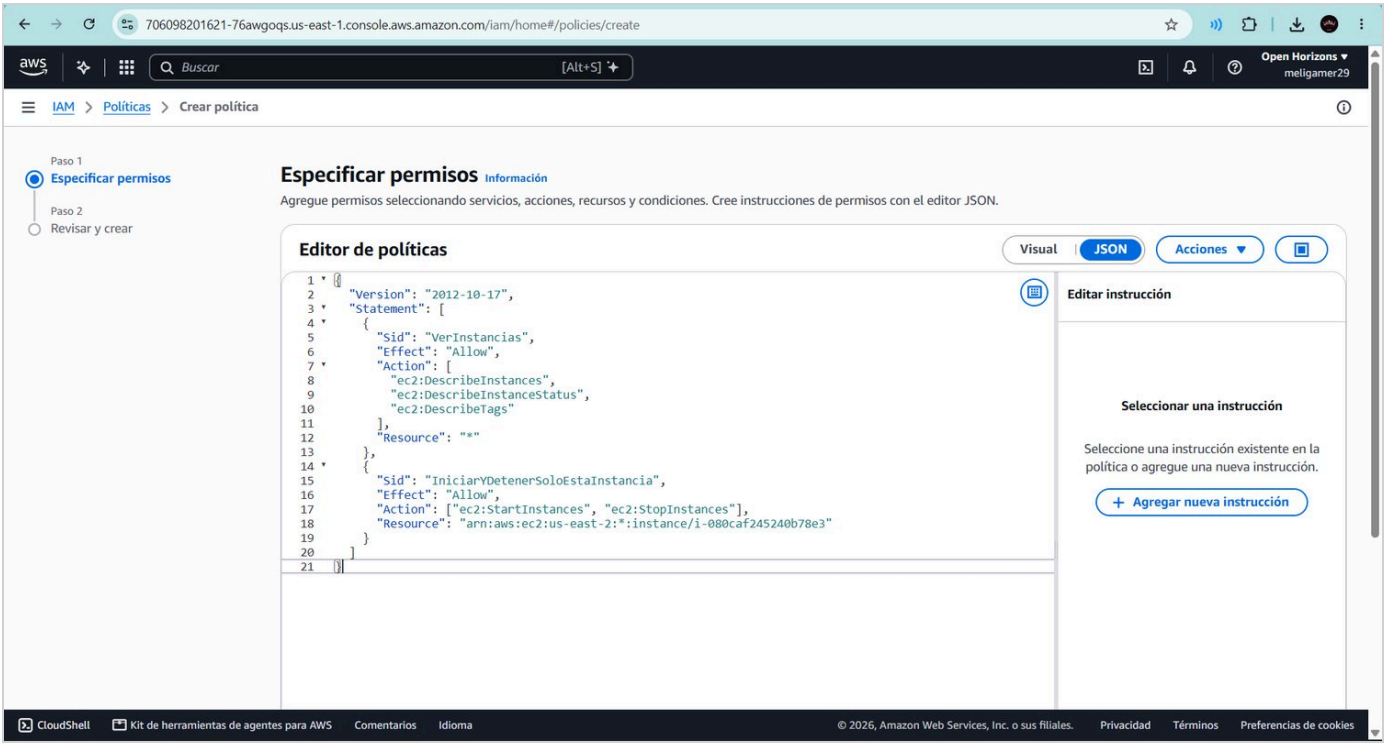


Figura 27. Política catedratico-ec2-start-stop: ver instancias, e iniciar y detener solo la instancia del proyecto.

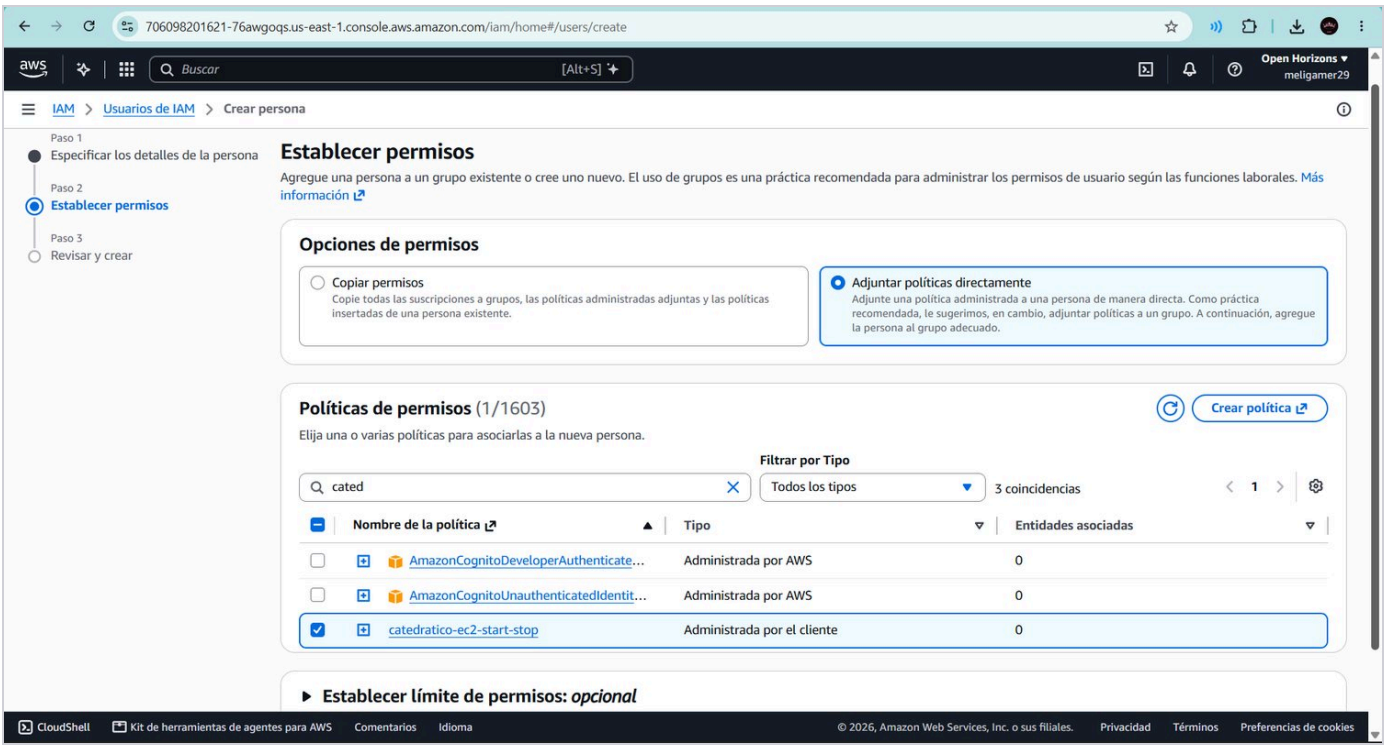


Figura 28. Política asociada al usuario catedratico.

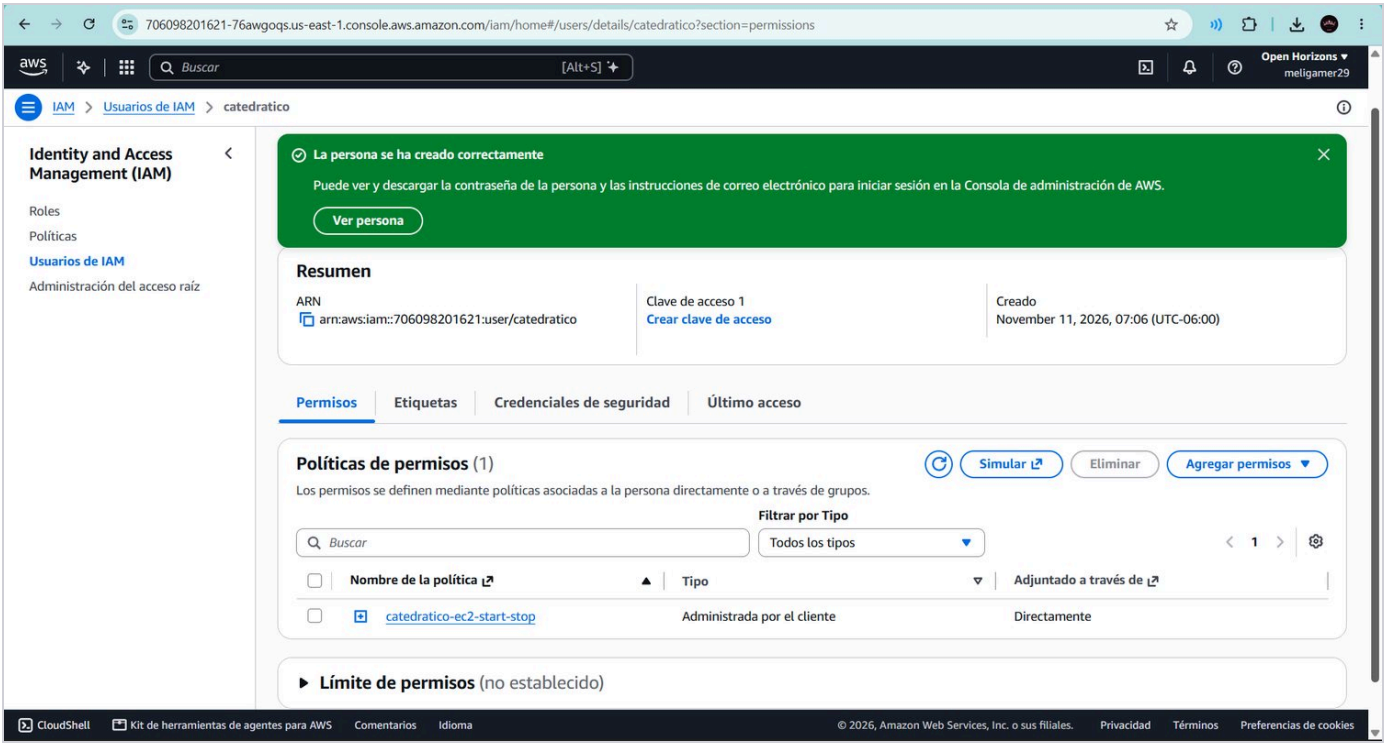


Figura 29. Usuario catedratico creado con la política de permisos limitados.

F. Aplicación en funcionamiento (https://miredsocialweb.duckdns.org)

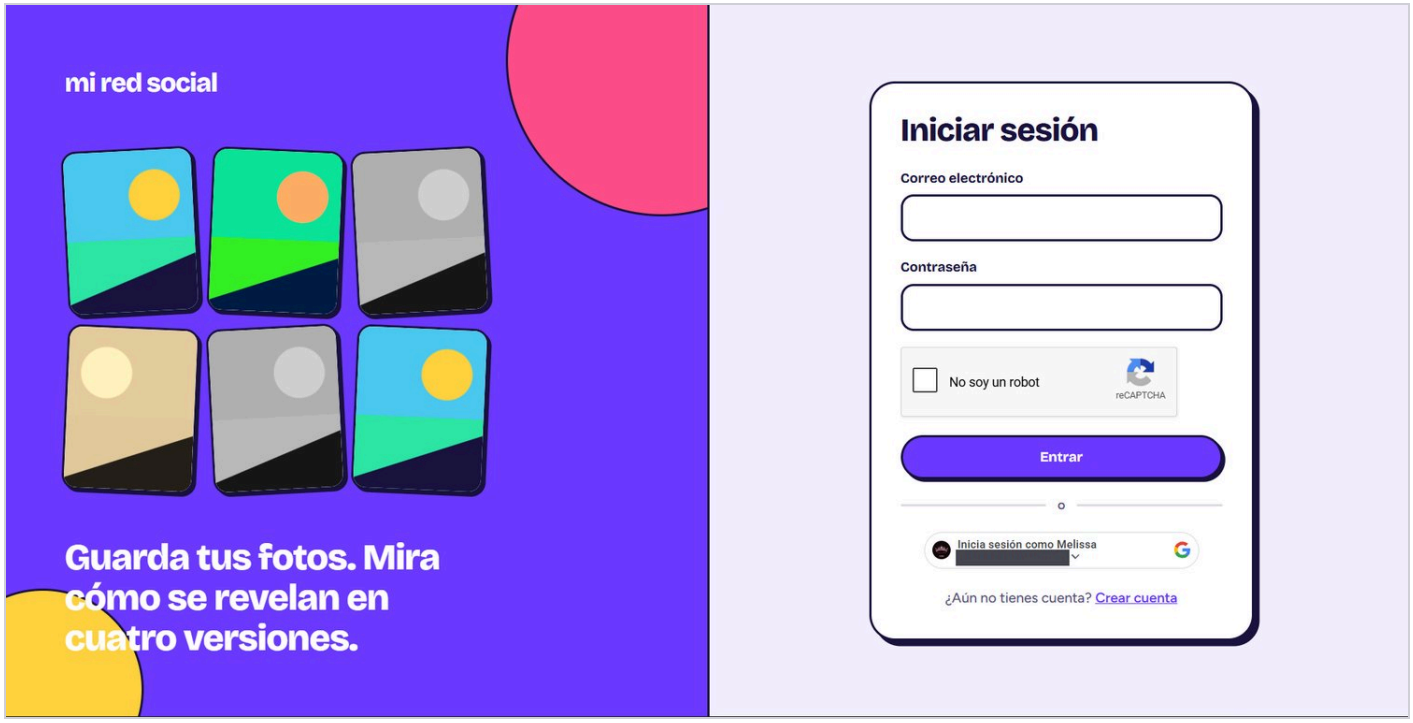


Figura 30. Inicio de sesión con correo y contraseña, reCAPTCHA e inicio de sesión con Google.

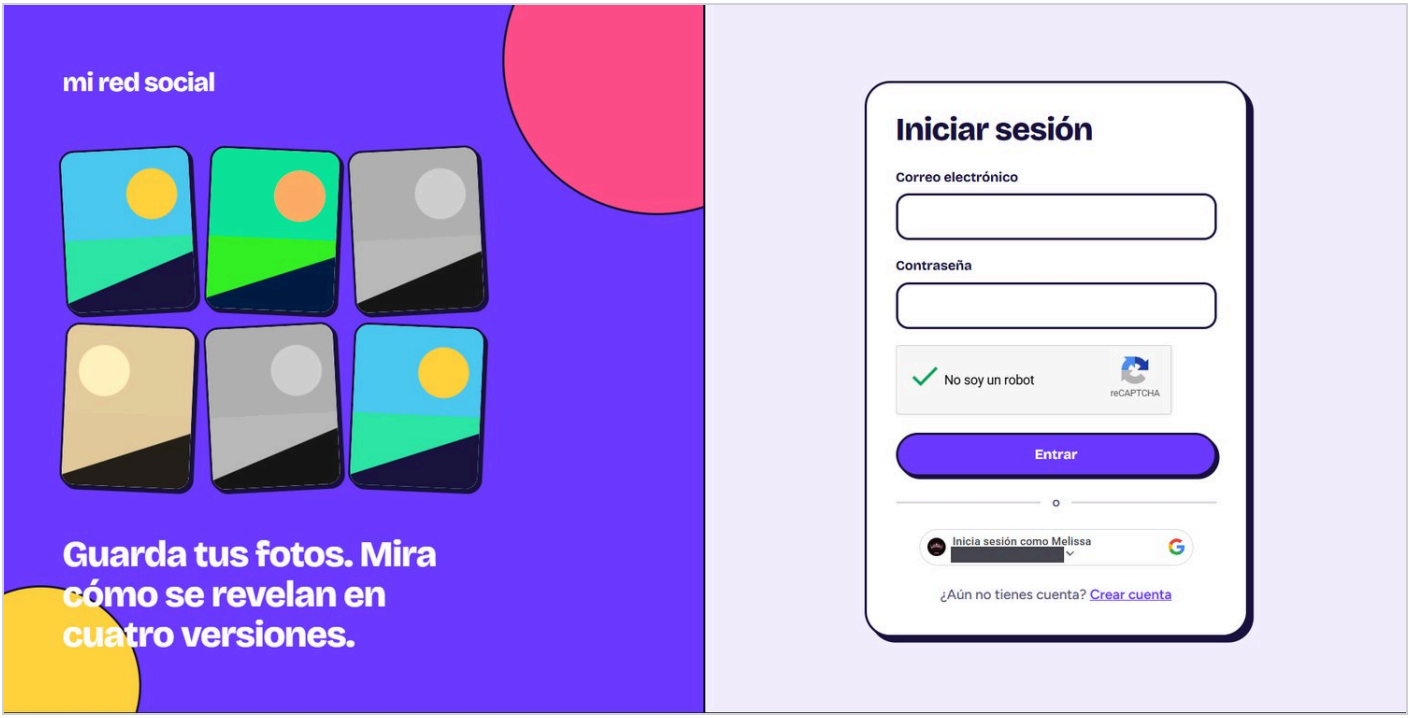


Figura 31. reCAPTCHA validado correctamente antes de entrar.

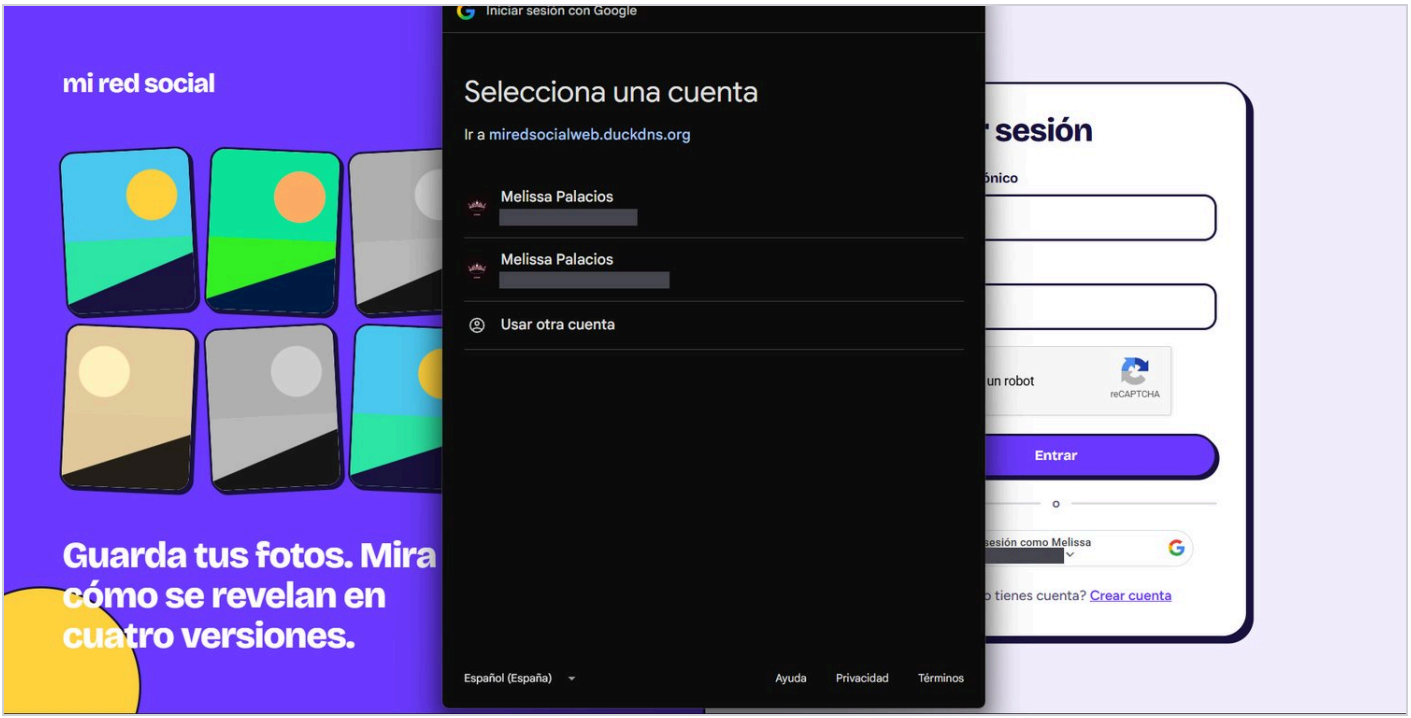


Figura 32. Selector de cuentas de Google sobre el dominio miredsocialweb.duckdns.org.

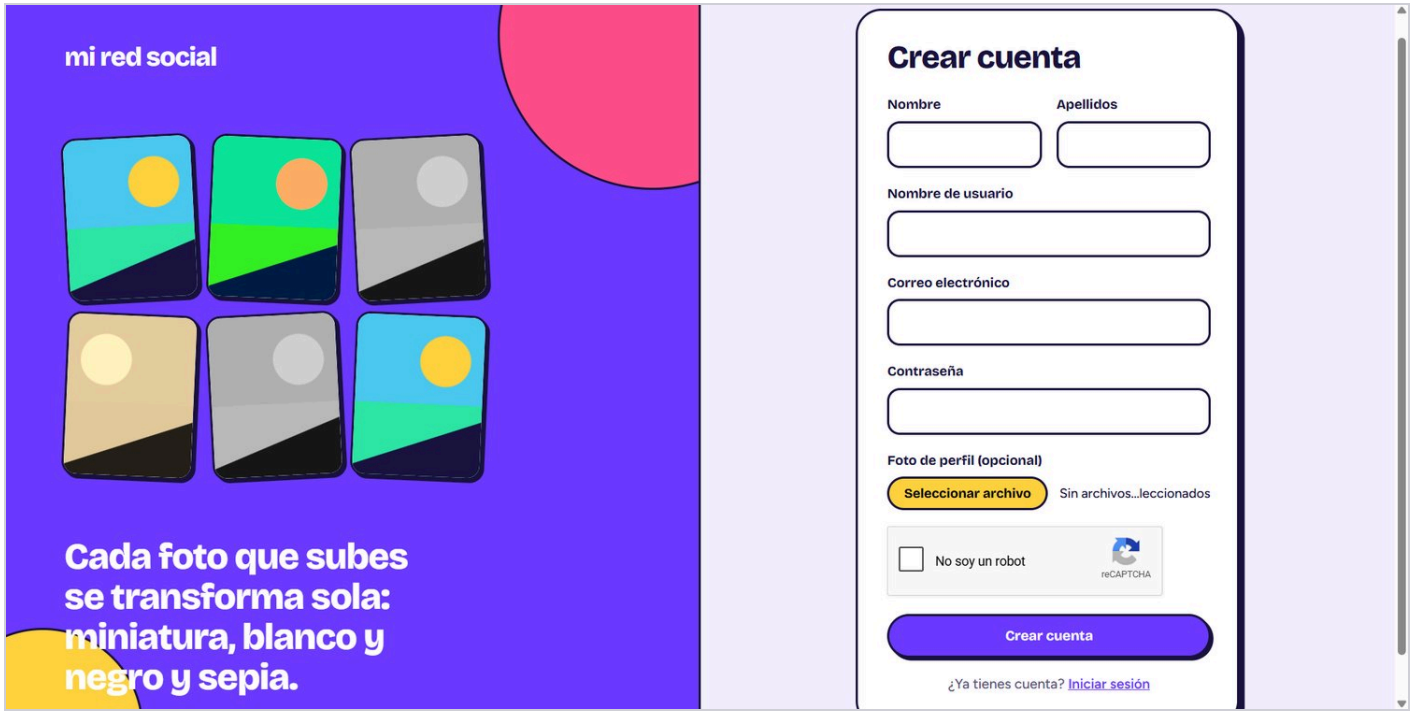


Figura 33. Formulario de registro con foto de perfil opcional y reCAPTCHA.

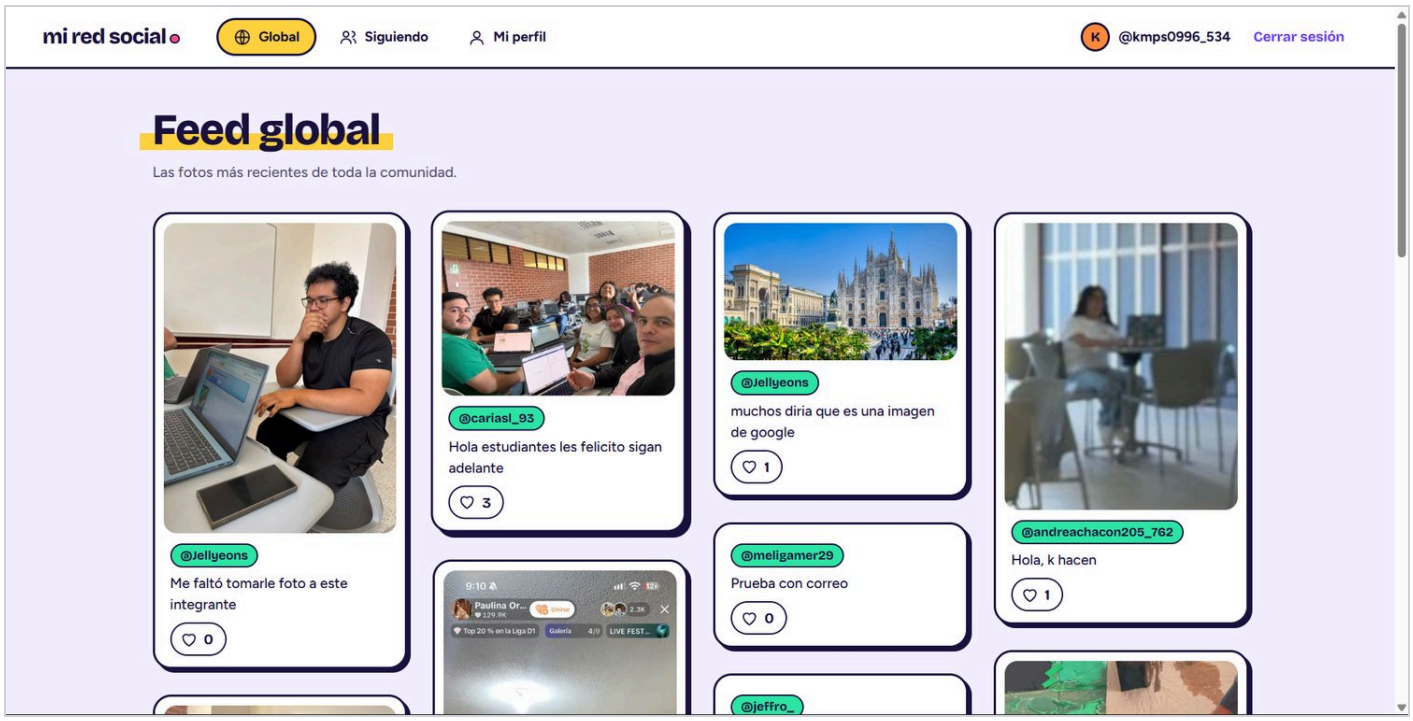


Figura 34. Feed global con las fotos más recientes de la comunidad.

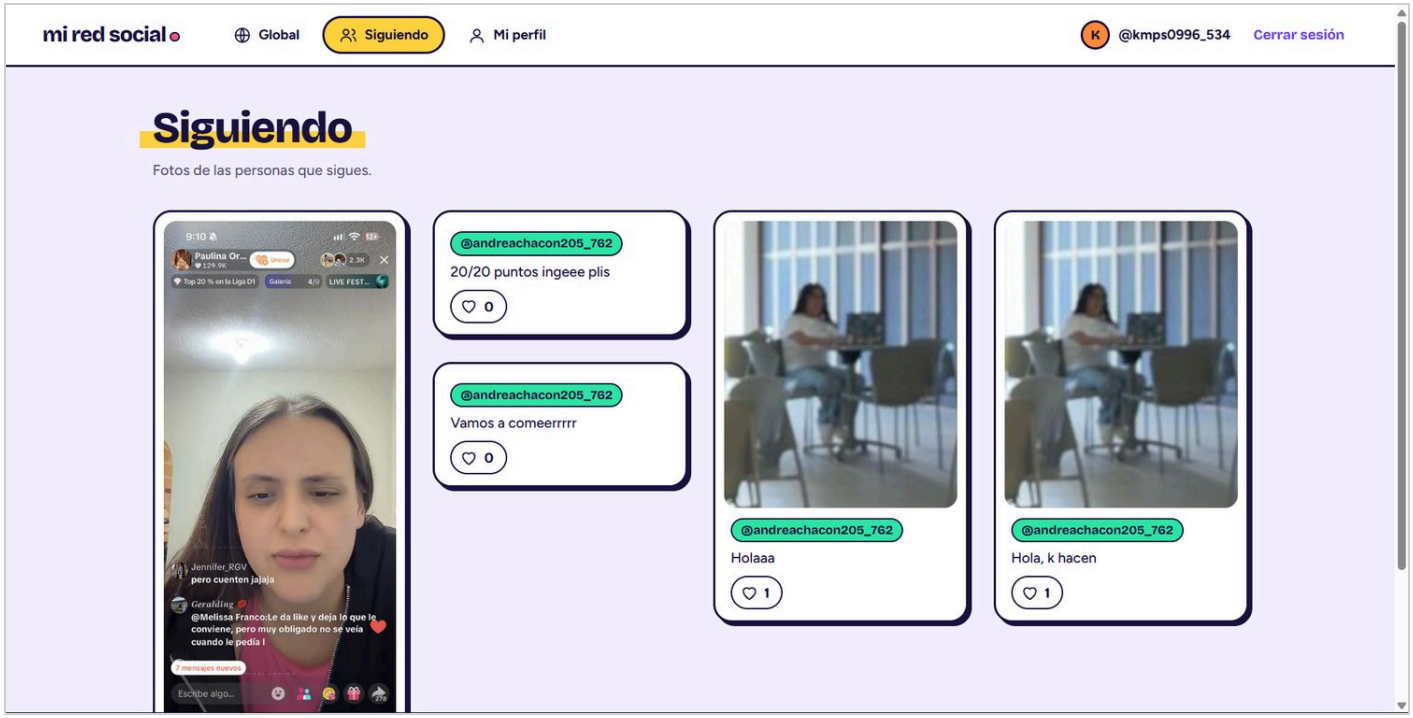


Figura 35. Pestaña Siguiendo: solo las publicaciones de las personas que sigues.

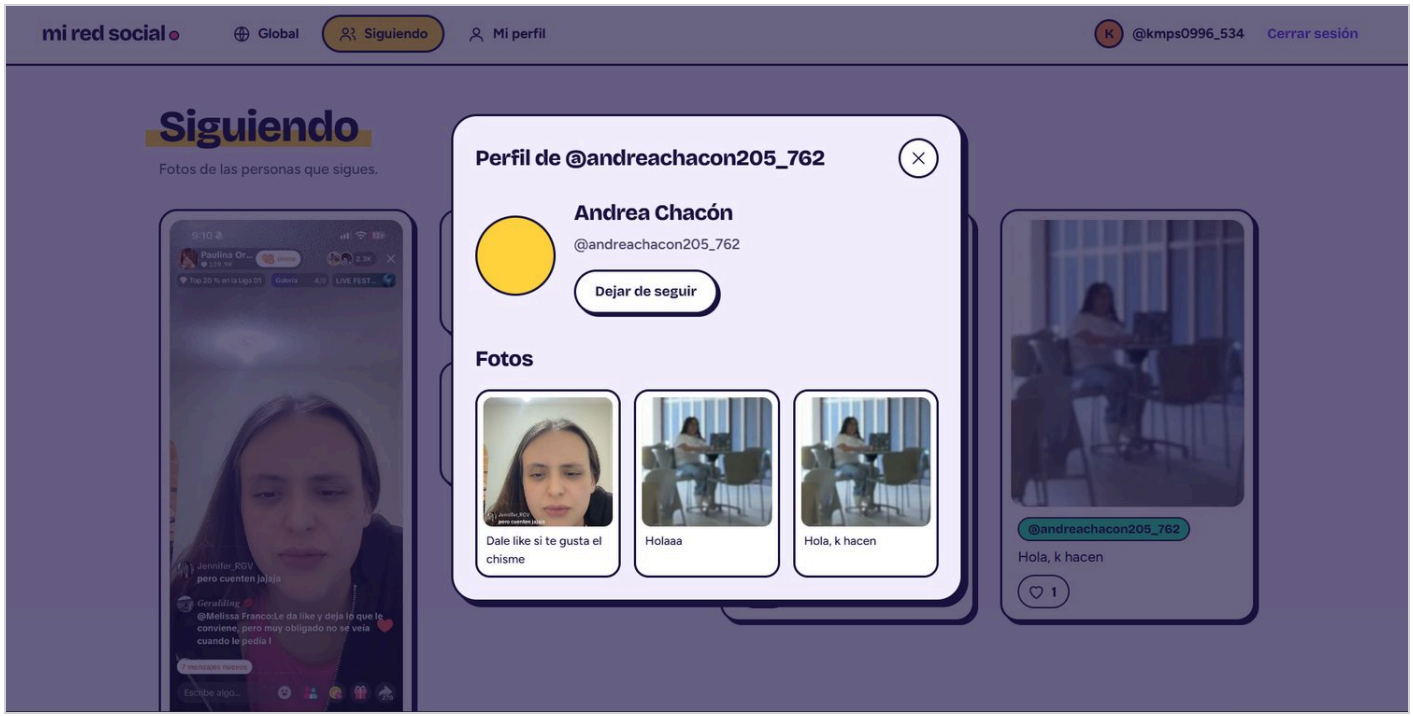


Figura 36. Mini-perfil de otro usuario con el botón Dejar de seguir y sus fotos.

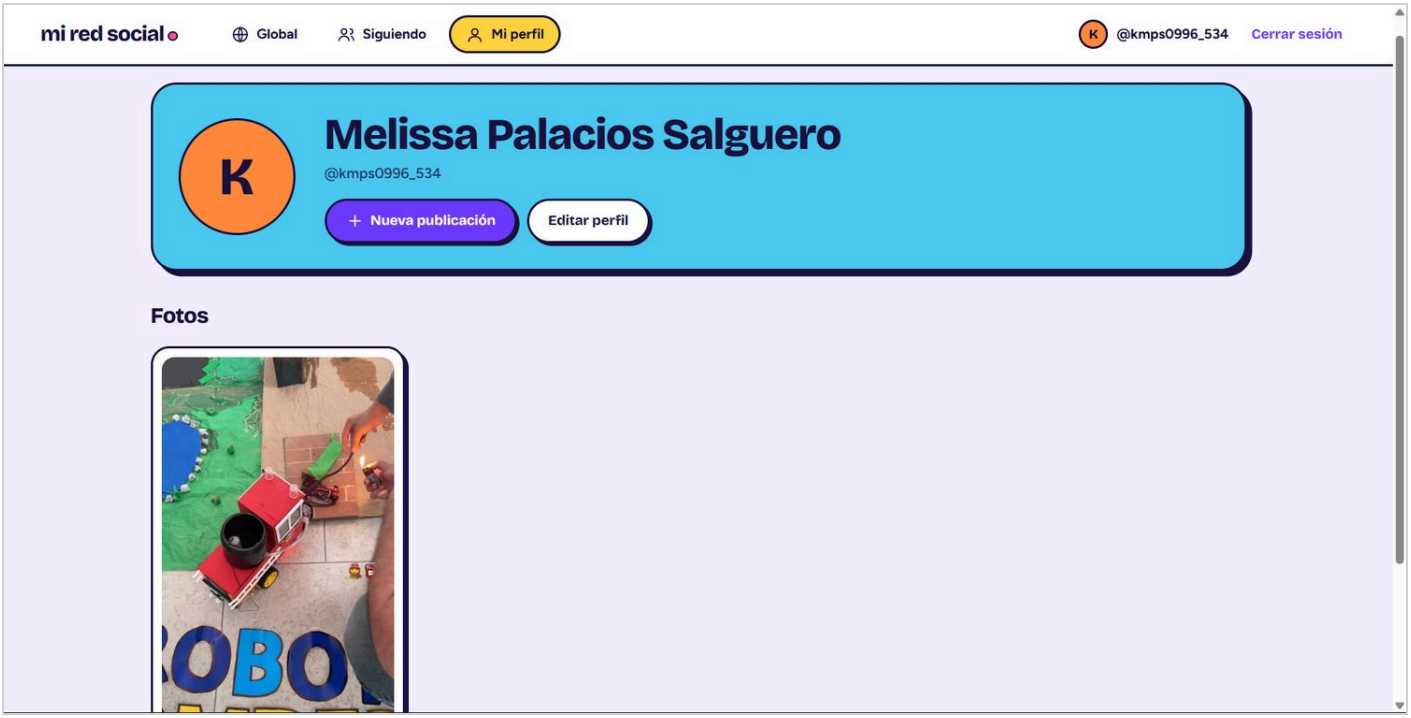


Figura 37. Mi perfil: datos de la cuenta, nueva publicación y edición del perfil.

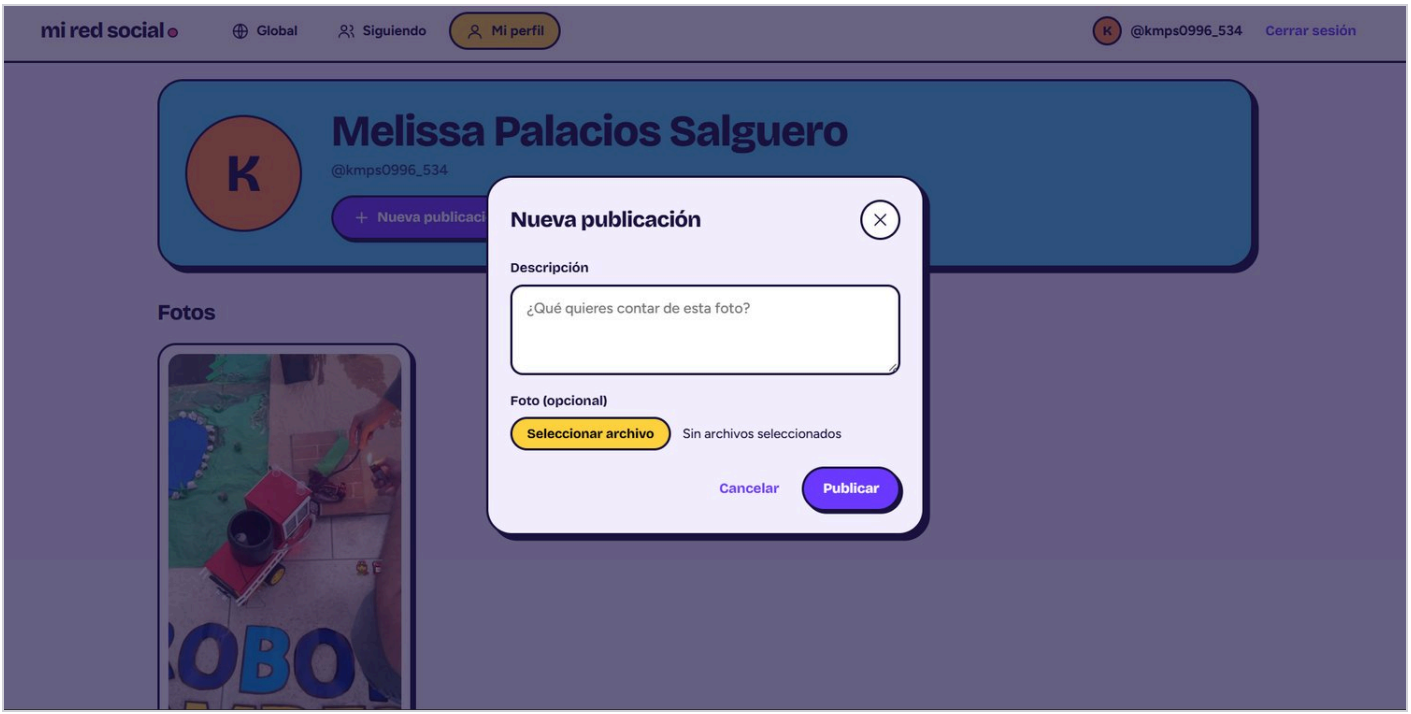


Figura 38. Ventana Nueva publicación: descripción y foto opcional.

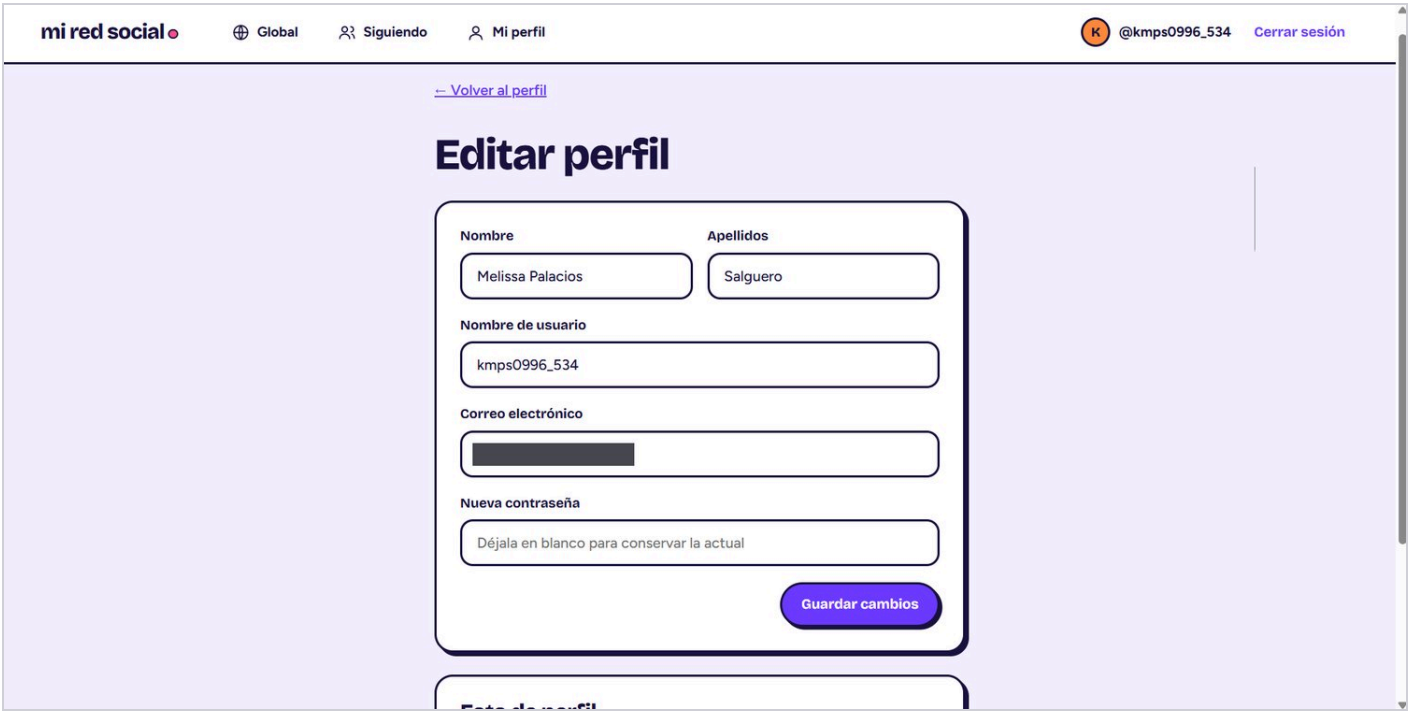


Figura 39. Editar perfil: nombre, usuario, correo y contraseña.

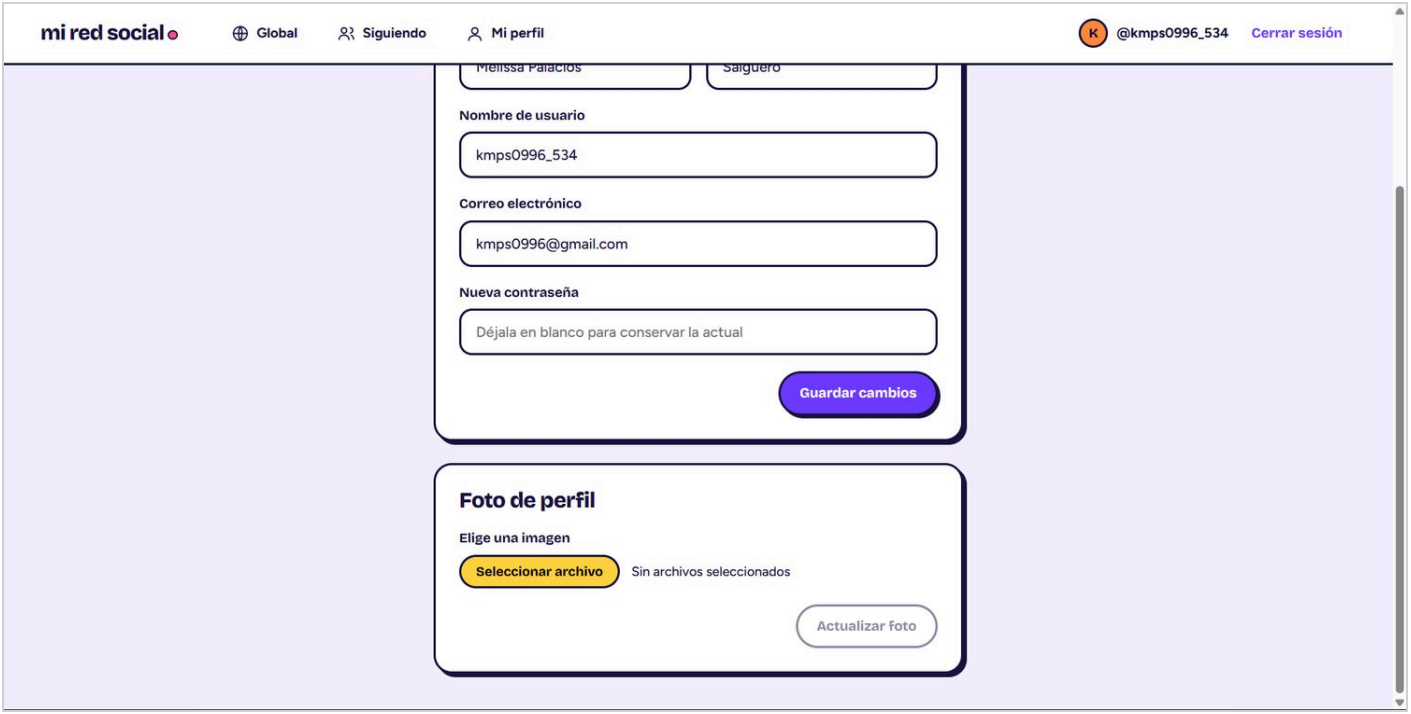


Figura 40. Editar perfil: cambio de foto de perfil.



Figura 41. Visor de la foto con las versiones (original, miniatura, blanco y negro, sepia), los 10 filtros y el botón de reconocimiento facial.

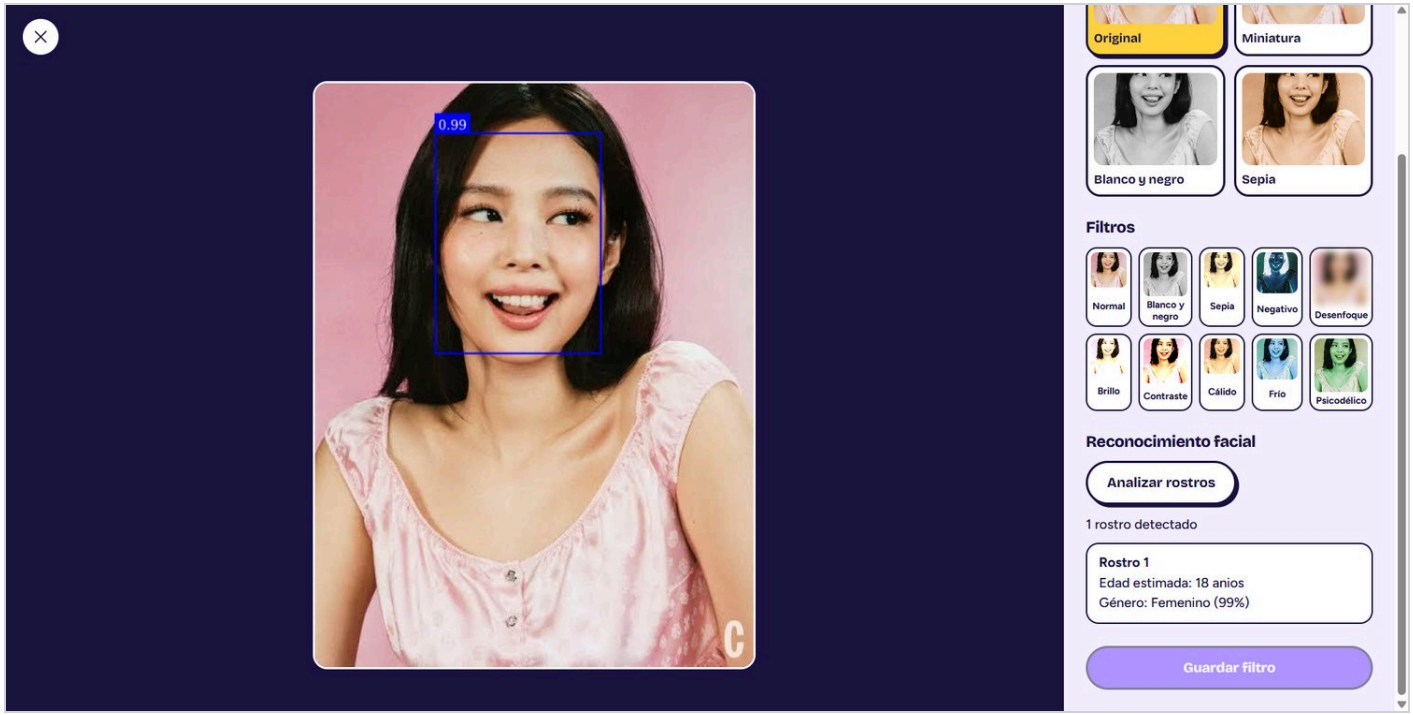


Figura 42. Resultado del reconocimiento facial: rostro detectado con edad y género estimados.

G. HTTPS y repositorio

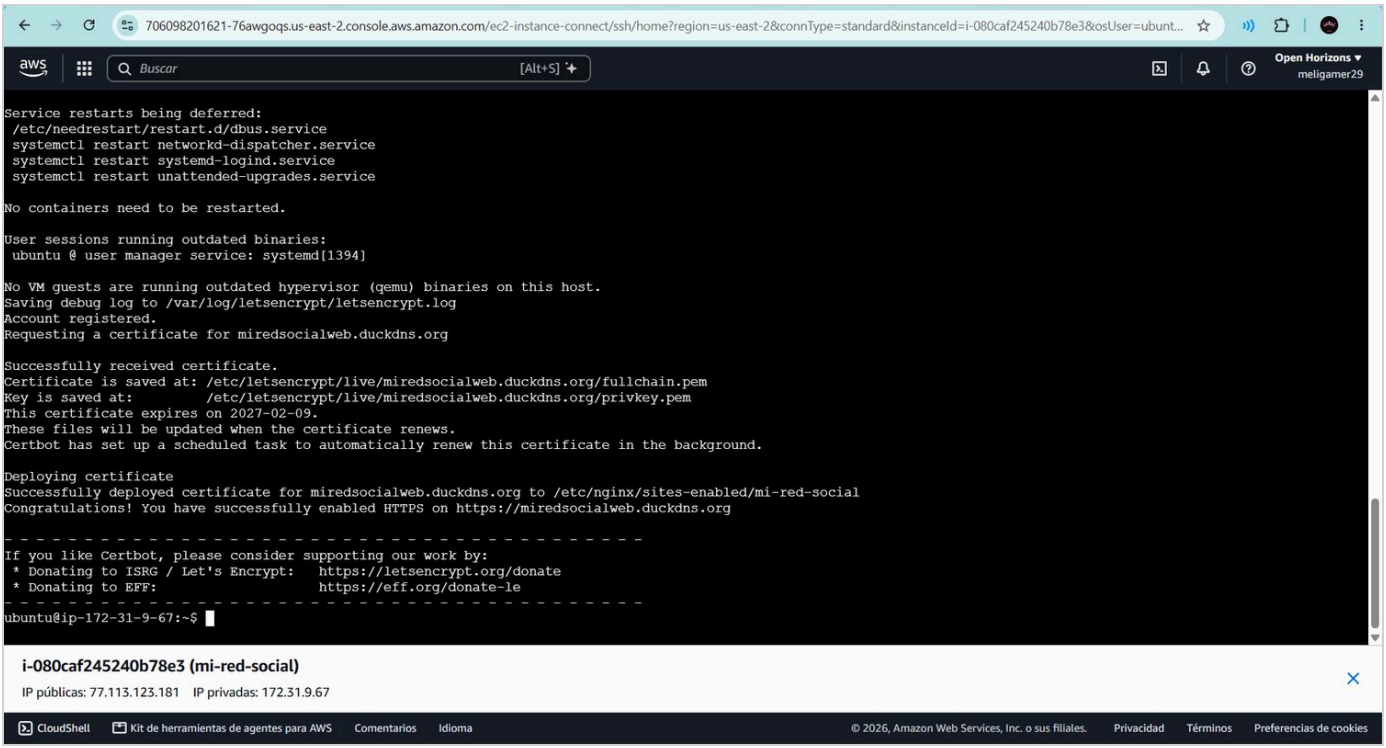


Figura 43. Certificado de Let's Encrypt emitido para miredsocialweb.duckdns.org; HTTPS activo.

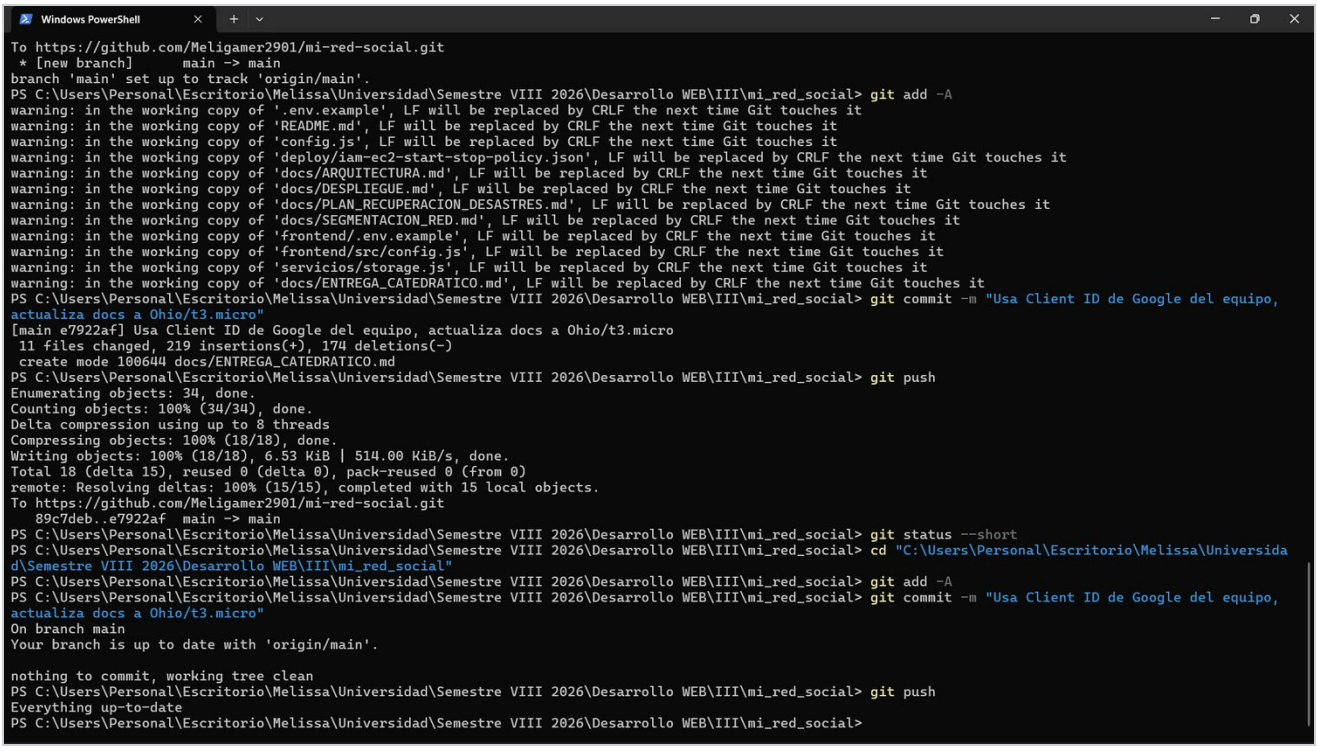


Figura 44. Cambios del proyecto subidos al repositorio de GitHub.

Manual de seguridad de la base de datos

Proyecto Final de Desarrollo Web (UMG) — Red social de fotografías. Motor: **PostgreSQL 14+** · Acceso desde la app: **Sequelize** (`database/conexion.js`).

Este manual distingue entre lo que **ya está implementado** en el proyecto y lo que es una **recomendación** para producción.

1. Qué se protege

Dato	Tabla	Sensibilidad
Correo, nombre y apellido	`users`	Datos personales
Contraseña	`users.password`	Crítica (solo se guarda el hash)
Contenido, likes, seguidores	`posts`, `followers`	Media
Ubicación de imágenes y metadatos faciales	`image_variants`, `face_detections`	Media (edad y género estimados)

Las imágenes **no** viven en la base de datos: solo su ruta o *key* de S3.

2. Estado actual de los controles

Control	Estado	Dónde
Contraseñas con hash bcrypt y sal única por usuario	Implementado	`Controlador/user.js` (`bcrypt.hash(password, 10)`)
Consultas parametrizadas (sin concatenar SQL)	Implementado	Sequelize ORM en todos los modelos
Restricciones de integridad (`UNIQUE`, `CHECK`, `FOREIGN KEY ... ON DELETE CASCADE`)	Implementado	`database/schema.sql`
Credenciales de la BD fuera del código	Implementado	Variables `DB_*` en `.env` (ignorado por git)
Base de datos no expuesta a internet	Implementado	PostgreSQL escucha solo en `localhost` (valor por defecto en Ubuntu) y el Security Group no abre el puerto 5432
Conexión cifrada (SSL)	Opcional	`DB_SSL=true` en `.env`
Usuario de BD con privilegios mínimos	Parcial	El script de despliegue crea un usuario propietario de la BD, no superusuario
Límite de intentos de login	**No implementado**	Ver sección 8
Validación estricta de entradas	**Parcial**	La librería `validator` está instalada pero ningún archivo la usa todavía

3. Control de acceso

Roles. Se usan dos: el superusuario `postgres` (solo administración, por consola local con `sudo -u postgres psql`) y el usuario de la aplicación (`DB_USER`), propietario de la base `mi_red_social`. La aplicación nunca debe conectarse como `postgres` en producción.

Endurecer el usuario de la aplicación:

```
ALTER ROLE app_user NOSUPERUSER NOCREATEDB NOCREATEROLE;
REVOKE CONNECT ON DATABASE mi_red_social FROM PUBLIC;
GRANT CONNECT ON DATABASE mi_red_social TO app_user;
```

Acceso por red. Dejar `listen_addresses = 'localhost'` en `postgresql.conf` y, en `pg_hba.conf`, aceptar solo conexiones locales con contraseña cifrada:

```
host  mi_red_social  app_user  127.0.0.1/32  scram-sha-256
```

Si en el futuro la BD pasa a Amazon RDS, el Security Group de la base debe aceptar el puerto 5432 **únicamente desde el Security Group de la instancia EC2** (ver `SEGMENTACION_RED.md`).

4. Protección de las contraseñas de usuario

- Se guarda únicamente el hash de **bcrypt** (factor de costo 10). El hash incluye una **sal aleatoria única** por usuario, que cumple el requisito del enunciado.
- La contraseña en texto plano nunca se guarda ni se registra en logs.

- Los usuarios que entran con Google reciben una contraseña aleatoria con hash, para que la columna nunca quede vacía.
- Para cambiar el costo en el futuro basta subir el número; los hashes viejos siguen siendo válidos.

5. Protección de la sesión

- Las sesiones usan **JWT** firmado con `JWT_SECRET` (`servicios/jwt.js`) y expiran a los 30 días. El middleware `middlewares/auth.js` rechaza peticiones sin token, con token inválido o vencido.
- `JWT_SECRET` debe ser una cadena aleatoria larga. Si falta, el código usa una clave de desarrollo y, en producción, imprime una advertencia: **nunca desplegar así**.

6. Cifrado

Momento	Medida
En tránsito (navegador a nginx)	HTTPS con Certbot / Let's Encrypt (ver `docs/DESPLIEGUE.md`, paso 6)
En tránsito (app a PostgreSQL)	Mismo servidor: `localhost`. Con RDS: `DB_SSL=true`
En reposo (disco)	Cifrar el volumen EBS al crear la instancia (casilla "Encrypted")
En reposo (S3)	Cifrado del lado del servidor (SSE-S3) activado por defecto en el bucket

7. Gestión de secretos

- `.env` está en `.gitignore`: **nunca** se sube a GitHub.
- Los secretos del proyecto: `DB_PASSWORD`, `JWT_SECRET` y `RECAPTCHA_SECRET_KEY`.
- La aplicación accede a S3 mediante un **rol IAM asociado a la instancia**: no hay claves de AWS guardadas en el servidor ni en el repositorio.
- El usuario IAM del catedrático tiene una clave de acceso con permisos mínimos (iniciar y detener solo esta instancia); se entrega por canal privado y se desactiva después de la revisión.
- Entregar al catedrático los secretos por un canal seguro, no por el repositorio.
- Rotación: si una clave se filtra, cambiarla en `.env`, ejecutar `pm2 restart mi-red-social` y, en el caso de `JWT_SECRET`, tener en cuenta que invalida todas las sesiones abiertas.

8. Brechas conocidas y mejoras recomendadas

Brecha	Riesgo	Mejora
`CORS_ORIGIN=*` por defecto	Cualquier sitio puede llamar a la API	En producción fijar el dominio real, p. ej. `https://miredsocial.com`
Sin límite de intentos de login	Fuerza bruta contra contraseñas	Añadir `express-rate-limit` en `/login/user` y `/registrar/user` (el reCAPTCHA ayuda, pero no sustituye al límite)
Sin cabeceras de seguridad HTTP	Clickjacking, sniffing de tipo MIME	Añadir `helmet` en `index.js`
JWT de 30 días sin revocación	Un token robado sirve hasta vencer	Reducir la vigencia (p. ej. 24 h)
Token guardado en `localStorage`	Lo puede leer un script inyectado	Mantener el escape de texto de React y, si crece el proyecto, pasar a cookie `HttpOnly`
`DB_SSL` con `rejectUnauthorized: false`	Acepta certificados no verificados	Con RDS, usar el certificado raíz de AWS y activar la verificación
`sequelize.sync({ alter: true })` en desarrollo	Cambios de esquema sin control	En producción usar migraciones (ya se desactiva `alter` con `NODE_ENV=production`)

9. Lista de verificación antes de entregar

- ☐ `JWT_SECRET` y `DB_PASSWORD` son valores aleatorios, no los de ejemplo
- ☐ El puerto 5432 **no** aparece en el Security Group
- ☐ `.env` no está en el repositorio (`git ls-files | grep .env` no debe mostrarlo)
- ☐ `CORS_ORIGIN` apunta al dominio real
- ☐ El volumen EBS y el bucket S3 tienen cifrado activado
- ☐ Hay un respaldo reciente y se probó restaurarlo (ver `PLAN_RECUPERACION_DESASTRES.md`)

Plan de mantenimiento de la base de datos

Proyecto Final de Desarrollo Web (UMG) — Red social de fotografías. Motor: **PostgreSQL 14+** · Tablas: `users`, `posts`, `followers`, `image_variants`, `face_detections`.

Un mantenimiento regular evita tres problemas: que la base crezca sin control, que las consultas se vuelvan lentas y que un respaldo no sirva cuando se necesita.

1. Calendario de tareas

Frecuencia	Tarea	Responsable
Diaria (automática)	Respaldo con `deploy/backup-db.sh` (cron, 02:00)	Infraestructura
Diaria	Revisar que el respaldo del día aparezca en S3 y que `backup.log` no tenga errores	Infraestructura
Semanal	Revisar espacio en disco, tamaño de tablas y conexiones (sección 3)	Base de datos
Semanal	Revisar `pm2 logs` y `logs/error.log` en busca de errores de BD	Aplicación
Mensual	Buscar y limpiar registros huérfanos (sección 4)	Base de datos
Mensual	Aplicar parches de seguridad: `sudo apt update && sudo apt upgrade`	Infraestructura
Mensual	Revisar índices que no se usan y consultas lentas	Base de datos
Trimestral	**Probar la restauración** de un respaldo en una base de prueba	Base de datos
Antes de cambios de esquema	Respaldo manual + snapshot/AMI	Quien haga el cambio

2. Vacuum y estadísticas

PostgreSQL ejecuta **autovacuum** por sí solo, y para este proyecto basta. Se comprueba así:

```
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC;
```

Si una tabla tiene muchos `n_dead_tup` y `last_autovacuum` está vacío o es muy viejo, forzarlo:

```
VACUUM (ANALYZE, VERBOSE) posts;
```

Reconstruir un índice solo si hay degradación evidente (bloquea escrituras mientras corre):

```
REINDEX TABLE posts;
```

3. Monitoreo semanal

```
df -h /                      # espacio libre en disco (alertar con menos de 20 %)
sudo systemctl status postgresql
```

```
-- Tamaño total de la base
SELECT pg_size_pretty(pg_database_size('mi_red_social'));

-- Tablas más grandes
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) AS tamano
FROM pg_catalog.pg_statio_user_tables ORDER BY pg_total_relation_size(relid) DESC;

-- Conexiones abiertas
SELECT count(*) FROM pg_stat_activity WHERE datname = 'mi_red_social';

-- Consultas que llevan más de 1 minuto
SELECT pid, now() - query_start AS duracion, left(query, 80) AS consulta
FROM pg_stat_activity
WHERE state = 'active' AND now() - query_start > interval '1 minute';
```

Las imágenes no ocupan espacio en la base (viven en S3); lo que más crece es `image_variants` (4 filas por foto) y `face_detections`.

4. Limpieza de registros huérfanos

`image_variants.owner_id` y `face_detections.owner_id` son referencias **lógicas**: no tienen clave foránea física, porque apuntan a `users` o a `posts` según `owner_type`.

- Al **borrar una publicación desde la app**, el backend elimina sus variantes, sus detecciones faciales y los archivos de almacenamiento (`Controlador/publicacion.js`).
- Si se borra un **usuario** (sus posts se eliminan por `ON DELETE CASCADE`) o se borran filas **directamente en la base**, pueden quedar filas y archivos huérfanos.

Buscarlos:

```
-- Variantes de publicaciones que ya no existen
SELECT v.id, v.storage_key
FROM image_variants v LEFT JOIN posts p ON v.owner_id = p.id
WHERE v.owner_type = 'post' AND p.id IS NULL;

-- Variantes de usuarios que ya no existen
SELECT v.id, v.storage_key
FROM image_variants v LEFT JOIN users u ON v.owner_id = u.id
WHERE v.owner_type = 'user' AND u.id IS NULL;

-- Detecciones faciales huérfanas
SELECT f.id FROM face_detections f LEFT JOIN posts p ON f.owner_id = p.id
WHERE f.owner_type = 'post' AND p.id IS NULL;
```

Procedimiento: (1) guardar la lista de `storage_key`; (2) borrar esos archivos de S3 (o de `imagenes/` si el modo es local); (3) borrar las filas; (4) anotarlo en la bitácora. Hacer un respaldo antes de borrar.

5. Índices

Los índices del esquema ya cubren el feed (`posts(usuario)`, `posts(fecha DESC)`, `image_variants(owner_type, owner_id)`). Para ver cuáles se usan:

```
SELECT relname AS tabla, indexrelname AS indice, idx_scan AS usos
FROM pg_stat_user_indexes ORDER BY idx_scan ASC;
```

Un índice con cero usos durante meses es candidato a eliminarse. Las claves primarias y las restricciones `UNIQUE` **no** se tocan.

6. Cambios de esquema y actualizaciones

- En desarrollo la app usa `sequelize.sync({ alter: true })`; en producción (`NODE_ENV=production`) solo crea tablas faltantes y no altera las existentes.
- Para cambios en producción: respaldo manual, ejecutar un script SQL versionado y anotarlo en la bitácora. Si el proyecto crece, migrar a `sequelize-cli db:migrate`.
- **Actualizaciones menores** de PostgreSQL (14.x a 14.y) llegan con `apt upgrade`; reiniciar fuera de horas de uso y comprobar `pm2 status` después.
- **Actualización mayor** (p. ej. 14 a 16): probarla antes en una instancia nueva con una copia restaurada de un respaldo, nunca directamente en producción.

7. Bitácora de mantenimiento

Cada intervención se anota con esta plantilla:

Fecha	Tarea	Responsable	Resultado	Observaciones
AAAA-MM-DD	Prueba de restauración trimestral			Tiempo que tomó:

8. Lista de verificación mensual

- ☐ Hay un respaldo de cada día del último mes en S3
- ☐ Se probó restaurar uno (trimestral)
- ☐ Espacio en disco por encima del 20 % libre
- ☐ Sin registros huérfanos pendientes de limpiar
- ☐ Parches de seguridad aplicados y `pm2 status` en `online`
- ☐ Bitácora actualizada

Plan de recuperación contra desastres (Disaster Recovery Plan)

Proyecto Final de Desarrollo Web (UMG) — Red social de fotografías.

Este documento cubre tanto el *plan de recuperación contra desastres* como el *Disaster Recovery Plan* que menciona el enunciado: es el mismo plan.

1. Objetivo y alcance

Restablecer el servicio (sitio, base de datos e imágenes) después de una falla, con la menor pérdida de datos y el menor tiempo posible. Cubre la instancia EC2, PostgreSQL, el bucket S3, el DNS en DuckDNS y el código en GitHub.

2. Inventario de activos y criticidad

Activo	Dónde vive	Criticidad	Cómo se protege
Código fuente	GitHub (ramas por integrante)	Alta	Repositorio remoto
Base de datos (usuarios, posts, seguidores)	PostgreSQL en la EC2	**Crítica**	`pg_dump` diario a S3 + snapshot EBS
Imágenes (originales y transformaciones)	Bucket S3	**Crítica**	Versionado de S3
Servidor (Node, nginx, PM2)	Instancia EC2	Alta	AMI + script `deploy/ec2-bootstrap.sh`
Secretos (`.env`)	Solo en la instancia	**Crítica**	Copia cifrada fuera del repositorio
DNS	DuckDNS	Media	Registros documentados

3. Objetivos de recuperación

Componente	RPO (pérdida de datos aceptable)	RTO (tiempo para volver)
Código	0 (está en GitHub)	30 minutos
Base de datos	24 horas (respaldo diario)	2 horas
Imágenes	Casi 0 (S3 con versionado)	1 hora
Servidor completo	—	2 a 3 horas (desde AMI o desde cero con el script)

4. Estrategia de respaldo

4.1 Base de datos: `pg_dump` diario a S3. Usa `deploy/backup-db.sh`. Pasos de una sola vez:

1. Crear un bucket S3 **privado** solo para respaldos, con una regla de ciclo de vida que borre los archivos a los 30 días.
2. Instalar el CLI: `sudo snap install aws-cli --classic`.
3. Dar a la instancia permiso `s3:PutObject` sobre ese bucket (rol IAM).
4. Programar con `crontab -e`:

```
0 2 * * * BUCKET_RESPALDOS=<bucket> /home/ubuntu/mi_red_social/deploy/backup-db.sh >>
/home/ubuntu/backup.log 2>&1
```

5. Comprobar al día siguiente que aparece el archivo en `s3://<bucket>/postgres/`.

4.2 Servidor: snapshot y AMI. Antes de la entrega y antes de cambios grandes, crear una AMI de la instancia (EC2, Actions, Image and templates, Create image). Opcionalmente programar snapshots semanales del volumen con *Amazon Data Lifecycle Manager*.

4.3 Imágenes: versionado de S3. Activar *Bucket Versioning* en el bucket de imágenes (S3, Properties, Bucket Versioning). Así un archivo borrado o sobrescrito se puede recuperar.

4.4 Código. Todo cambio va a GitHub; cada integrante trabaja en su rama.

4.5 Secretos. Guardar una copia del `.env` en un gestor de contraseñas del equipo o en AWS Systems Manager Parameter Store. **Nunca** en el repositorio.

5. Escenarios y procedimientos

5.1 La aplicación se cae

```
pm2 status
pm2 logs mi-red-social --lines 100
pm2 restart mi-red-social
```

PM2 reinicia el proceso solo si falla y, con `pm2 startup`, también al reiniciar la instancia.

5.2 Falla la instancia o el disco

1. En EC2, lanzar una instancia nueva **desde la última AMI** (o Ubuntu 22.04 limpio).
2. Si es limpio: clonar el repositorio y ejecutar `deploy/ec2-bootstrap.sh`, restaurar el `.env` y luego la base de datos (sección 5.3).
3. Asociar la **Elastic IP** a la instancia nueva. Si no se usó Elastic IP, actualizar el registro A en DuckDNS con la IP nueva.
4. Verificar con la lista de la sección 7.

5.3 Base de datos dañada o borrada

```
# 1. Traer el respaldo más reciente
aws s3 ls s3://<bucket>/postgres/
aws s3 cp s3://<bucket>/postgres/<archivo>.dump /tmp/

# 2. Detener la app para que nadie escriba durante la restauración
pm2 stop mi-red-social

# 3. Recrear la base (solo si estaba dañada) y restaurar
sudo -u postgres dropdb --if-exists mi_red_social
sudo -u postgres createdb -O <DB_USER> mi_red_social
PGPASSWORD=<DB_PASSWORD> pg_restore -h localhost -U <DB_USER> -d mi_red_social --no-owner
/tmp/<archivo>.dump

# 4. Levantar la app
pm2 start mi-red-social
```

Se pierden como máximo las últimas 24 horas de actividad (RPO).

5.4 Se borraron imágenes del bucket

Con el versionado activo: S3, activar *Show versions*, abrir el archivo y restaurar la versión anterior (o borrar el marcador de eliminación). Las filas de `image_variants` no cambian, porque solo guardan la ruta.

5.5 Se filtran credenciales

Credencial	Acción
Llaves IAM de S3	Desactivarlas en IAM, crear nuevas, actualizar `AWS_*` en `.env`, `pm2 restart`
`JWT_SECRET`	Generar una nueva y reiniciar; esto cierra todas las sesiones abiertas
`DB_PASSWORD`	`ALTER USER <DB_USER> PASSWORD '...'`, actualizar `.env`, reiniciar
`RECAPTCHA_SECRET_KEY`	Regenerar en el panel de reCAPTCHA y actualizar `.env`
Usuario IAM del catedrático	Desactivarlo y crear otro

5.6 La región us-east-2 no está disponible

Es poco probable, pero el plan es reconstruir en otra región: crear el bucket de imágenes nuevo (o activar replicación entre regiones por adelantado), lanzar una instancia con el script de despliegue y restaurar el último `pg_dump`. El RTO sube a medio día.

6. Roles durante un incidente

Rol	Responsabilidad
Responsable de infraestructura	Ejecuta la recuperación (AWS, instancia, DNS)
Responsable de base de datos	Restaura y valida los datos
Responsable de la aplicación	Verifica que el sitio funcione y revisa los logs
Coordinador	Comunica el estado al equipo y al catedrático y registra lo ocurrido

Cada rol debe tener un titular y un suplente del equipo.

7. Verificación después de recuperar

- [ ] `http://<dominio>/` responde "API de red social funcionando"
- [ ] Se puede iniciar sesión con un usuario existente
- [ ] El feed muestra publicaciones y las fotos cargan
- [ ] Subir una foto de prueba genera sus 3 transformaciones
- [ ] `pm2 status` muestra `online` y `pm2 save` está hecho
- [ ] Se anotó el incidente (qué pasó, cuándo, cuánto duró, qué se hizo)

8. Pruebas del plan

Un plan que no se prueba no sirve. Cada trimestre, y antes de la entrega:

1. Restaurar el último respaldo en una base de prueba (`createdb mi_red_social_prueba`) y comprobar que las tablas y los conteos coinciden.
2. Lanzar una instancia desde la AMI y confirmar que arranca.
3. Anotar el tiempo que tomó y compararlo con el RTO.

Sistema de auditoría de transacciones

Proyecto Final de Desarrollo Web (UMG) — Red social de fotografías.

Una **auditoría** responde a: *¿quién hizo qué, cuándo y desde dónde?* Este documento describe qué evidencia deja hoy la aplicación y propone una tabla de auditoría para cerrar las brechas.

1. Transacciones de la aplicación

Transacción	Endpoint	Tablas que cambian
Registro de usuario	`POST /api/registrar/user`	`users`
Inicio de sesión	`POST /api/login/user`, `POST /api/login/google`	(ninguna; crea un token)
Actualizar perfil / contraseña	`PUT /api/actualizar/user`	`users`
Cambiar foto de perfil	`PUT /api/upload/user`	`users` (y el archivo de imagen)
Crear publicación	`POST /api/crear/publicacion`	`posts`
Subir imagen	`POST /api/subir-imagen/:id`	`posts`, `image_variants`, `face_detections`
Editar / borrar publicación	`PUT` / `DELETE` sobre `publicacion`	`posts`
Me gusta	`PUT /api/likes/publicacion`	`posts`
Seguir / dejar de seguir	`POST` / `DELETE /api/follow/...`	`followers`
Aplicar filtro a una foto	`POST /api/imagen/transformar`	`image_variants`, `posts` o `users`

2. Evidencia que existe hoy

Fuente	Qué registra	Dónde
Columnas de fecha	Cuándo se creó cada registro	`users.created_at`, `posts.fecha`, `followers.fecha_follow`, `image_variants.created_at`, `face_detections.created_at`
Logs de PM2	Salida y errores de la app, con fecha y hora	`logs/out.log`, `logs/error.log` (`time: true` en `ecosystem.config.js`)
Logs de acceso de nginx	IP, método, ruta, código de respuesta y hora de cada petición	`/var/log/nginx/access.log`
AWS CloudTrail	Acciones sobre la cuenta de AWS (crear/detener instancias, cambios de IAM, S3)	Consola de AWS
Restricciones de la BD	Bloquean duplicados y auto-seguimiento a nivel de base de datos	`UNIQUE`, `CHECK` en `database/schema.sql`

Limitaciones actuales

- La app **no guarda quién ejecutó cada acción** de forma estructurada: hay que cruzar logs de nginx con tablas.
- Los **borrados** (publicaciones, seguidores) no dejan rastro.
- Los cambios de perfil no guardan el valor anterior.
- Los logs de PM2 mezclan mensajes técnicos con eventos de negocio.

3. Qué debe registrarse

Evento	Importancia	Detalle a guardar
Inicio de sesión correcto y fallido	Alta	Usuario o correo intentado, IP, método (clásico o Google)
Registro de usuario	Alta	Usuario creado, IP

Evento	Importancia	Detalle a guardar
Cambio de contraseña o correo	Alta	Usuario, qué campo cambió (**nunca el valor**)
Crear / editar / borrar publicación	Media	Usuario, id del post
Subir imagen o aplicar filtro	Media	Usuario, id del post, filtro
Seguir / dejar de seguir	Baja	Usuario origen y destino
Acceso denegado (token inválido o vencido)	Alta	IP, ruta

Nunca guardar contraseñas, tokens JWT ni el contenido de los archivos en el registro.

4. Propuesta: tabla `audit_log`

```
CREATE TABLE IF NOT EXISTS audit_log (
    id           BIGSERIAL PRIMARY KEY,
    ocurrio_en   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    user_id      INTEGER,                -- NULL si no hay sesión (login fallido)
    accion       VARCHAR(60)  NOT NULL,  -- p. ej. 'LOGIN_OK', 'POST_BORRADO'
    metodo       VARCHAR(10)  NOT NULL,  -- GET, POST, PUT, DELETE
    ruta         VARCHAR(255) NOT NULL,
    status_code  INTEGER      NOT NULL,
    ip           VARCHAR(45),
    detalle      JSONB DEFAULT '{}'
);
CREATE INDEX IF NOT EXISTS idx_audit_usuario ON audit_log(user_id, ocurrio_en DESC);
CREATE INDEX IF NOT EXISTS idx_audit_accion  ON audit_log(accion, ocurrio_en DESC);
```

`user_id` no es una clave foránea a propósito: el registro debe sobrevivir aunque el usuario se borre.

Middleware de auditoría (borrador)

```
// middlewares/auditoria.js  (propuesta)
const { sequelize } = require("../database/conexion");

// Uso en rutas:  router.delete("/borrar/publicacion/:id", auth.auth, auditar("POST_BORRADO"), ...)
module.exports = (accion) => (req, res, next) => {
    res.on("finish", () => {
        sequelize.query(
            `INSERT INTO audit_log (user_id, accion, metodo, ruta, status_code, ip, detalle)
             VALUES (:userId, :accion, :metodo, :ruta, :status, :ip, :detalle)`,
            {
                replacements: {
                    userId: req.user ? req.user.id : null,
                    accion,
                    metodo: req.method,
                    ruta: req.originalUrl,
                    status: res.statusCode,
                    ip: req.ip,
                    detalle: JSON.stringify({ params: req.params })
                }
            }
        ).catch((e) => console.error("No se pudo escribir la auditoría:", e.message));
    });
    next();
};
```

Dos detalles al implementarlo:

- Detrás de nginx, activar `app.set("trust proxy", true)` en `index.js`; si no, `req.ip` mostraría siempre `127.0.0.1`. nginx ya envía `X-Forwarded-For`.
- El registro se escribe al terminar la respuesta (`finish`), así un fallo de auditoría no rompe la petición del usuario.

5. Protección del registro de auditoría

- El usuario de la aplicación solo debe poder **insertar y leer**, nunca modificar o borrar:

```
REVOKE UPDATE, DELETE, TRUNCATE ON audit_log FROM app_user;
```

- Retención: 12 meses en la base; luego exportar a S3 y borrar de la base.
- Revisar semanalmente los inicios de sesión fallidos y los accesos denegados.

6. Consultas de ejemplo

```
-- Últimas acciones de un usuario
SELECT ocurrio_en, accion, ruta, status_code FROM audit_log
WHERE user_id = 5 ORDER BY ocurrio_en DESC LIMIT 50;

-- IPs con más inicios de sesión fallidos en las últimas 24 h
SELECT ip, COUNT(*) AS intentos FROM audit_log
WHERE accion = 'LOGIN_FALLIDO' AND ocurrio_en > NOW() - INTERVAL '24 hours'
GROUP BY ip ORDER BY intentos DESC;

-- Publicaciones borradas esta semana
SELECT ocurrio_en, user_id, detalle FROM audit_log
WHERE accion = 'POST_BORRADO' AND ocurrio_en > NOW() - INTERVAL '7 days';
```

7. Alternativa: auditoría dentro de PostgreSQL

La extensión **pgaudit** registra las sentencias SQL a nivel de motor, sin tocar el código de la app. Es más completa, pero está disponible en Amazon RDS y requiere instalación extra en una instancia propia; para este proyecto se recomienda la tabla `audit_log`.

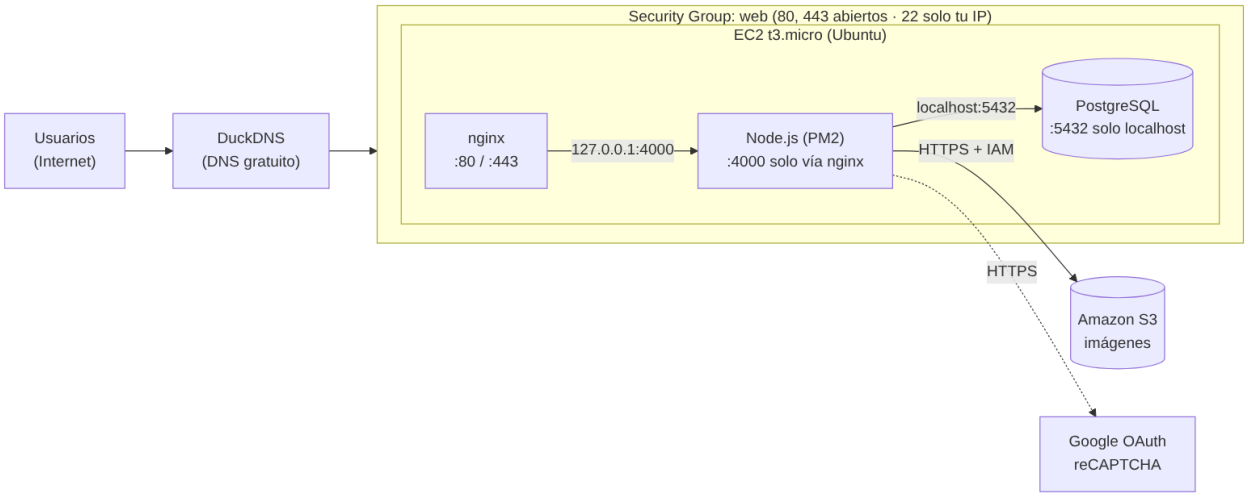
Manual de segmentación de red

Proyecto Final de Desarrollo Web (UMG) — Red social de fotografías.

Describe cómo se separan los componentes en la red: primero la **arquitectura actual** (una sola instancia EC2, la que pide el enunciado) y luego la **arquitectura recomendada** para un entorno real.

1. Arquitectura actual (despliegue de la entrega)

Todo corre en una instancia **EC2 t3.micro** en Ohio (us-east-2), protegida por un Security Group. La separación entre componentes es **lógica** (procesos y puertos), no física.



Reglas del Security Group

Dirección	Tipo	Puerto	Origen / destino	Motivo
Entrada	SSH	22	**Solo la IP del administrador**	Administración remota
Entrada	HTTP	80	0.0.0.0/0	Sitio web y desafío de Certbot
Entrada	HTTPS	443	0.0.0.0/0	Sitio web cifrado
Entrada	—	4000	**Cerrado**	La API solo se alcanza a través de nginx
Entrada	—	5432	**Cerrado**	PostgreSQL solo acepta conexiones locales
Salida	Todo	Todo	0.0.0.0/0	Paquetes, S3, Google

Cómo queda segmentado

Segmento	Qué contiene	Quién puede llegar
Público	nginx (80/443)	Cualquier usuario de internet
Aplicación	Node.js en el puerto 4000	Solo nginx, por `127.0.0.1`
Datos	PostgreSQL en el puerto 5432	Solo la app, por `localhost`
Almacenamiento	Bucket S3	La app, con una llave IAM de mínimo privilegio; lectura pública solo de las fotos
Administración	SSH	Solo la IP del administrador

2. Firewall del servidor (refuerzo opcional)

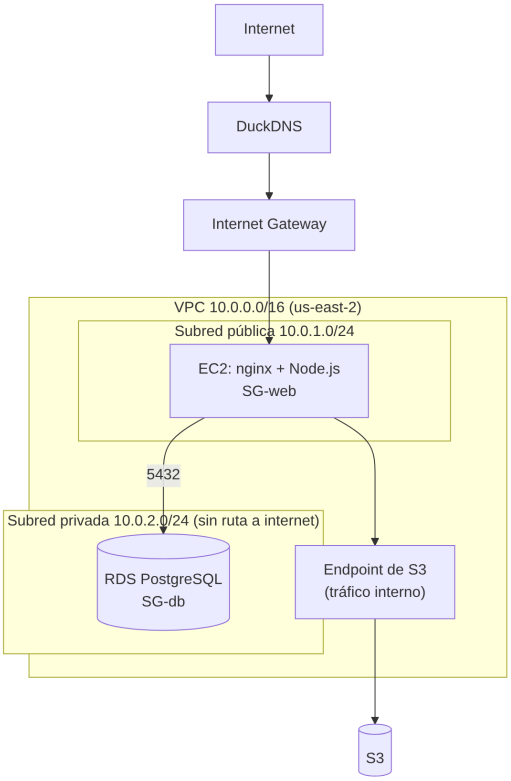
Además del Security Group, se puede activar el firewall del propio Ubuntu. **Permitir SSH antes de activarlo** o se pierde el acceso:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from <TU_IP> to any port 22 proto tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
```

```
sudo ufw enable
sudo ufw status verbose
```

3. Arquitectura recomendada para producción

La mejora principal es **sacar la base de datos de la instancia web** y ponerla en una subred privada, sin ruta a internet.



Grupo	Entrada permitida	Salida
SG-web	80 y 443 desde internet · 22 solo desde la IP del administrador	Todo
SG-db	5432 **únicamente desde SG-web**	Ninguna necesaria

Mejoras adicionales:

- **Acceso administrativo sin SSH abierto**: usar AWS Systems Manager Session Manager.
- **Listas de control de red (NACL)** en la subred privada: denegar todo lo que no venga de la subred pública.
- **Endpoint de S3 (gateway)**: el tráfico a S3 no sale a internet.
- **AWS WAF** delante de nginx si la aplicación se vuelve pública de verdad.
- **HTTPS obligatorio**: Certbot con redirección de 80 a 443.

4. Cómo verificar la segmentación

Desde **otra máquina** (no desde la instancia):

```
nmap -Pn -p 22,80,443,4000,5432 <IP-publica-EC2>
```

Resultado esperado: 80 y 443 abiertos; 4000 y 5432 **filtrados/cerrados**; 22 abierto solo si se prueba desde la IP autorizada.

Desde la instancia:

```
sudo ss -tlnp | grep -E ':(80|443|4000|5432)\s'
```

Esperado: nginx en `0.0.0.0:80`, PostgreSQL en `127.0.0.1:5432`. Node escucha en el puerto 4000, pero el Security Group lo bloquea desde fuera.

Plan de capacitaciones para el equipo de IT

Objetivo: que cualquier persona de IT pueda operar, mantener y recuperar **Mi Red Social** sin depender de quienes la desarrollaron.

1. Perfiles y a quién va dirigido

Perfil	Qué necesita saber
Administrador de infraestructura	AWS (EC2, S3, IAM, Security Groups), nginx, certificados, costos.
Administrador de base de datos	PostgreSQL, respaldos y restauración, seguridad de datos.
Desarrollador de mantenimiento	Código, API, despliegue de cambios, registros.
Soporte / mesa de ayuda	Uso de la aplicación, problemas comunes, cuándo escalar.

2. Módulos de capacitación

#	Módulo	Contenido	Perfil	Duración	Documento de apoyo
1	Visión general	Qué hace la aplicación, componentes y flujo de una foto.	Todos	1 h	`ARQUITECTURA.md`, `DIAGRAMA_COMPONENTES.md`
2	Uso de la aplicación	Registro, login con Google, publicar, filtros, seguir.	Soporte	1 h	`MANUAL_USUARIO.md`
3	AWS básico del proyecto	Consola, región Ohio, EC2, Elastic IP, S3, roles IAM.	Infraestructura	2 h	`DIAGRAMAS_CLOUD.md`
4	Encender, apagar y desplegar	Start/stop de la instancia, `deploy/ec2-bootstrap.sh`, PM2, nginx.	Infraestructura, desarrollo	2 h	`DESPLIEGUE.md`, `MANUAL_TECNICO.md`
5	Red y seguridad	Security Group, puertos, HTTPS, secretos, roles de mínimo privilegio.	Infraestructura	1,5 h	`SEGMENTACION_RED.md`, `SEGURIDAD_BD.md`
6	Base de datos	Esquema, respaldos, restauración, mantenimiento.	Base de datos	2 h	`ESQUEMA_BD.md`, `MANTENIMIENTO_BD.md`
7	Recuperación ante desastres	Escenarios y procedimientos, simulacro.	Infraestructura, base de datos	2 h	`PLAN_RECUPERACION_DESASTRES.md`
8	Auditoría y monitoreo	Registros de PM2/nginx, qué se audita, consultas de ejemplo.	Desarrollo, infraestructura	1,5 h	`AUDITORIA_TRANSACCIONES.md`
9	Desarrollo y cambios	Estructura del código, API, ramas de Git, compilar el frontend.	Desarrollo	3 h	`MANUAL_TECNICO.md`

3. Calendario sugerido

Semana	Módulos	Modalidad
1	1, 2	Presentación y práctica con la aplicación
2	3, 4	Laboratorio guiado en la consola de AWS
3	5, 6	Taller de seguridad y respaldos
4	7, 8	Simulacro de recuperación

Semana	Módulos	Modalidad
5	9	Taller de desarrollo; evaluación final

4. Prácticas de laboratorio

1. Iniciar y detener la instancia con la CLI y con la consola.
2. Reiniciar la aplicación con PM2 y revisar sus registros.
3. Subir una foto y localizar las 4 versiones en S3 y sus filas en la base de datos.
4. Hacer un respaldo con `deploy/backup-db.sh` y restaurarlo en una base de prueba.
5. Simular la caída de la aplicación y recuperarla siguiendo el plan de recuperación.
6. Renovar el certificado HTTPS (`sudo certbot renew --dry-run`).
7. Rotar un secreto (`JWT_SECRET`) y comprobar sus efectos.

5. Evaluación

Evidencia	Criterio de aprobación
Prueba práctica de despliegue	Levanta la aplicación en una instancia nueva con el script, sin ayuda.
Simulacro de recuperación	Restaura la base y vuelve a servir la aplicación dentro del RTO del plan.
Cuestionario de seguridad	Identifica correctamente qué va en el `.env`, en el repositorio y en IAM.

6. Mantenimiento de la capacitación

- Revisar este plan cada trimestre o cuando cambie la arquitectura.
- Incorporar a toda persona nueva de IT durante su primera semana (módulos 1, 3 y 4 como mínimo).
- Registrar las lecciones aprendidas de cada simulacro y actualizar los documentos.

Entrega para el catedrático

Los secretos (clave de acceso y llave `.pem`) se entregan por un canal privado, no en este repositorio.

Datos de la instancia

Dato	Valor
Región	us-east-2 (Ohio)
ID de instancia	`i-080caf245240b78e3`
IP pública (Elastic IP)	`77.113.123.181`
Sitio web	`https://miredsocialweb.duckdns.org`
Usuario SSH	`ubuntu`
Estado al entregar	Detenida (Stopped)

Credenciales entregadas por canal privado

1. **Usuario IAM** `catedratico`: archivo `.csv` con Access key ID y Secret access key. Permisos: ver instancias e iniciar/detener solo esta instancia.
2. **Llave SSH** `mi-red-social-key.pem`.

Iniciar la instancia y revisar

```
aws configure            # pegar las claves; región us-east-2; formato json
aws ec2 start-instances --instance-ids i-080caf245240b78e3 --region us-east-2
aws ec2 describe-instances --instance-ids i-080caf245240b78e3 --region us-east-2 \
    --query "Reservations[0].Instances[0].State.Name"
```

Esperar 1-2 minutos y abrir `https://miredsocialweb.duckdns.org`. La aplicación arranca sola (PM2).

Conexión por terminal (opcional):

```
ssh -i mi-red-social-key.pem ubuntu@77.113.123.181
pm2 status
```

Detener la instancia al terminar

```
aws ec2 stop-instances --instance-ids i-080caf245240b78e3 --region us-east-2
```

Desviaciones del enunciado

- Región Ohio (us-east-2) en lugar de N. Virginia, por restricción del plan gratuito.
- Instancia t3.micro en lugar de t2.small (único tipo gratuito); se añadió swap de 2 GB.
- Acceso del catedrático por clave de acceso (CLI), ya que la consola del plan gratuito no permite contraseña de consola para usuarios IAM.
- DNS con DuckDNS (gratuito) en lugar de Route 53, con HTTPS de Let's Encrypt.
- El Login con Google usa el Client ID del equipo; los correos que lo prueben deben estar agregados como usuarios de prueba. El registro con correo y reCAPTCHA funciona siempre.